

Proxmox Storage DRS -- Internals

How the tool works, for whoever changes it

Bernd Zeimetz bernd@bzed.de

2026-09-19

Rendered from docs/internals/*.md, SHA-256 09ad9d681b956ea2c8ccb2d9325711d1a89fcdea299324aef46dc45a7228310f
(sha256sum --check docs/internals.pdf.sha256 in the repository must succeed).

Copyright © 2026 Bernd Zeimetz. Licensed under the GNU Affero General Public License, version 3 or later.

Contents

The pipeline, as specified and as built	6
Module layout, as it stands	7
Why config.py depends on forecast.py	9
Where to read next	9
How a YAML file becomes a validated Config	10
Resolution order	10
The two-stage validation	10
Secrets from the environment	11
What is deliberately <i>not</i> checked here	11
ResolvedConfig: what the caller actually gets	11
Persistent state: state.json	11
One dataclass tree, matching section 11.2's JSON exactly	11
Reading degrades; writing does not	11
Locking: flock() is the mechanism, the JSON lock field is a label	12
What today's wiring into cli.py actually changes	12
Cooldowns: read by topology.py and heuristic.py, not by this module	13
Locking and writing: apply's own responsibility	13
Deliberately not implemented in this pass	13
Forecasting: the protocol and its three implementations	14
The one rule, in one place	14
Why the upper bound, never the point estimate	14
storage_upper_bound(): section 10.1's per-disk sum, not a per-storage forecast	14
QuantileForecaster	15
SeasonalNaiveForecaster	15
HoltWintersForecaster	15
The section 10.2 backtest validation gate (phase 9)	15
build_forecaster()	16
The Prometheus client and verify-metrics	16
_SessionLike / _ResponseLike: why not requests.Session directly	16
_get(): one request path, three endpoint shapes	16
PromQL construction	16
Node-scoping: build_node_selector() / resolve_node_selector()	17
safe_range_step_seconds() / decimate_to_configured_step(): the gigapipe step >= range workaround	17
RANGE_QUERY_CHUNK_SECONDS / stitch_range_results(): chunking a wide query_range	18
verify_metrics(): six checks, six functions	19
What cli.py does with the report	20
Command dispatch, the mode-override rule, and why logs go to stderr	20
One parser, global options first	20
Command handlers: a dict, not a chain of ifs	20
--group: filtered once, right after build_topology()	21
The --mode escalation rule	21
Why logs go to stderr, not stdout	21
Levels, the mandatory floor, and why the handler is on root	21

JsonFormatter: what ends up in one log line	22
--manual: preferring man(1), falling back only when necessary	22
The Proxmox VE API client	22
Why proxmoxer	22
build_client(): the one place auth is assembled	23
PveClient: one method per section 3.5 endpoint	23
move_disk(): the one and only bytes/s -> KiB/s conversion	25
Building the disk/storage/group model	25
D means every disk this module places, pinned or not	25
One pass, in the order section 3.5 lists	25
The lock field comes from vm_config(), not a separate call	26
Pin priority: _pin_reason()	26
The pending-change pin	26
The per-disk cooldown pin	27
Sizes: content is authoritative, config is the fallback of last resort	27
U ^s e ^x t: everything not referenced	28
Section 11.4: /.../ storage patterns are expanded here, once	28
reserve.py: one (C4)/(C5) evaluator, shared	29
free_space resolution: soft_s/hard_s, resolved once, here	29
(C2) format eligibility: storage_type/allowed_formats	30
The load model	30
One function, one group, already-built input	30
Six queries, not six-times-groups queries	30
Coverage rejection excludes a disk from the group total, not just from its own ℓ_d	31
tpmstate0/unusedN are never rejected; efidisk0 is	31
T _g = 0 is “idle”, computed only from accepted disks	31
L _s /u _s are the <i>current</i> assignment, not a candidate one	31
compute_disk_load_series(): the same blend, as a time series	32
What loadmodel.py is, and is not, responsible for	32
Gating: deciding whether to act at all	32
One function, three gates, in the order section 6 lists them	32
last_load is real now, for show-load and plan	33
show-load’s gate line is diagnostic, not a real decision	33
Cooldowns are not this module’s job	33
The heuristic solver	34
evaluate_assignment() is the objective, kept separate from the search	34
Repair is unconditional, not weight-driven	34
Descend explores swaps, not only single moves	35
Descend also tries relocating a whole VM at once — a real gap, found by dogfooding	35
The storage cooldown excludes a destination, never a source	35
objective.spread_metric: two different quantities, not one rescaled	36
Proof this reproduces the plan, not just itself	36
w _v : kappa weighted by a VM’s own I/O, and D ^{big}	36
What this pass deliberately does not do	37
best_single_disk_alternative(): why explain says nothing moved	37

The MILP backends: CP-SAT and CBC	38
Both backends solve, then hand off to <code>heuristic.evaluate_assignment()</code>	38
Lexicographic, not big-M — and why that is the only mode implemented	38
CP-SAT: section 5.5’s exact integer scaling	38
CBC: “direct transcription”, deliberately not scaled	39
κw_v and D^{big} : the same weighting in both backends	40
$(C1)/(C3)/(C4)/(C5)$ are shared code, factored once per backend	40
<code>cli.py</code> ’s dispatch: auto cascades, an explicit backend still falls back	40
The storage cooldown: enforced in both stages, both backends (REVIEW.md S-04)	41
Deliberately not implemented in this pass	41
Executing a plan: <code>execute.py</code> and its <code>cli.py</code> wiring	42
<code>execute_plan()</code> re-validates every move against the live cluster, not the plan	42
The live transient check: provisioned, never allocated	43
Why “done” needs four conditions, not one	44
VM locks are an open set, waited out, never whitelisted	45
The <code>move_disk</code> task’s own flock: a retry as the fallback under <code>min_wipe_seconds</code>	45
The injectable Clock	46
Orphaned volumes: reported, never deleted	46
<code>confirm</code> ’s callback lives in <code>cli.py</code> , not here	46
What <code>execute_plan()</code> deliberately does not do	46
<code>cli.py</code> : one planning pipeline, shared by <code>plan</code> and <code>apply</code>	47
The payback gate: <code>apply</code> refuses on its own verdict (REVIEW.md S-02)	48
auto mode: <code>timewindow.py</code> and <code>_run_auto_group()</code> (phase 8)	48
Concurrent execution: <code>_execute_concurrent()</code> and its non-blocking helpers	49
Crash and two-instance recovery: <code>crashrecovery.py</code>	50
The two failure modes section 13 names, one mechanism	51
<code>parse_upid()</code> : PVE’s UPID grammar, confirmed live	51
<code>/cluster/tasks</code> ’ own convention	51
<code>reconcile_inflight()</code> : both halves	51
Writing before the crash: <code>execute.py</code> ’s callbacks	52
<code>cli.py</code> : the startup scan, before planning anything	52
Ordering the moves	53
One loop, reusing <code>heuristic.py</code> ’s objective for the ratio	53
<code>cost_m</code> is <code>z_d</code> — except below <code>tiny_disk_bytes</code> , where it is 0	53
The transient invariant, called with a single-move set always	53
The heuristic can accept a residual violation; the scheduler cannot execute one	54
<code>final_assignment</code> is the schedule’s own truth, not the heuristic’s target	54
What this pass deliberately does not do	55
Wired into <code>plan</code> , not yet into <code>apply</code>	55
Migration cost and the payback test	55
Cost and benefit are computed from data other modules already have	55
<code>headroom_src/headroom_dst</code> : a plan formula this project cannot fill in	56
The reserve-override exemption	56
The section 7.3 saturation guard: <code>compute_move_cost()</code> ’s optional target	57
What this pass deliberately does not do	58
Wired into <code>plan</code> and <code>apply</code> alike, via <code>_plan_group()</code>	59

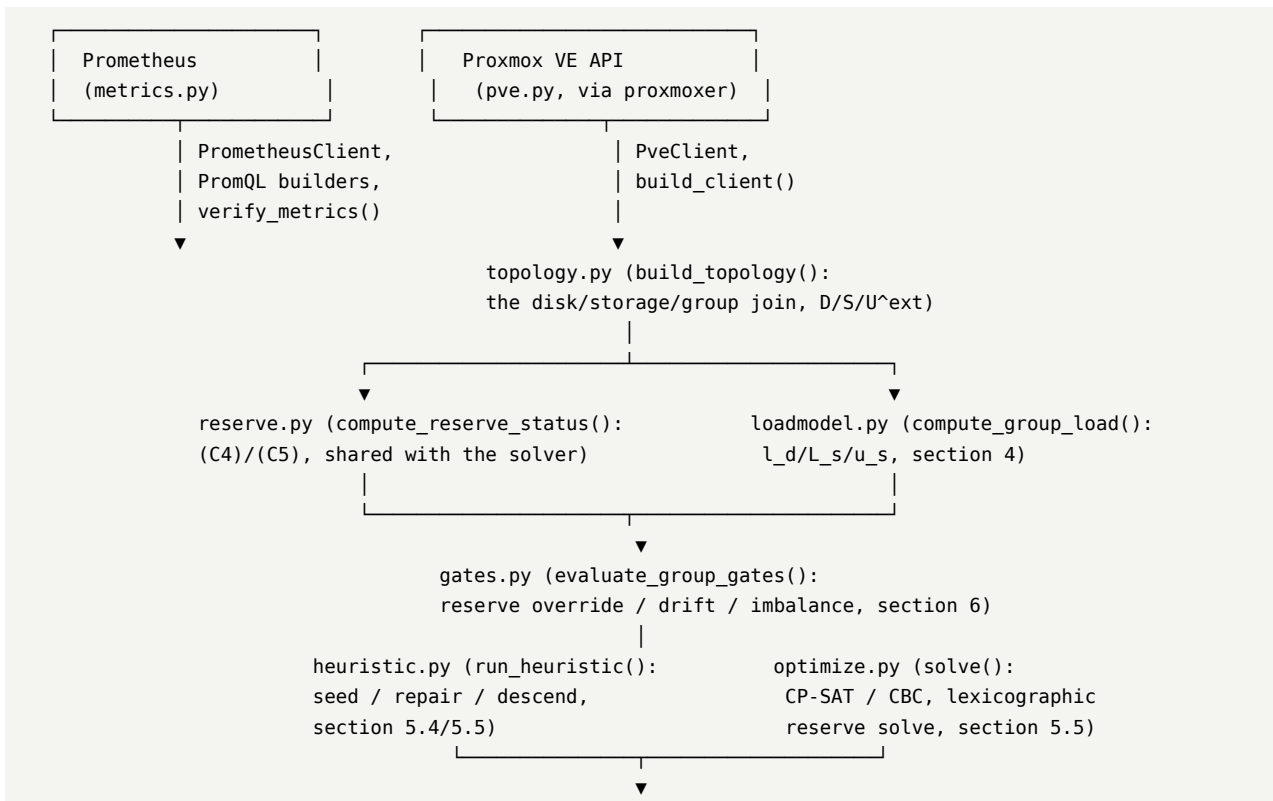
anonymize.py, collect.py, replay.py — diagnostic bundles	59
Why three modules, not one	59
Capture: recording clients, not a second code path	59
Range queries: chunk, then stitch by query text	60
now has to be controllable, all the way down	60
Node selector: anonymized names, captured verbatim	60
Anonymization: allowlist, then pseudonym, then timestamp	60
Replay: real subclasses, not lookalikes	61

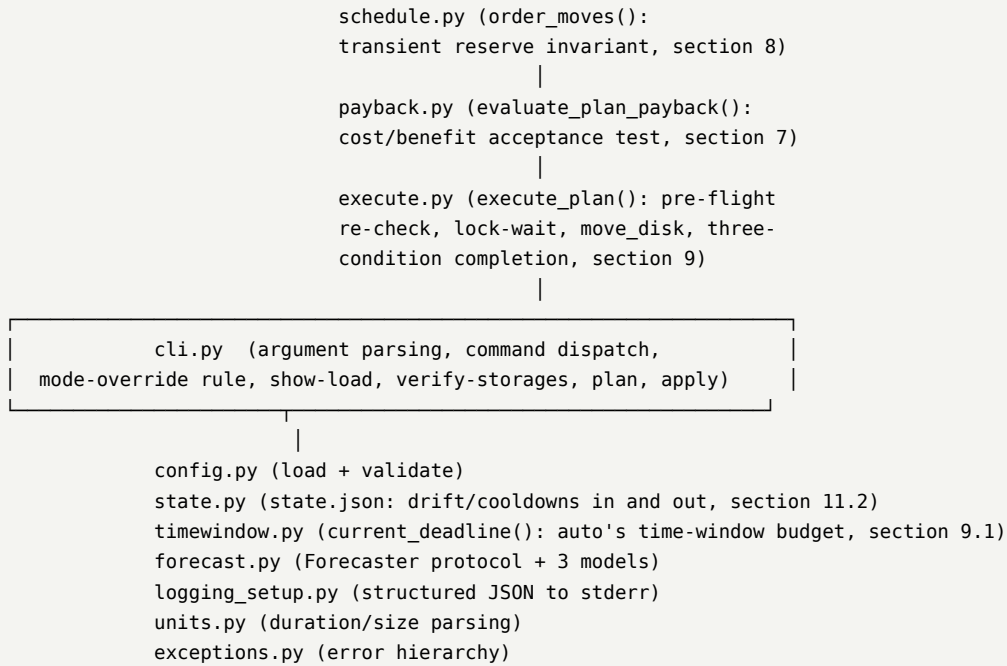
What does this page answer? Where does an invocation of `pve-storage-drs` go, which module owns which piece, and how much of `IMPLEMENTATION_PLAN.md`'s section 2 architecture actually exists right now?

The pipeline, as specified and as built

`IMPLEMENTATION_PLAN.md` section 2 describes seven stages: collect, join, gate, solve, cost, order, execute. As of this page, all seven exist, with **two** interchangeable stage-4 (solve) backends — the dependency-free heuristic and the MILP (CP-SAT/CBC) path — selected by `solver.backend`. Stage 7 (execute) is real for all three `execution.mode` values, including `auto`'s own section 9.1 time-window/migration-count budgets, section 9.2 re-plan loop, and section 8.1/9.2's concurrent execution (`execution.max_concurrent_migrations/max_concurrent_per_storage` above their default of 1, `auto-only`, strictly FIFO — see `92-execute.md`'s own "Concurrent execution" section). Do not take this page as a claim that every phase-8 knob does something — `../manual/30-safety-and-status.md` is the authoritative per-command status: `plan` prints an ordered, transient-feasible, payback-checked plan (see `../manual/27-plan.md`), and `apply` (see `../manual/28-apply.md`) executes it in any mode, re-validating every move against the live cluster immediately before issuing it (section 9.2), waiting out a VM lock rather than failing (section 9.3.1), and not considering a move done until the task succeeded *and* the source volume is gone *and* the VM's lock is clear (section 9.3.2) — see `92-execute.md`. `state.json` itself (`state.py`) is read by `show-load` and `plan` for drift history and cooldowns, and now **written** by `apply` after any run that actually executed a migration — see `15-state.md`, `80-gates.md` and `92-execute.md`.

A handful of short symbols recur below and in every other page this one links to, so they are worth fixing once, here: D is the set of every managed disk a group places (pinned or not), S its set of storages, and U^{next} the bytes on a storage that belong to no managed disk at all — orphans, other groups' foreign volumes (`60-topology.md`). $(C2)$ is the plan's per-disk eligibility constraint (a pinned disk cannot be reassigned); $(C4)/(C5)$ are the largest-disk and capacity/snapshot- reserve constraints `reserve.py` evaluates per storage, both in `60-topology.md`. ℓ_d is one disk's own blended load; L_s/u_s a storage's summed load and I/O-share utilization; u^* the group's average utilization — all four in `70-loadmodel.md`.





Module layout, as it stands

Module	Responsibility	Plan section
exceptions.py	The DrsError hierarchy every deliberate failure raises	AGENTS.md section 5
units.py	Duration/size parsing ("24h", "200MiB") and human-readable formatting	section 11
config.py	Load /etc/pve/drs.yaml, jsonschema + semantic validation, the frozen dataclass config model	section 11
config_schema.json	The jsonschema structural half of validation	section 11.1
state.py	state.json: the load vector as of the last executed balance, cooldowns, the node-local advisory flock() — reading degrades, writing raises	section 11.2
forecast.py	The Forecaster protocol, required_range_seconds, and quantile/seasonal_naive/holt_winters	section 10
logging_setup.py	Structured JSON logging to stderr	section 2.1 (amended, see 40-cli-and-logging.md)
metrics.py	PrometheusClient, PromQL construction, verify_metrics(), compute_disk_coverage()	sections 3.1-3.4
pve.py	PveClient (built on proxmoxer), build_client()	section 3.5
topology.py	build_topology(): the disk/storage/group join, D, S, U ^{sex} t, (C2) pins; resolves free_space.soft/.hard onto Storage.free_space_soft_bytes/.free_space_hard_bytes (section 5.3.1) and storage_type/allowed_formats for storage_accepts_format() (C2)	sections 3.5-3.7, 5.1, 5.3 (C2), 5.3.1

Module	Responsibility	Plan section
reserve.py	compute_reserve_status(): (C4)/(C5) against storage.free_space_soft_bytes, shared by show-load/plan/explain and the solver; transient_charge_ok(): section 8.1's transient invariant against hard_b, generalized to any number of concurrent charges on one target, shared by schedule.py (model-based) and execute.py (live); total_shortfall_bytes(): Σr_s , section 7.3's outcome trigger and revert test	section 5.3 (C4)/(C5)/5.3.1, section 8.1
loadmodel.py	compute_group_load(): the raw-series-to- l_d blend, min_coverage rejection, current L_s/u_s	section 4
gates.py	evaluate_group_gates(): reserve override, drift, imbalance — the act/no-act verdict, with reasoning	section 6
heuristic.py	run_heuristic(): seed/repair/descend, and evaluate_assignment(), the section 5.4 objective shared with the MILP path too; every candidate-generating helper excludes a (C2) format-ineligible target	sections 5.4/5.5, 5.3 (C2)
optimize.py	solve(): CP-SAT/CBC, section 5.3's constraints (including (C2) format eligibility, _fixed_zero_pairs()), the lexicographic two-stage reserve solve	section 5.5, 5.3 (C2)
schedule.py	order_moves(): transient-feasible ordering of a target assignment's moves, deadlock reporting	section 8
payback.py	evaluate_plan_payback(): the cost/benefit acceptance test, with a plan-outcome repair exemption (current_shortfall_bytes/final_shortfall_bytes, section 7.3) mirroring gates.py's reserve override in spirit but scored on the plan's own Σr_s , not a per-move flag; repair_markers()/executed_assignment(): the section 7.3 revert test	section 7, 5.3.1
execute.py	execute_plan(): pre-flight re-check per move, VM-lock wait, move_disk, the three-condition completion criterion, orphan detection on failure, auto's own time-window/migration-count budgets	section 9
timewindow.py	current_deadline(): is now (local time) inside a configured execution.time_windows entry, and when does it close	section 9.1
cli.py	Argument parsing, command dispatch, --manual, the mode-override rule, show-load, verify-storages, plan, apply, collect-testdata, the global --replay	section 11.3

Module	Responsibility	Plan section
<code>anonymize.py</code>	Allowlists, <code>pseudonym()</code> /Mapper, timestamp rebasing. Pure, no I/O — the shared implementation behind <code>collect.py</code> and <code>tests/corpus/validate_corpus.py</code> 's scrub audit	section 16.3
<code>collect.py</code>	<code>capture_bundle()</code> : the recording client wrappers, the estimate/refusal, the deterministic writer	section 16.1-16.4
<code>replay.py</code>	<code>ReplayPveClient/ReplayPrometheusClient</code> real <code>PveClient/PrometheusClient</code> subclasses serving a bundle, no network	section 16.5

Every phase 8 knob is now real: time windows, `max_migrations_per_run`, the re-plan loop, and concurrent execution (`execution.max_concurrent_migrations/max_concurrent_per_storage` above their default of 1, auto-only, strictly FIFO) — see [92-execute.md](#) and [../manual/28-apply.md](#). `schedule.py` itself still only ever produces a strictly-sequential *order* (see below); `execute.py`'s concurrent executor uses that same order as a launch queue rather than needing `schedule.py` to reason about overlapping in-flight windows itself. `optimize.py` (phase 6) is done: plan/apply pick CP-SAT, CBC or the heuristic per solver.backend, and both MILP backends now enforce the storage cooldown identically to the heuristic (see [91-optimize.md](#)). Cooldowns and drift history are read *and written*: `topology.py`'s (C2) per-disk pin and `heuristic.py`'s per-storage target exclusion consume `state.py`'s cooldown data, and `apply` writes a fresh one for every disk and **both** of a move's storages a run actually migrated (see [15-state.md](#), [60-topology.md](#), [90-heuristic.md](#) and [92-execute.md](#)). Within modules that do exist: heuristic step 4 “polish” and (C2) format-compatibility eligibility in `heuristic.py` (see [90-heuristic.md](#)); `schedule.py` itself still only ever produces a strictly-sequential order (not `execute.py`'s own concurrent *execution* of it, which is real – see above – but reasoning about which pairs of moves could safely be *scheduled* to overlap, priority-2 ordering, and staging, per [95-schedule.md](#)); and the payback re-solve-and-retry loop in `payback.py` (see [96-payback.md](#) – the section 7.3 saturation check itself is implemented there now, mirroring-phase only).

Why config.py depends on forecast.py

`config.py`'s semantic validation (`_check_forecast_window`) must reject a window.lookback too short for the configured `forecast.model` at startup (section 11.1) — that check needs `forecast.required_range_seconds()`. Since `forecast.py` in turn needs `config.ForecastConfig` for its type signatures, the import is deferred to inside the validation function rather than at module level, breaking what would otherwise be an import cycle. See the comment at the top of `config.py`'s `_check_forecast_window` if you touch this.

Where to read next

- [10-configuration.md](#) — how a YAML file becomes a validated Config, and where every default actually lives.
- [15-state.md](#) — `state.json`'s dataclasses, why reading it degrades but writing raises, the real `flock()` lock and the rename-vs -in-place bug it takes to get that wrong, what `show-load/plan` actually get from it today, and how its cooldown data reaches `topology.py/heuristic.py`.
- [20-forecasting.md](#) — the forecaster protocol and its three implementations.
- [30-metrics.md](#) — the Prometheus client and the six `verify-metrics` checks.
- [40-cli-and-logging.md](#) — command dispatch, the `--mode` escalation rule, and why logs go to `stderr`.
- [50-pve-api.md](#) — the PVE API client, why it is built on `proxmoxer`, and two things verified against a real cluster.

- [60-topology.md](#) — the disk/storage/group join, the section 5.1.1 foreign-volume accounting, the shared (C4)/(C5) evaluator, and the per-disk cooldown pin.
- [70-loadmodel.md](#) — the section 4 raw-series-to-*l_d* blend, `min_coverage` rejection, and why `tpmstate0/unusedN` are exempt from it but `efidisk0` is not.
- [80-gates.md](#) — the section 6 act/no-act verdict, how `state.json`'s drift history now reaches it, and why the per-disk/ per-storage cooldowns live in `topology.py/heuristic.py` instead.
- [90-heuristic.md](#) — the section 5.4/5.5 objective and the seed/repair/descend search, cross-checked against section 14's exact objective totals, the storage-cooldown destination filter and its repair-side exemption, and what “polish” and format eligibility still owe.
- [91-optimize.md](#) — the CP-SAT and CBC MILP backends, section 5.5's exact integer coefficient scaling (and the γ -term trap it exists to avoid), why the reserve is solved in two lexicographic stages rather than one big-M objective, and `cli.py`'s `solver.backend` dispatch (auto's cascade, and why an explicitly forced backend still falls back to the heuristic rather than failing).
- [95-schedule.md](#) — ordering a target assignment's moves under the section 8 transient invariant, why `cost_m = z_d` is exact today (not an approximation), and why the heuristic accepting a residual violation can still mean the scheduler reports a deadlock.
- [96-payback.md](#) — the section 7 cost/benefit test, the `headroom_src/headroom_dst` gap in the plan's own cost formula, and why a reserve-fixing plan always passes the economic test.
- [92-execute.md](#) — why a completed move needs three conditions, not one, why a VM lock is waited out rather than whitelisted, the injectable `Clock`, how `cli.py`'s `_plan_group()` gives plan and apply one shared pipeline instead of two, and auto mode's time-window budget and re-plan loop (`timewindow.py`, `cli._run_auto_group()`).
- [93-crashrecovery.md](#) — section 13's startup scan for a `move_disk` left running by a crash or a second instance, the cluster-wide `/cluster/tasks` read that `state.json`'s own `flock()` cannot do, and how a UPID reaches disk *before* the move that owns it can crash the engine.

How a YAML file becomes a validated Config

What does this page answer? What exactly happens between `pve-storage-drs -c foo.yaml plan` and a type-checked Config object, and where does each of the section 11.1 validation rules actually live? Describes `proxmox_storage_drs/config.py` and `config_schema.json`.

Resolution order

`resolve_config_path()` implements `IMPLEMENTATION_PLAN.md` section 11's three-source order: `--config` (`cli_path`), then `$PVE_STORAGE_DRS_CONFIG`, then `config.DEFAULT_CONFIG_PATH(/etc/pve/drs.yaml)`. It also returns `was_explicit`, which `load_config()` uses to decide the error message on a read failure: an explicitly named path that cannot be read is always fatal with no fallback, per section 11, but the *message* additionally points at `config/drs.example.yaml` only when the failing path was the unnamed default — there is nothing useful to suggest when the operator named the path themselves.

The two-stage validation

1. `_validate_schema()` loads `config_schema.json` via `importlib.resources` (it ships as package data — see `pyproject.toml`'s `[tool.setuptools.package-data]`) and runs it through `jsonschema`. Every object in the schema sets `additionalProperties: false`: a typo'd key is caught here as a validation error, not silently ignored, which is what keeps section 15.1's knob-to-formula table trustworthy (a key with no formula would otherwise go unnoticed).
2. `_build_config()` walks the now-schema-valid raw dict into the frozen dataclasses defined earlier in the module, applying every default inline via `.get(key, default)`. These defaults are also what `config/drs.example.yaml` documents and what the manual's [../manual/10-configuration.md](#) describes — there is deliberately no second copy of a default anywhere else in the codebase.

3. `_validate_semantics()` runs the section 11.1 rules jsonschema cannot express — cross-field comparisons, or rules needing a value derived from two independently-optional settings. Each rule is a small `_check_*` function appending to a shared errors/warnings list (kept separate functions rather than one large one, partly for flake8’s max-complexity and partly because each is independently testable). errors become one `ConfigError` naming every problem found, not just the first; warnings are returned to the caller (`ResolvedConfig.warnings`) for `cli.py` to log — section 11.1’s `saturation_load` check is the one warning-only rule today.

Every duration or size field is parsed **once**, at this point, via `units.parse_duration_seconds/parse_size_bytes`, and stored on the dataclass already in its canonical unit (seconds, bytes) — nothing downstream re-parses a string.

Secrets from the environment

`_build_config()` fills `proxmox.auth.password/token_secret` from `PVE_PASSWORD/PVE_TOKEN_SECRET` **only when the file’s own value is null** — a secret actually written in the file is used as-is, never silently overridden by an environment variable a deploy script forgot to unset. This one-directional fallback is what section 11’s “keep secrets out of `/etc/pve`” guidance is for: the file can omit the field entirely and rely on the environment, or set it explicitly and take responsibility for the file’s own permissions.

What is deliberately *not* checked here

Two section 11.1 rules need live cluster data this module does not have and are therefore **not** part of `config.py`:

- storage ids actually existing in the cluster;
- `execution.source_release.timeout` / `gates.cooldown_per_storage` against a storage’s real `saferemove_throughput`.

Both belong to `pve.py/topology.py` (which do the live cluster fetch) and `pve-storage-drs verify-storages` — see [../manual/30-safety-and-status.md](#) for current status.

ResolvedConfig: what the caller actually gets

`load_config()` returns a `ResolvedConfig`, not a bare `Config`: the resolved path and the file’s sha256 travel with it, because `IMPLEMENTATION_PLAN.md` section 11 requires every run to log both — a plan must always be traceable to the exact file that produced it, even though the file is never re-read after startup.

Persistent state: `state.json`

What does this page answer? What does `state.py` actually store, how does reading it degrade instead of failing, why is the advisory lock a real `flock()` rather than a JSON field, and what does today’s wiring into `show-load/plan` actually change? Describes `proxmox_storage_drs/state.py`.

One dataclass tree, matching section 11.2’s JSON exactly

State mirrors the plan’s own example document field for field: `lock (LockInfo | None)`, `last_balance` (at plus a `"<group>:<vmid>:<device>"` -> `l_d load_vector`), `cooldowns` (`disk/storage`, each a key -> timestamp mapping), `inflight_upids`, `staged_disks`. `disk_state_key()/storage_state_key()` build the two key shapes section 11.2 specifies — group-prefixed specifically so a vmid reused after a VM is destroyed and recreated in a *different* group cannot collide with its old entry. `empty_state()` is what a fresh install, or a lost file, degrades to: no lock, no balance, no cooldowns, nothing in flight.

Reading degrades; writing does not

`load_state()` never raises. A missing file is the ordinary first-run case (silent); a present-but-corrupt, empty, or wrong-schema_version one is still just `empty_state()`, but logged at WARNING — section 11.2’s

own words are “losing this file is safe but not free: cooldowns and drift history reset”, not “state.json existing and being readable is required to run”. This mirrors `show-load`'s choice to degrade a group's load to “unavailable” on a Prometheus outage rather than fail the whole command — neither Prometheus nor `state.json` is an *explicitly requested* input the way `-c/--config PATH` is, and only the latter is a hard failure on error (`.agents/domain-invariants.md` section 9).

`save_state_atomic()`, by contrast, raises `StateError` on any failure — writing (or locking) is something a caller actively asked this module to do, not a read with a documented, safe fallback. It writes to a temp file in the same directory, `fsyncs`, then `os.replace()`s over the real path, so a concurrent `load_state()` can never observe a half-written file — which is exactly what lets every read in this codebase skip locking entirely (next section).

Locking: `flock()` is the mechanism, the JSON lock field is a label

`acquire_lock()` takes a real, non-blocking `fcntl.flock(LOCK_EX | LOCK_NB)` on the state file itself. That is the actual, kernel-enforced mutual exclusion section 11.2 asks for, and it is correct by construction for two instances on the *same host* — `.agents/domain-invariants.md` section 9 is explicit that “the `fcntl` lock cannot see another node”; cross-node coordination is the separate in-flight-UPID scan (section 13), not this module's job.

The `lock: {pid, host, acquired_at}` field recorded *inside* the JSON is **descriptive metadata for a human reading the file, not the exclusion mechanism** — by the time this module's code overwrites it, the OS has already granted exclusive ownership, so whatever a previous run last wrote there is necessarily stale and is unconditionally replaced, never inspected to decide whether to proceed. This is also how section 11.2's “if `lock.pid` is not alive on `lock.host`, reclaim it” is satisfied, without a separate liveness check to get subtly wrong: a process that died released its `flock()` the moment its file descriptor closed (any exit, including a crash), so the *next* `acquire_lock()` call simply succeeds. `acquire_lock()` returns `None` — not an exception — when another live instance already holds it, matching section 11.2's “a live PID means another instance is running: exit 0 quietly”; deciding what “quietly” means, and which exit code, is left to the caller.

A real bug this design surfaced during testing, worth remembering if you touch this code: the first implementation had `acquire_lock()` write its lock metadata via `save_state_atomic()` — the same temp-file-plus-rename used everywhere else. That silently breaks the lock: a rename replaces the directory entry with a *new* inode that was never `flock()`'d, so a second `acquire_lock()` opens that new inode fresh and locks it with no conflict at all — `test_a_second_acquire_while_the_first_is_held_returns_none` caught this immediately. The fix, `_write_state_to_locked_fd()`, writes **in place** into the already-open, already-locked file descriptor (`lseek` to 0, `write`, `ftruncate` to the new length, `fsync`) — never a rename — for as long as the lock is held.

What today's wiring into `cli.py` actually changes

`show-load` and `plan` both call `_last_loads_by_group()` once, right after reading configuration, and use the result for both `loadmodel.compute_group_load()`'s `last_known_loads` and `gates.evaluate_group_gates()`'s `last_load` — one state read per run, one function computing the per-group slice, not two independent implementations (`AGENTS.md` section 5). Neither command takes the advisory lock: both are read-only reports that never execute a migration, and taking the exclusive lock for one would make an in-progress apply block `plan/show-load` for no safety reason the `plan` text supports. `apply` (`docs/internals/92-execute.md`) is the one command that does take it — before touching PVE or Prometheus at all, so a second concurrent instance exits quietly without paying for either round-trip first.

The practical effect: a `state.json` with a real `last_balance` now makes the drift gate genuinely suppress an ACT verdict the imbalance alone would otherwise trigger — see `test_show_load_gate_reflects_`

`real_drift_history_from_state_json` and `test_plan_gate_also_reflects_real_drift_history_from_state_json` in `tests/unit/test_cli.py` for the exact before/after. Without a `state.json` on disk (the common case until `apply` has actually executed a migration), behaviour is unchanged from before this module existed: `last_load=None`, drift gate skipped, “reserve override, else imbalance”.

Cooldowns: read by `topology.py` and `heuristic.py`, not by this module

`state.py` only stores and queries cooldown timestamps (`active_disk_cooldowns()`/`active_storage_cooldowns()`); it does not decide what a cooldown *means* to the solver. `build_topology()` (via a new `state/now` parameter, both defaulting to “no cooldowns”/“real clock”) computes each group’s active disk cooldowns once and feeds them into `topology._pin_reason()` — section 5.3 (C2)’s “d is within its per-disk cooldown -> also pin to current”, reported as `cooldown: moved recently, <time> left on gates.cooldown_per_disk`, in the plan’s own priority order (after a snapshot pin, before a lock pin). `cli.py` separately computes each group’s active *storage* cooldowns and passes them to `heuristic.run_heuristic()`, which excludes them as a move/swap **destination** in `_descend()` only — see `docs/internals/60-topology.md` and `docs/internals/90-heuristic.md` for the two halves in detail, including the deliberate asymmetry: a live (C4)/(C5) repair move ignores the storage cooldown entirely (section 13’s reserve-override principle), while nothing yet exempts a disk-cooldown pin from blocking a repair the same way — a known, documented limitation, not an oversight.

`apply` is what actually calls `with_recorded_cooldown()` now, once per group, for every disk/storage a run’s `execute_plan()` call actually migrated — a fresh disk cooldown at the disk’s *new* location, and a fresh storage cooldown for the move’s **destination** only, mirroring `_descend()`’s own asymmetry above (`docs/internals/92-execute.md`). A group `apply` never acts on (the gate said NO ACTION, or nothing executed before a `load_error/replan_needed` stopped it) leaves that group’s cooldowns untouched, exactly like before this was wired in.

Locking and writing: `apply`’s own responsibility

`acquire_lock()/release_lock()/save_locked_state()` are exercised now: `apply` is the one command that can execute a migration, so it is the one this lock protects (see `docs/internals/92-execute.md`). It acquires the lock first, builds up the run’s State in memory as each group finishes executing (`with_recorded_balance()/with_recorded_cooldown()`), and writes it once at the very end via `save_locked_state()` — **never** `save_state_atomic()`’s `rename`, which would silently detach the very lock just taken (see this page’s own account of that bug, above) — before `release_lock()` in a `finally`, so a mid-run exception never leaves the lock held. `show-load/plan` still never take it, for the same read-only-report reason as always.

Deliberately not implemented in this pass

- **A disk-cooldown pin is not exempted for an active reserve violation the way the storage cooldown is.** `topology.py` decides a disk’s pin before the group’s `reserve.ReserveStatus` is even computable (it needs every disk in the group, which is still being built), so there is no cheap way today to ask “would un-pinning this disk help fix a live violation?” at the point the pin decision is made. The practical effect is bounded and safe, never silently wrong: `heuristic._repair()` already reports an unresolvable residual violation via `reserve_penalty_term` whenever no movable disk can fix it (the same path `test_repair_does_not_oscillate_when_no_target_can_fully_absorb_the_violation` exercises) — a disk-cooldown pin can only ever add to the set of cases that hit that path, never bypass it. Revisiting this needs restructuring `topology.py` into two passes (build every disk first, compute reserve status per storage, then finalize cooldown pins), which is a real, separately-scoped change.
- **inflight_upids** is now written and read — see `docs/internals/93-crashrecovery.md` for the full mechanism (`with_inflight_upid()/without_inflight_upid()`, `crashrecovery.py`’s startup scan,

and `cli.py`'s synchronous-write callback into `execute_plan()`.

- **staged_disks** exists in the dataclass and round-trips correctly, but nothing writes or reads it yet — it belongs to `schedule.py`'s staging (section 8.3 option 1), not implemented.

Forecasting: the protocol and its three implementations

What does this page answer? How does `forecast.py` decide how much history a model needs, and what does each of `quantile/seasonal_naive/holt_winters` actually compute? Describes `proxmox_storage_drs/forecast.py`.

The one rule, in one place

Every model needs at least `window.lookback` seconds of history (the *decision* window, since a forecaster's point estimate is always over that window) — `quantile` needs nothing more than that. `seasonal_naive` needs whichever is larger of `window.lookback` and `forecast.seasonal_lookback_days` converted to seconds ($\times 86400$): enough calendar days that every hour-of-day bucket (see `SeasonalNaiveForecaster` below) has actually been sampled at least once, not just the decision window itself. `holt_winters` needs whichever is larger of `window.lookback` and $2 \times \text{forecast.holt_winters.seasonal_periods} \times \text{metrics.step}$ seconds — two full seasonal cycles' worth of samples at the configured scrape step, matching `statsmodels`'s own “needs at least two periods to fit a seasonal component at all” requirement (see its own fallback rule below). That rule has exactly one implementation: three small pure functions, `_quantile_required_range_seconds`, `_seasonal_naive_required_range_seconds` and `_holt_winters_required_range_seconds`, each called from **two** places — the free function `required_range_seconds()` that `config.py`'s startup validation uses (before any `Forecaster` is constructed), and the matching `Forecaster.required_range()` method on the concrete class. Duplicating the arithmetic between those two call sites, rather than sharing the pure function, is exactly the kind of “second copy of a rule” `AGENTS.md` section 5 forbids — a change to the Holt-Winters requirement made in only one of them would silently desynchronize config validation from what the forecaster itself believes it needs. (`IMPLEMENTATION_PLAN.md` section 10.1 has the full per-model table this generalizes from.)

Why the upper bound, never the point estimate

Every `Forecaster.predict()` returns a `Forecast(point_estimate, upper_bound)`. Nothing in this module enforces that callers use `upper_bound` — that discipline belongs to the caller: the section 7.3 saturation guard (`payback.py`'s `compute_move_cost()`, via `storage_upper_bound()` below) already only ever reads `.upper_bound`; the optimizer does not consume a forecast at all yet. The asymmetry is stated in the module docstring and repeated here because it is easy to get backwards under time pressure: overestimating costs a slightly worse balance, underestimating risks scheduling a mirror onto a storage that is about to saturate.

storage_upper_bound(): section 10.1's per-disk sum, not a per-storage forecast

Section 10.1 is explicit that $\hat{L}_s(\Delta)$ is $\sum_{d : x_{d,s}=1} \hat{u}_d(\Delta)$ — every disk on s forecast **independently**, then summed — never the forecast of s 's own already-summed series. `storage_upper_bound()` is exactly that sum, given a `Forecaster`, `{disk_key: TimeSeries}` (typically `loadmodel.compute_disk_load_series()`'s own output), the disk keys currently on one storage, and a horizon. Summing upper bounds this way is deliberately conservative (it assumes every disk peaks together — the right direction for a guard whose failure mode is starting a mirror onto an already-busy array), and is real, measurable extra conservatism whenever a group's disks do not actually peak in lockstep: forecasting one already-summed series directly would let one disk's trough offset another's peak, understating the storage's own worst case. A disk key with no fetched series at all forecasts as an empty one (`0.0`), never a `KeyError` — this run may simply have no history for a disk yet.

QuantileForecaster

The default. `predict()` takes the `window.quantile/window.upper_quantile` percentiles of the raw series with no fitting at all. `_quantile()` implements linear-interpolation percentile matching `numpy.percentile`'s default method, by hand — deliberately not using `numpy`, since the tool must stay usable with only `requests + ruamel.yaml + jsonschema` installed (IMPLEMENTATION_PLAN.md section 2.1). `horizon` is accepted (to satisfy the Forecaster protocol) and immediately discarded: the quantile model has no notion of a horizon-dependent forecast.

SeasonalNaiveForecaster

Groups historical samples by hour-of-day (`int((ts // 3600) % 24)`) and takes the median as the point estimate, the configured upper quantile across the same bucket as the bound. `now_epoch_seconds` selects which hour-of-day bucket the *current* moment falls into; `predict()` ignores everything outside that bucket. A bucket with no samples returns `Forecast(0.0, 0.0)` rather than raising, matching IMPLEMENTATION_PLAN.md section 4's rule for an idle group's raw quantities: no data is zero, not an error, at this layer (the caller decides whether zero is trustworthy).

HoltWintersForecaster

`statsmodels` is an optional dependency (Suggests, not Depends — see `debian/control`), so it is imported **only inside `predict()`**, never at module level; `debian/tests/import-all` is what would catch a regression here. Two situations fall back to `QuantileForecaster`, both logged at warning level rather than failing silently:

1. fewer than $2 * \text{seasonal_periods}$ samples in the series (section 10.2's own rule — a fit on too little data is worse than no fit);
2. `statsmodels` is not importable at all.

The upper bound is `point_estimate + residual_z * stdev(in-sample residuals)` — not derived from the model's own confidence interval, which `statsmodels`'s `ExponentialSmoothing.fit()` does not expose directly for every configuration. `residual_z` (`forecast.holt_winters.residual_z`, default 2.0) is what an operator tunes if this bound turns out too tight or too loose in practice.

The section 10.2 backtest validation gate (phase 9)

Fitting successfully (`HoltWintersForecaster`'s own two fallback conditions above) is not the same question as fitting *accurately* — section 10.2's backtest is what actually answers the second one, once per group, before `seasonal_naive/holt_winters` is trusted to drive the section 7.3 saturation guard at all. `quantile` is never backtested: it does no fitting, so there is nothing to validate and nothing more conservative to fall back to.

`backtest_error()` implements the plan's own recipe literally: fit the candidate forecaster on `[now-2W, now-W)`, predict `W` ahead (`W` is `window.lookback_seconds`, the same “decision window” every forecaster is already tied to for its point estimate), and compare that single point estimate against the *actual* mean observed over `[now-W, now]`. The result is a **relative** error — $|\text{predicted} - \text{actual}| / \text{actual}$, normalizing by `predicted` instead only when `actual` is exactly zero (a nonzero prediction against true silence then reads as a full, bounded miss, 1.0, rather than a division by zero; two zeros, correctly predicted silence, score a perfect 0.0) — so it is directly comparable against `gates.imbalance_threshold` (both plain fractions in `[0, 1]`, section 10.2's own choice of yardstick). `backtest_error()` returns `None`, not 0.0 or an exception, when series does not cover a full `2W` to split into two non-empty halves at all; `backtest_validated()` treats that exactly like a failed backtest — a fresh deployment with no track record yet does not get a free pass just because it has not been disproven, matching `HoltWintersForecaster`'s own existing “not enough samples yet → fall back” precedent rather than contradicting it.

Validated once per group, against the group’s own aggregate series, not once per disk. `group_aggregate_series()` sums every disk’s own series at the union of their timestamps (a disk missing a sample at some timestamp contributes 0, the identical “absence means no I/O” convention `loadmodel._blend_loads()` already uses). `forecast.model` is a single, deployment-wide configuration choice — it cannot differ disk by disk — so validating it once, cheaply, against one representative series is the right granularity; backtesting separately per disk per move would be both more expensive and would raise an unanswerable question (use the fancier model for the disks it happens to fit and quantile for the rest, *within the same run?*) that the plan itself never poses.

`cli._backtest_gated_forecaster()` is the one caller: built once per group inside `_saturation_forecast_inputs()`, right after `build_forecaster()` and `loadmodel.compute_disk_load_series()`, before either is used to derive any move’s `l_hat_src/l_hat_dst`. A model that fails validation falls back to a fresh `QuantileForecaster` (routed through `build_forecaster()` again with `forecast.model` overridden to “quantile”, never hand-constructed — the one factory, see below), logged at warning with the model name that failed.

build_forecaster()

The one factory function that turns a `ForecastConfig` plus the ambient window/metrics values into a live `Forecaster` instance. Nothing else in the codebase should construct a `QuantileForecaster/SeasonalNaiveForecaster/HoltWintersForecaster` directly once a caller exists that needs one from config — route it through here so the model-name-to-class mapping stays in one place.

The Prometheus client and verify-metrics

What does this page answer? How does `metrics.py` talk to Prometheus without ever touching the network in a test, and what does each of the six `verify-metrics` checks actually query? Describes `proxmox_storage_drs/metrics.py`.

_SessionLike / _ResponseLike: why not requests.Session directly

`PrometheusClient.__init__` accepts `session: _SessionLike | None`, a `Protocol` defined in this module with only the one `get(...)` method the client actually calls — not `requests.Session` itself. `.agents/testing.md` forbids any test talking to a real Prometheus; a structural protocol lets a test hand in a small dataclass with a matching `get()` instead of mocking or subclassing the real (network-capable) session. `PrometheusClient()` with no `session` argument still constructs a real `requests.Session()`, which satisfies the protocol structurally — nothing special is needed to make the real thing fit.

_get(): one request path, three endpoint shapes

Every one of `instant_query`, `range_query` and `label_values` goes through the private `_get()`, which does the HTTP call, checks the status code, parses JSON, checks Prometheus’s own status: “success” field, and returns `payload[“data”]`. That field’s *shape* genuinely differs by endpoint — a dict with a `result` list for `query/query_range`, a bare list for `label/<name>/values` — which is why `_get()` is typed `Any` rather than picking one shape; each public method knows which shape it asked for and narrows accordingly. Every failure mode (a transport error, a non-200 status, unparseable JSON, an unsuccessful Prometheus response) raises `MetricsError` from exactly this one place, so a caller catching `MetricsError` around any of the three public methods sees every failure mode uniformly.

PromQL construction

`build_rate_promql()` builds `sum by (vmid, device) (rate(<metric>{<selector>}[<rate_window>]))` — the `sum by` deliberately collapses the node/host labels, so a VM that live-migrated between nodes mid-window still reads as one series, and `selector` (see “Node-scoping” below) sits

inside `rate()`'s own vector selector, before those labels are collapsed. `build_quantile_over_time_promql()` wraps a rate expression in `quantile_over_time(q, (<rate_expr>)[<lookback>:<step>])` — `metrics.py`'s own six-row-quantity fetch calls the first to build each `rate_expr`, then the second around it, once per configured quantile. Both are pure string-building functions, deliberately factored out of any query-issuing code so they are unit-testable without a fake session at all (IMPLEMENTATION_PLAN.md section 3.4 has the full per-metric derivation). `raw_metric_name()` is the one place that maps a RAW_METRIC_FIELDS entry ("read_ops", ...) to the configured metric name on a MetricsConfig, via `getattr` — this is what lets `verify_metrics()` iterate "every configured raw metric" without hand-listing the six field names a second time anywhere in this module.

Node-scoping: `build_node_selector()` / `resolve_node_selector()`

`build_rate_promql()`'s selector parameter is what makes a query correct on a Prometheus that serves more than one PVE cluster (or anything else emitting a same-named metric): without it, `sum by (vmid, device)` happily sums across clusters, since `vmid` is only unique *within* one. The selector text itself is resolved once per command invocation by `resolve_node_selector(metrics, node_names)`, in this order:

1. `metrics.extra_selector` verbatim when the operator set one (trusted completely; it may not even name a node label at all).
2. `build_node_selector(metrics.labels.node, node_names)` otherwise, which escapes every name (`_escape_promql_regex_literal()` — an FQDN's `.` is a regex metacharacter otherwise) and joins them into one `==` alternation. This is the default tier, from this cluster's own node list.
3. None when nothing above resolved anything (an empty `node_names`).

An earlier revision of this function had a middle tier here matching a cluster-naming label against the live cluster's own name (`PveClient.cluster_name()`, `GET /cluster/status`). It was removed — operator correction: the premise that this project's deployments carry such a label "as standard practice" was simply wrong, not a real convention, so the tier existed only on a mistaken assumption and the extra `Sys.Audit-gated` API call it needed bought nothing. Use `metrics.extra_selector` for a cluster-naming (or any other) label your own Telegraf/InfluxDB tagging scheme actually has.

`node_names` comes from `PveClient.node_names()` (`GET /nodes`), fetched at most once per run and threaded down through `loadmodel.py`'s functions as a plain `Sequence[str] | None` — this module and `loadmodel.py` never call the PVE API themselves.

`cli._resolve_node_selector_for_run()` is the one place that decides *whether* to make that API call at all: skipped when `metrics.extra_selector` is already set. `verify-metrics` never reaches this function — it calls `resolve_node_selector(metrics, None)` directly, applying only an explicit override, because it is deliberately independent of the PVE API and has no node list of its own to build the default tier from (`_check_coverage()` / `_check_observed_spacing()` are the only two of its six checks that build a `rate()` / `range` query at all; the others query a raw metric name or sample one series directly, where cross-cluster contamination is not a correctness question the way a summed rate is).

`safe_range_step_seconds()` / `decimate_to_configured_step()`: the gigapipe step >= range workaround

Two live-confirmed deployments of a recently updated gigapipe backend return **zero series** from `/api/v1/query_range` (and the PromQL `[range:step]` subquery form) for any range-vector function (`rate()`, `irate()`, `increase()`, `delta()`, `deriv()` all confirmed) whenever the query's own step is >= the function's own range-vector duration — even though the identical expression as a plain instant `/api/v1/query` (no step at all) returns correct data. Binary-searched to the exact second: `rate(x[300s])` gets a full response at `step=299s`, nothing at all at `step=300s`. This project's own defaults set `metrics.step == metrics.rate_window(300s each)` — exactly the failing boundary — so a fresh install with untouched

defaults can hit this on an affected backend with no misconfiguration.

`safe_range_step_seconds(step_seconds, rate_window_seconds)` is a no-op once `step_seconds` is already strictly below `rate_window_seconds` (nothing to work around); otherwise it returns the largest whole-second step strictly below `rate_window_seconds` that divides `step_seconds` as evenly as a whole-second value can (150s at the 300s/300s default; not necessarily an exact divisor at other ratios — 3600s/300s divides to 276s, $13 \cdot 276 = 3588 \neq 3600$, a documented $\sim 0.33\%$ -per-point grid drift). `compute_disk_coverage()` and `loadmodel.py`'s raw-series fetch both call it unconditionally — not only when the affected backend is detected, since the symptom (zero series) is indistinguishable from genuinely absent data — and pass its result as the step on every `query_range` call, then `decimate_to_configured_step()` recovers the originally-configured grid by keeping every Nth point of the denser response, anchored at the same start. For an exact divisor this makes the workaround a true no-op: the retained points are the identical instants and values a plain `metrics.step` query would have returned on an unaffected backend.

`quantile_over_time`'s subquery cannot decimate — an instant query returns one scalar, not a series — so it evaluates its inner expression on the denser, safe-step grid unconditionally, on *every* backend, healthy or not: at the 300s/300s default this doubles the inner sample count the 95th percentile is taken over, a small but real shift in the reduced statistic (REVIEW.md Z-04). `collect.py`'s `estimate_capture()` and `--replay`'s `compute_disk_coverage()` fallback (a bundle captured before this function existed, at the plain configured step) both account for this step, not the configured one, for the same reason (REVIEW.md Z-05).

RANGE_QUERY_CHUNK_SECONDS / stitch_range_results(): chunking a wide query_range

A range wide enough — IMPLEMENTATION_PLAN.md section 10.2's backtest-gate floor ($2 \cdot \text{window.lookback}$) alone can double it — issued as a single `query_range` call can exceed a VictoriaMetrics/gigapipe backend's own max-points-per-timeseries limit (11,000 by default), confirmed live with a 500 "exceeded maximum resolution of 11,000 points per timeseries". `collect.py`'s capture path (`_issue_range_chunks`, section 16.2) was already immune to this — day-sized chunking there predates the live-fetch bug — but `loadmodel.py`'s own raw-series fetch, the one that feeds a plan/apply run's section 7.3 saturation guard, issued one unchunked request regardless of range.

`stitch_range_results(captures)` is the shared merge both paths now use: given several (`start`, `end`, `result`) captures of the *same* query — any combination of chunking and repeat calls — it returns one logical result, one series per (`vmid`, `device`) pair with every point from every chunk, deduplicated by timestamp and sorted. Order-independent (a duplicate timestamp across overlapping captures just overwrites with the same value), so it does not matter whether the captures arrived in chunk order. `collect.py`'s own `_stitch_range_captures` is now a thin wrapper around it that adds the (`start`, `end`, `step`) bookkeeping a captured bundle's range file needs.

`loadmodel._issue_chunked_range_query()` is `_fetch_raw_quantity_series()`'s own counterpart to `collect.py`'s `_issue_range_chunks`: it splits [`start`, `end`] into `RANGE_QUERY_CHUNK_SECONDS` (one day, matching `collect.py`'s own chunk size) sub-requests, boundaries falling on `start_epoch_seconds` never on wall-clock "now" (deterministic across repeated calls — matters under `--replay`, where the same promql text is looked up per chunk from one stored bundle file and trimmed to each chunk's own window by `replay.py`'s `_trim_range_result()`), then stitches. It carries the same `RangeStepMismatch/BundleError` fallback the unchunked call always had, applied per chunk: both exceptions are a property of the query text (a `--replay` bundle's own recorded step, or "no recorded response for this query at all"), not of which chunk asks for it, so once the step has fallen back once, every later chunk reuses it directly without re-entering the fallback branch.

verify_metrics(): six checks, six functions

Each `_check_*` function is a single, focused verification and returns Findings (plus, where relevant, the data the report carries forward):

Function	What it checks	Returns
<code>_check_metric_names_exist</code>	every one of the six configured <code>metrics.* names (read_ops, ...)</code> actually exists as a Prometheus series name (<code>label_values("__name__")</code>) — an error per name that does not, naming the likely cause (never configured/emitted, or dropped — see below)	findings only
<code>_check_sample_series</code>	one instant query per metric returns at least one sample, that the expected <code>vmid/device</code> labels are present on it, and (via its private <code>_check_cross_metric_disk_consistency()</code>) that all six metrics report the identical set of (<code>vmid, device</code>) disks — a metric whose disk set is a strict subset of the others' is flagged	findings, {metric_name: sample_labels}
<code>_check_device_label_collision</code>	warns if <code>metrics.labels.device</code> is literally "instance", which collides with Prometheus's own scrape-target instance label — a common Telegraf misconfiguration	one Finding or None (pure, no I/O)
<code>_check_coverage</code>	runs <code>compute_disk_coverage()</code> and flags every disk whose sample coverage over the window falls below <code>window.min_coverage</code>	findings, {DiskKey: coverage_fraction}
<code>_check_observed_spacing</code>	measures the modal delta between consecutive timestamps of one live series over a short recent range, and compares it against what <code>metrics.rate_window >= 4x</code> (section 11.1) assumes the real scrape interval to be	findings, observed_spacing_seconds

`_check_coverage` and `_check_observed_spacing` both use `metrics.read_ops` as the one representative metric rather than probing all six: a coverage or spacing gap is a property of the underlying Telegraf scrape, not of which of the six raw quantities is read, so probing all six would be six times the Prometheus load for no additional information.

That assumption has one real exception: InfluxDB's line protocol fixes a field's type from its first write, and Telegraf's Prometheus-compatible output silently drops a field the moment it sees a non-numeric value for it (Prometheus/OpenMetrics has no string sample type) — independently of the other five fields for the same disk. If the dropped field happens not to be `read_ops`, `_check_coverage` alone would never notice. `_check_sample_series` already runs an instant query against all six metrics anyway (step 2), so its private `_check_cross_metric_disk_consistency()` reuses those same results — no extra Prometheus load — to compare each metric's own (`vmid, device`) set against the union across all six, and warns naming the metric and the specific disk(s) missing from it whenever one metric's set is a strict subset of another's.

`VerifyMetricsReport.ok` is True iff no Finding has `level == "error"` — `cli.py`'s `verify-metrics` hand-

ler uses exactly this property to decide the process exit code, so there is one definition of “the verification passed.”

What cli.py does with the report

`_handle_verify_metrics` in `cli.py` constructs a `PrometheusClient` from `resolved.config.prometheus`, calls `verify_metrics()`, and renders either the human form (`[level]` message lines) or, under `--json`, a JSON-serializable dict — `DiskKey` keys are turned into “vmid:device” strings there, since JSON object keys must be strings and this module keeps no opinion about output formatting.

Command dispatch, the mode-override rule, and why logs go to stderr

What does this page answer? How does `cli.py` turn `argv` into a dispatched command, what exactly does overriding `--mode log`, and why does structured logging go to `stderr` instead of the `stdout` the plan originally specified? Describes `proxmox_storage_drs/cli.py` and `proxmox_storage_drs/logging_setup.py`.

One parser, global options first

`build_parser()` defines every global option (`-c/--config`, `--group`, `--mode`, `--json`, `-v/--quiet`, `--version`, `--manual`) on the **top-level** `ArgumentParser`, not on the subparsers, which is what makes `argparse` itself enforce `IMPLEMENTATION_PLAN.md` section 11.3’s “global options before the subcommand” for free — there is no hand-written ordering check anywhere. `--help`’s defaults come from the real `default=` values passed to `add_argument`, combined with `argparse.ArgumentDefaultsHelpFormatter`; there is deliberately no second, hand-maintained list of options and defaults anywhere in this module, the manpage, or the manual (`AGENTS.md` section 8.5).

`main()` handles `--version` and `--manual/help` **before** requiring or loading any configuration — printing the version or the manual page must never depend on `/etc/pve/drs.yaml` existing. Only once a real subcommand is present does it call `config.load_config()`.

`-v/--verbose` is **otherwise purely a `logging_setup.py` concern** (see below) — every subcommand’s own `stdout` report is identical regardless of how many times it was given, with one deliberate exception: `_handle_explain()` reads `args.verbose` directly and adds a `data source:` line to its own report when set, naming the section 3.4 node-scoping filter and window/rate settings the run’s queries used (`docs/manual/29-explain.md`). This is the one place a global option’s effect is not identical across every command, because `explain`’s entire purpose is narrating *how* a result was produced — provenance the other commands’ reports have no occasion to print.

Command handlers: a dict, not a chain of ifs

`_COMMAND_HANDLERS: dict[str, CommandHandler]` maps each subcommand name to a function of `(ResolvedConfig, argparse.Namespace, effective_mode) -> int`. A subcommand not yet implemented gets a handler from `_make_not_yet_implemented_handler(name)`, a closure factory rather than a lambda with a default-argument trick, because `mypy`’s strict mode cannot infer a bare lambda’s parameter types cleanly here — see the comment at that function if you are tempted to simplify it back to a one-liner. Every subcommand in `_SUBCOMMANDS` now has a real handler (`explain` was the last one — see [./manual/30-safety-and-status.md](#)), so this factory is currently only the extension point for the next one: `_COMMAND_HANDLERS`’s dict-comprehension initializer still runs it for every name before each real handler overwrites its own entry, so a new command added to `_SUBCOMMANDS` without a handler assignment fails honestly instead of a `KeyError` from `main()`’s dispatch. `_COMMAND_HANDLERS["verify-metrics"]` is overwritten with the real `_handle_verify_metrics` once `metrics.py` exists to back it, and likewise for every other command as its own module landed — `apply`’s handler is `_handle_apply` (`execute.py/crashrecovery.py`); `explain`’s is `_handle_explain`, which runs the identical `_plan_group()` pipeline `plan` does and narrates

the pins, the fragmentation they cause, and the section 5.4 objective’s six terms that plan itself never prints. Adding a new implemented command is exactly this: write the handler, assign it into the dict, done — `main()`’s dispatch does not change.

--group: filtered once, right after build_topology()

`_filter_groups(topology, args.group)` is the one place `--group` is actually read. `show-load`, `verify-storages` and `plan` each call it immediately after `build_topology()`, replacing that `Topology`’s groups tuple with the (still topology-order) subset named — every render function downstream just iterates `topology.groups` as before and needs no `--group` awareness of its own. `verify-metrics` does not call it: that command validates configured metric/label names against Prometheus directly and never iterates groups at all, so there is nothing for `--group` to restrict there. A name that matches no configured group raises `DrsError` rather than silently producing an empty report — the same “an explicitly named thing that doesn’t resolve is a hard failure” rule `config.load_config()` already applies to `-c/--config PATH` (REVIEW.md R-03: `--group` was previously parsed and documented, but no handler ever read `args.group` at all).

The --mode escalation rule

`apply_mode_override(configured_mode, override)` is the entire implementation of section 11.3’s rule: moving *down* the ordering `dry-run < confirm < auto` (toward more caution) logs at `INFO`; moving *up* it (toward less caution — removing a barrier the operator’s own config set) logs at `WARNING`, naming both values. This is deliberately a small pure-ish function (its only side effect is the log call) rather than inlined into `main()`, so it is unit-testable against a `caplog` fixture without constructing a full CLI invocation.

Why logs go to stderr, not stdout

IMPLEMENTATION_PLAN.md section 2.1 originally specified `stdout` for structured JSON logs. This was amended in the same commit that added `logging_setup.py: --json` (section 9.5) emits the plan report on **stdout**, and interleaving log lines with that report on the same stream would make it unparseable by anything downstream. Logging instead goes to **stderr**; a `systemd` service unit captures both streams into the same journal regardless, so nothing about “captured by journald” is lost. This is the one place in the codebase where the implementation and IMPLEMENTATION_PLAN.md disagree with the *original* plan text on purpose — AGENTS.md section 7 rule 2 is why the plan text itself was corrected in the same commit rather than left to silently diverge from the code.

`configure_logging()` is called exactly once, from `main()` via `_start_logging_and_announce_run()`, after the `--version/--manual` early exits (which must not touch logging configuration at all) — and, since the section 2.3 rework, *after* `load_config()` rather than before it. That ordering is deliberate and slightly counter-intuitive, so it is worth stating why: the mandatory `INFO` floor depends on the effective execution mode, which is not known until the config is read, and `apply_mode_override()` itself logs, so it has to run after the level it should be logged at has been decided. The one thing that moves *before* logging exists as a result — a config file that could not be read or parsed — is reported with a plain `print()` to `stderr`, which is what it always was: a usage failure, not an event in a run that never started.

Levels, the mandatory floor, and why the handler is on root

IMPLEMENTATION_PLAN.md section 2.3 is the policy; `logging_setup.py` is three small pure functions plus the installer, so that every part of that policy is testable without running a command:

- `resolve_level()` — the `--quiet < default < -v < -vv` ladder, with `--log-level` winning over all of it.
- `floor_for_command()` — `INFO`, and only for `apply` in `confirm/auto`. This is the “a run that can change the cluster logs what it did whether or not anyone asked” rule, and it is a *floor* rather than an override precisely so `--quiet` can still win.

- `resolve_format()` — auto resolves to text at a TTY and json everywhere else. A systemd unit therefore gets JSON without the unit saying anything, and an operator at a terminal gets prose without passing a flag; that one rule is the entire fix for “why is my terminal full of JSON”.

The handler goes on the root logger, and the package logger carries only a level. The obvious-looking alternative — handler on `proxmox_storage_drs`, `propagate = False` — was tried first and is wrong twice over: it hides every record from `pytest`’s `caplog` (whose handler sits on root) and from any application embedding this package, and it needs a *second* handler on root anyway for third-party records to be visible at `-vv`. Setting levels instead gets the same separation with one handler: our records are filtered by the package logger’s level and then propagate to root’s handler, while `urllib3` and friends are filtered by root’s own level, which stays at `WARNING` unless `-vv` lowered it.

Because logging is process-global, `tests/conftest.py` restores the package logger’s level and root’s handlers around every test. Without it a test that runs a command leaks its verbosity into whatever runs next — which showed up exactly once, as a `caplog`-based topology test that passed alone and failed in the suite.

JsonFormatter: what ends up in one log line

Every field on a `logging.LogRecord` that is *not* one of the standard attributes (computed once, at import time, into `_STANDARD_RECORD_ATTRS`) is folded into the JSON payload — this is what lets any module call `logger.info(..., extra={"event": "gate_decision", "threshold": 0.2})` and have `event/threshold` appear as top-level JSON keys with no formatter changes required. `sort_keys=True` keeps output byte-stable for anything that diffs log lines in a test or a log-aggregation pipeline.

Two constraints on what may go in `extra=`. First, **every record carries an event** — the names are an interface (`jq 'select(.event=="move_started")'` is a supported way to use this tool), and `test_logging_setup.py` parses `src/` with `ast` to prove no call site forgot one. Second, `extra=` may not shadow a standard `LogRecord` attribute: `{"message": ...}` raises `KeyError: "Attempt to overwrite 'message' in LogRecord"` at runtime, which is how the deadlock record’s payload key came to be `detail`.

`TextFormatter` is the auto-at-a-TTY counterpart: `LEVEL: message` for anything above `INFO`, and bare prose at `INFO`, because prefixing every line of a requested narrative with `INFO:` is noise rather than information.

--manual: preferring `man(1)`, falling back only when necessary

`show_manual()` tries `man pve-storage-drs` first, via `subprocess.run` rather than `os.execvp`: `man(1)` itself detects a non-tty stdout and disables its pager automatically, which is what satisfies “never answer with a URL alone” (`AGENTS.md` section 8.5) whether the invocation is interactive or piped — there is no need for this module to duplicate that tty detection. `_fallback_manual_text()` is reached only when `man(1)` itself is missing or has no entry (an uninstalled source checkout, or a minimal system with no `man-db`): it walks up from `cli.py`’s own file location looking for `man/pve-storage-drs.1.md` in a source tree, and only if that also fails prints a short message naming the real installed-package path and the in-tree file — a local, actionable path, never a bare URL.

The Proxmox VE API client

What does this page answer? Why is `pve.py` built on `proxmoxer` instead of a hand-rolled ticket/CSRF client, and what does each of its methods actually call? Describes `proxmox_storage_drs/pve.py`.

Why proxmoxer

`pve.py` is built on `proxmoxer` rather than a hand-rolled ticket/CSRF client for its **backend abstraction**. The same `ProxmoxAPI` attribute-chaining interface (`api.nodes(node).qemu(vmid).config.get()`) is available

over plain HTTPS (what `build_client()` uses today) or over SSH, either shelling out to the system's own `ssh+pvesh` (`openssh`) or with an in-process client (`ssh_paramiko`). A deployment that cannot open the API port to the management host becomes a different keyword argument in `build_client()`, with no change anywhere else — not to `PveClient`'s methods, and not to any of their callers. (`IMPLEMENTATION_PLAN.md` section 3.5 has the full evaluation of alternatives that led here.)

`build_client()`: the one place auth is assembled

`config.py`'s `ProxmoxConfig` keeps `verify_ssl` (bool) and `ca_file` (path-or-None) as two separate knobs, because that is how an operator thinks about them; requests (and so `proxmoxer`'s https backend) wants one value, a bool or a CA-bundle path, passed as `verify`. `build_client()` is the one place that reconciles the two. It is also the one place `proxmox.auth.token_id`'s `user@realm!tokenname` format is split into `proxmoxer`'s separate `user/token_name` keyword arguments — never duplicate that parsing anywhere else.

Every failure `ProxmoxAPI(...)` itself can raise (`proxmoxer.AuthenticationError`, or a `requests.RequestException` if the host is simply unreachable) is caught here and re-raised as `PveApiError`, so every caller of `build_client()` and every `PveClient` method has exactly one exception type to catch.

PveClient: one method per section 3.5 endpoint

Every method is a single API call, wrapped through the private `_call()` helper, which is the one place `proxmoxer.ResourceException` (HTTP-level API errors) and `requests.RequestException` (transport failures `proxmoxer`'s https backend does not wrap itself) both become `PveApiError`. No method caches anything, and the per-run topology cache belongs to `topology.py` (section 3.5), which is also where the bounded thread pool for the O(VMs) config fetch lives (60-topology.md, REVIEW.md P-02).

`_call()` does retry exactly one failure mode: `proxmoxer.AuthenticationError` gets one reauthenticate-and-retry cycle (REVIEW.md P-01) before becoming a `PveApiError`, via a reauthenticate callback `build_client()` wires in — a full fresh login through `_build_api()` again, not a reuse of the rejected ticket. This is not a general retry policy (a `ResourceException` or a transport failure still fails immediately, on the first attempt); it exists specifically because a ticket can expire for reasons with nothing to do with the call that hits it — a long apply --confirm wait, a suspended process, a clock jump — and `proxmoxer`'s own lazy per-request renewal only notices the age of its own clock, not whether the server-side ticket actually outlived a gap that long. A test double built with a bare `PveClient(fake_api)` (no reauthenticate=) gets none of this — it behaves exactly as it did before P-01, which is what every existing fake in `tests/unit/fakes.py` relies on.

`build_client()` also applies `proxmox.ticket_refresh_seconds` to the constructed session, best-effort: `proxmoxer 2.x` has no constructor argument for a password/ticket auth's refresh interval (it is a hard-coded `renew_age = 3600` class attribute on `ProxmoxHTTPAuth`, checked lazily on whichever request happens to run after it elapses), so `_apply_ticket_refresh_seconds()` reaches into `api._backend.auth` — undocumented `proxmoxer` internals, not its public interface — to override that instance's `renew_age` after login. Deliberately narrow: it only ever *tightens* `proxmoxer`'s own schedule, never replaces its refresh logic, and if a future `proxmoxer` version reshapes this (or the backend is not https, or the auth is API-token, which has no ticket at all), the `getattr/hasattr` guards make it a silent no-op — the P-01 reauthenticate-and-retry above still catches the case this can't reach.

`storage_definitions()` deliberately uses `GET /storage` (the list form), never the singular `GET /storage/{id}`, even though only one storage's config is needed in some callers. This was verified empirically against a real PVE 9.2.11 cluster during development ([[dev-cluster-access]]) in the maintainer's notes, not reproduced here): an API token granted only `Datastore.Audit` can list every storage's

full config — including `saferemove/saferemove_throughput` — via `GET /storage`, but the identical data via `GET /storage/{id}` for the same storage returns 403 Forbidden (`Datastore.Allocate`). The single-item route apparently backs an edit-UI flow gated by the ability to change the config, not merely read it. Since the list form returns the same per-storage object for every storage in one call, there is no reason to ever call the singular form, and using it would quietly force a Storage DRS token to hold more privilege than the tool actually needs. `IMPLEMENTATION_PLAN.md` section 3.5 was corrected to match once this was found — see that section’s note.

`storage_content()` returned an empty list — not an error — on every storage, node and content-type filter tried, until the token also held `Datastore.Allocate` on that storage. This was genuinely surprising and worth calling out precisely because it is a *silent* false negative: `Datastore.Audit` alone gives a clean 200 with `{"data": []}`, which looks exactly like “this storage legitimately has no tracked content” rather than “the caller cannot see it.” It was not guessed at — see `IMPLEMENTATION_PLAN.md` section 3.5’s note, which records the exact before/after (0 entries with `Audit` only, 37 and 1 entries respectively for two real storages once `Datastore.Allocate` was added) on the same live cluster. **The practical consequence: `pve.py`’s own “keep the token to Audit-only” goal for `storage_definitions()` does not extend to `storage_content()` — a Storage DRS token genuinely needs `Datastore.Allocate` (bundled with `Datastore.Audit` in one role) on every storage it manages, or `topology.py` will silently compute `Usext` (section 5.1.1) as if every storage were empty of untracked volumes, with no error to notice by. `docs/manual/00-installation.md` states the exact minimum ACL grant an operator needs; this is not a place to economize on privilege out of a preference that turned out not to match reality.**

Two more things about Proxmox’s ACL model this surfaced, both now reflected in the manual’s token-setup instructions: **an API token’s effective permission is the *intersection* of its owning user’s permissions and whatever is granted to the token itself** when privilege separation is on (the default) — granting a role to only one of the two has no effect, both need it; and the grant that was confirmed to work was made explicitly at each `/storage/{id}` rather than only at the parent `/storage` path, so that is what the manual instructs, rather than relying on propagation that was not cleanly isolated as working or not.

`node_names()` uses `GET /nodes`, not the node fields already visible on `vm_resources()/storage_resources()`. Section 3.4’s PromQL node-scoping filter (`metrics.build_node_selector()`) is built from this list, and it needs *every* cluster node, not only the ones currently hosting a VM or a shared storage — an idle node would silently be missing from either of those, and dropping a real node out of the filter is exactly the kind of quiet under-count section 3.4 exists to avoid (a VM that migrated onto the missing node partway through the window would look like it has less history than it really does). Called once per command invocation (`cli._resolve_node_selector_for_run()`), not per group.

There used to be a `cluster_name()` method here (`GET /cluster/status`, `Sys.Audit` at /), backing a middle scoping tier in `metrics.resolve_node_selector()` that matched a cluster-naming label against this cluster’s own name. It was removed along with that tier – operator correction: the premise that this project’s deployments carry such a label “as standard practice” was wrong, so the extra `Sys.Audit`-gated API call it existed for bought nothing. See `docs/internals/30-metrics.md`’s node-scoping section.

`vm_config()` returns a key’s *pending* value, not the one in effect, once one exists — confirmed against a real cluster (`config/drs.yaml`’s dev cluster, VM 102): called with no current parameter, which is every call `vm_config()` ever makes, `GET .../qemu/{vmid}/config` merges in whatever change is queued for a key rather than showing what PVE is actually running right now. Only `GET .../qemu/{vmid}/pending` (`vm_pending()`, section 3.8) exposes both — each entry’s value is the real, in-effect one, and a `pending/delete` field appears only on a key with something queued. `topology.py`’s `pending_disk_reasons()` is what turns this into the section 5.3 (C2) pin (60-`topology.md`’s “The pending-change pin”); `pve.py` itself does nothing with the distinction beyond exposing both calls.

move_disk(): the one and only bytes/s -> KiB/s conversion

Section 9.2 is explicit that `bwlimit`'s bytes/s-to-KiB/s conversion happens “at the call site and nowhere else.” `PveClient.move_disk()` is that call site: every caller in this codebase, present and future, passes `bwlimit_bytes_per_sec` in the same unit `migration.bwlimit_bytes_per_sec` already uses, and the division by 1024 (rounded, since the API wants an integer) happens exactly once, inside this method. `format` defaults to `None` and is only ever forwarded when a caller explicitly passes one — `move_disk` without `format=` preserves the source format, which is what section 3.5 requires by default.

Building the disk/storage/group model

What does this page answer? How does `topology.py` turn `pve.py`'s raw API responses into the per-group D/S sets the rest of the engine needs, and where does each `IMPLEMENTATION_PLAN.md` section 5.3 (C2) pin condition actually get decided? Describes `proxmox_storage_drs/topology.py` and `proxmox_storage_drs/reserve.py`.

D means every disk this module places, pinned or not

This is stated at the top of `topology.py`'s own docstring because it is the single easiest thing to get backwards: `D` (section 5.1) is *every* disk this module puts into a group's `Group.disks`, whether or not **(C2)** pins its placement. **(C2)** is the plan's per-disk eligibility constraint: a pinned disk is still a full member of `D` — its bytes and its I/O still count — it is simply excluded from ever being *reassigned*; the “Pin priority” section below enumerates every condition that triggers it. A disk this module never sees at all — a stopped VM excluded by `exclude.running_only`, or a disk whose current storage is not in any configured group — is what “foreign” (`Usext`, section 5.1.1) means. Config-excluded disks (`exclude.vmsids/exclude.disks/tags`) are *not* foreign: **(C2)** pins them into `D` specifically so their bytes still count toward a storage's reserve check — **(C4)/(C5)**: `Zs`, the largest disk resident on a storage, sets a reserve floor `Rs = max(reserve_factors · Zs, softs)` that must stay free on top of every disk's actual usage there (`softs` is section 5.3.1's configured free-space requirement, resolved per storage onto `Storage.free_space_soft_bytes` below — the exact shortfall arithmetic is in “`reserve.py`: one **(C4)/(C5)** evaluator, shared” below) — and toward `κ`, the objective's per-VM fragmentation penalty, which charges a VM for every extra storage its disks are spread across (section 5.4; see `90-heuristic.md`) (section 3.6, “Pinned disks are modelled, not ignored”). An earlier draft of section 5.1.1 listed config-excluded disks as foreign, which directly contradicted **(C2)** — that self-contradiction was found and fixed in the same commit that first implemented this join; see that section's note if you need the history.

One pass, in the order section 3.5 lists

`build_topology()` fetches everything it needs exactly once, in six stages: `_fetch_cluster_data`, a concurrent per-VM fetch phase (`_fetch_vm()`) over `client.vm_resources()`, a concurrent per-(node, storage) content-fetch phase, the single-threaded `join(_join_vm_disks())` that consumes both, and finally `_build_storages`:

1. `storage_definitions()` (the list form — see `50-pve-api.md`), which is also what every `/.../` storage pattern (section 11.4, below) is matched against, then the configured groups are expanded and validated against it (`_expand_and_validate_groups`): a literal storage that does not exist, or that lacks images in its content types, is a fatal `TopologyError` (section 11.1: “storage ids exist in the cluster”). A storage that exists but is not marked shared is a warning, not a fatal error — plausible, if unusual, for a single-node group.
2. `storage_resources()` once, to pick one active node per storage (`_pick_active_node`, preferring one reporting status: “available”).
3. `storage_content()` and `storage_status()` per group storage.

4. `vm_config()`, `vm_snapshots()` and `vm_pending()` per considered VM. A VM is *not* fetched at all when `exclude.running_only` is set and it is stopped — the one case `cluster/resources`’s own fields can decide without a config fetch. Every other VM is fetched, including config-excluded ones: whether a specific disk turns out to be “ungrouped” is only knowable after seeing which storage it is actually on, and a config-excluded VM’s disk still needs its size for (C4)/(C5) (previous section).

This is the one step that runs across a bounded thread pool (`config.proxmox.read_workers`, REVIEW.md P-02): section 3.5’s whole point is that `GET .../qemu/{vmid}/config` has no batch form, so for a several-hundred-VM cluster this is the read phase concurrency actually helps. It is split into `_fetch_vm()` (network I/O only, run by the pool, one call per considered VM) and `_join_vm_disks()` (pure — the same function the old sequential `_collect_vm_disks()` was renamed from, unchanged in what it computes). `ThreadPoolExecutor.map()` returns each VM’s fetch in the same order `client.vm_resources()` listed it, even though the fetches themselves complete in whatever order the pool schedules them, so the join phase runs single-threaded and in the original order — `disks_by_group`, `referenced_volds` and `warnings` end up byte-for-byte identical to a fully sequential run regardless of `read_workers` or of thread scheduling, and never need a lock. `PveClient` itself may reauthenticate mid-fetch if several threads hit an expired ticket at once (50-pve-api.md’s P-01 note); that reauthentication is what its own lock serializes, not this join.

5. `storage_content()` again, per `(node, storage)` pair `_needed_content_node_pairs()` finds among the just-fetched VMs’ own disks that step 3’s single per-storage fetch does not already cover — see “Sizes” below for why a second per-node fetch is worth its own concurrent pool pass rather than reusing step 3’s listing for everything. Runs the same `ThreadPoolExecutor(read_workers)` pattern as step 4, between it and the join. Bounded by distinct `(node, storage)` pairs, not by VM count (REVIEW.md W-05): worst case (VMs spread across every node, disks on every managed storage) is $|nodes| \cdot |storages|$ extra full content listings, though a typical cluster’s VM placement is far more concentrated than that. IMPLEMENTATION_PLAN.md §3.5’s expected call-count formula carries the $|extra\ content\ pairs|$ term for this.
6. Non-QEMU resources (`type != "qemu"`) are skipped outright — this tool never touches LXC containers, and section 3.5’s read/write paths are qemu-only throughout.

The lock field comes from `vm_config()`, not a separate call

Section 9.3’s own pseudocode says `lock` is readable from either `/qemu/{vmid}/config` or `/status/current`. `_join_vm_disks` reads it from the config response already being fetched for the disk join, so planning-time lock detection costs no extra API call. `/status/current` is still needed later, but only immediately before a specific move (section 9.2’s pre-move re-validation, `execute.py`’s `_preflight()` — see 92-execute.md) — never here.

Pin priority: `_pin_reason()`

One function, checked in the fixed order section 5.3 (C2) lists its conditions: config exclusion (VM-level, then `exclude.disks`), a real snapshot or an unreferenced companion volume (section 3.7, `_disk_snapshot_or_orphan_reason` — the “current” entry `GET .../snapshot` always returns, verified on a live cluster, is filtered out before counting), an unapplied pending config change (section 3.8, below), the per-disk cooldown (below), a VM lock, then an unusedN disk when `exclude.include_unused_disks` is false. A disk gets at most one reason; the first that applies wins, matching how an operator would explain it.

The pending-change pin

`GET .../qemu/{vmid}/config(vm_config())` is confirmed, against a real cluster, to return a key’s **pending** value once one exists — not the value actually in effect. Only `GET .../qemu/{vmid}/pending(vm_`

`pending()` exposes both, so `_fetch_vm()` fetches it alongside `vm_config()/vm_snapshots()` for every considered VM, and `pending_disk_reasons()` (public, unlike its sibling helpers here — see below) reduces it to device -> reason for exactly the disk keys (matching `DISK_KEY_RE`) that carry a "pending" field (an edit) or a truthy "delete" field (a queued removal). `_join_vm_disks()` computes this dict once per VM, the same way it already does for `snapshot_reason`, and passes each disk's own entry into `_pin_reason()`.

Why this exists at all: `move_disk` acts on a disk's *current* volume, and PVE does not reconcile that disk's pending entry as a side effect of the move — the entry keeps describing a change relative to whatever the disk looked like before it moved. Migrating a disk in this state would leave a config key that is wrong the moment the operator's next reboot applies it. Section 3.8 has the full reasoning and the live-cluster confirmation.

`pending_disk_reasons()` is exported (not underscore-prefixed) specifically so `execute.py`'s `_preflight()` can call the identical parsing immediately before issuing a move (section 9.2 step 6, 92-execute.md) — one implementation, per `AGENTS.md` section 5, rather than a second /pending parser that could silently classify an entry differently from this one.

The per-disk cooldown pin

`build_topology()` takes an optional `state: State` and `now: datetime` (both default to "no cooldowns"/the real clock — see `docs/internals/15-state.md`) and computes `state.active_disk_cooldowns()` once per group, before the per-VM join loop, into `cooldowns_by_group: dict[str, dict[str, float]]` (bare `vmid:device` -> seconds remaining). Inside the loop, once a disk's `group_name` is known, `cooldowns_by_group[group_name].get(key, 0.0)` is passed to `_pin_reason()` as `cooldown_remaining_seconds` — a plain float, not a second `State` lookup inside that function, keeping `_pin_reason()` a pure decision over already-resolved flags exactly like every other condition it checks. The reported reason names how much of `gates.cooldown_per_disk` remains: `cooldown: moved recently, 1.0h left on gates.cooldown_per_disk`.

This pin is **not** exempted when the disk's storage is actively violating (C4)/(C5) — see `docs/internals/15-state.md`'s "Deliberately not implemented" section for why, and why that gap is bounded and safe rather than silently wrong.

Sizes: content is authoritative, config is the fallback of last resort

`_resolve_disk_size_and_format` looks up the volume in a content listing first (section 3.5: authoritative for what is actually allocated), in three tiers: the entry's own size if present; its approximate-size if size is not; the VM config's own size= (logged as a warning naming the disk and which of the two content-listing gaps applied) only when the entry has neither, or is missing from the listing entirely. Two independently verified reasons the config fallback is not merely defensive: the `Datastore.Allocate-vs-Audit` privilege gap (50-pve-api.md), and an unusedN volume, deleted directly on the storage backend outside Proxmox (confirmed with the cluster's operator), that PVE's own config still referenced and that PVE itself never noticed or refused to boot the VM over — `IMPLEMENTATION_PLAN.md` section 3.6's note. `_build_storages`'s equivalent sum over *foreign* (unreferenced) volumes uses the same size/approximate-size tiering, but has no VM config to fall back to for a size at all — it skips the volume instead (warning, undercounting the reserve) when neither is present. `_parse_pve_config_size_bytes` parses PVE's own config-file size suffixes (512G meaning binary GiB, no explicit i) — deliberately not `units.py`'s parser, which is for *this project's* config file, a different and coincidentally similarly-shaped format.

Which node's content listing, though, is not a detail this function gets to be careless about. Confirmed live, by the operator who hit it: PVE 9.2's qcow2-on-shared-LVM snapshot support means size for one of these volumes is only readable from the node that has its LV *active* — cheap on the node currently running the owning VM (PVE keeps it active there), expensive or blocked entirely on a shared storage's

other nodes, which report approximate-size instead (PVE's own answer from LVM metadata, without activating anything). `_pick_active_node()`'s one pick per *storage* — the node every other section 3.5 call uses — is not guaranteed to be that node. So a managed disk's size/format is resolved against a content listing fetched from **that disk's own VM's node** specifically (`_needed_content_node_pairs()`, step 5 above), not `_pick_active_node()`'s pick; only the *foreign*-volume sum in `_build_storages` still uses the storage-wide listing, since an unreferenced volume has no owning VM to pick a node from. Before this fix, a disk on this kind of storage could silently carry the storage's generic pick's approximate-size (or fall further, to a possibly-stale VM config size=) even when the exact figure was one call away.

U^se^xt: everything not referenced

Every disk actually placed into a group's D records its valid in `referenced_valids[storage_id]` as it is processed. `_build_storages` then sums the size of every content entry on that storage whose valid was never referenced — orphans, templates, other groups' foreign volumes, and disks of VMs this run never fetched (stopped-and-excluded, or ungrouped) all fall out of this one computation with no special-casing per category, which is exactly section 5.1.1's definition read literally.

Section 11.4: /.../ storage patterns are expanded here, once

`groups[].storages[].id` may be a /.../ pattern instead of a literal id. Expansion happens inside `_fetch_cluster_data`, right after `storage_definitions()` and `storage_resources()` both return — the latter's fetch was moved ahead of everything else that used to follow it (REVIEW.md U-01) specifically so expansion can check a pattern match against it — and is the one thing that runs *before* the ordinary literal-id validation described above. `_expand_group()` matches every pattern entry in a group against the sorted list of the cluster's storage ids with `re.fullmatch` (whole-id, case-sensitive: `/prod/` must not also catch `preprod`), then folds in the group's literal entries on top, a literal always overwriting whatever a pattern matched for that same id in a plain dict assignment — precedence is a dict overwrite, not a positional rule, so entry order in the file never matters.

Three things a pattern can get wrong are checked right there, before anything downstream sees the result, all in `_match_pattern_entries`: a pattern matching zero storages is a `TopologyError` (same treatment section 11.1 already gives a literal id that doesn't exist — a typo is the likelier cause); a matched storage that has a GET `/storage` definition but is reported active by no node — the shape of a disabled storage — is also fatal, naming the pattern and the storage (REVIEW.md U-01: a literal reference to such a storage already hit the same wall in `_pick_active_node`, just with a worse message, since that call is three steps downstream of expansion and has no way to know a pattern was ever involved; catching it here instead means the operator is told which pattern is responsible); and two *different* patterns in the same group matching the same storage is fatal too (`claimed_by`) — which entry's options should apply would be arbitrary. A fourth check, `_check_cross_group_uniqueness`, runs once every group has been expanded: it is what catches a pattern in one group and a literal (or another pattern) in a different group matching the same real storage — `config._check_group_storage_membership` cannot see that case at load time because it only ever compares two groups' raw entry *text*, and `"san-a"` and `"/san-.*/"` are different text even when they'd resolve to the same storage.

One deliberate asymmetry remains, called out explicitly in section 11.4's own closing paragraph: a pattern-matched storage does **not** get the literal path's “must have images in its content list” check. An operator who names a storage explicitly gets a hard error for a typo; a pattern that happens to pick up an ISO-only or otherwise unusable LUN is a balance-quality concern, not a safety one — (C2) still keeps any disk from ever landing there, and $u^* = L_s / c_s$ (section 4) simply divides by one more `c_s` than intended. The not-shared warning is *not* a second such asymmetry (an earlier draft left it literal-only, REVIEW.md U-03): it is computed once, after expansion, uniformly over every final member of the group regardless of whether a literal or a pattern named it — a warning carries no typo-safety rationale for staying quiet,

unlike the content check.

Every pattern’s expansion – the raw entry and the ids it matched – is logged at INFO (`logger.info(..., extra={"event": "storage_pattern_expanded", ...})`) and carried on `Topology.pattern_expansions` for `cli.py`’s `verify-storages` to print (section 3.5). `Topology.unmanaged_storage_ids` is computed once, right after groups – every cluster storage id `storage_definitions()` returned minus every id `storage_group_of` resolved to – and is the storage-level twin of the per-disk “ungrouped, not managed” warning `_join_vm_disks` already emits: it does not need a disk to actually be sitting on the storage to be visible.

By the time `_build_storages` runs, `data.expanded_by_group[group.name]` holds real storage ids only – it iterates that, not `group_cfg.storages`, so `Storage.id` and everything keyed off it (`storage_group_of`, `cooldown` keys, `state.json`) never carries pattern text. Nothing past `_expand_and_validate_groups` in this module, or in any module downstream of it, needs to know a pattern was ever involved.

reserve.py: one (C4)/(C5) evaluator, shared

`compute_reserve_status()` computes, for one storage: $used = \sum_{d \in \text{on } s} z_d + U^{s \times t}_s$ (every managed disk’s bytes plus the foreign/unreferenced bytes from “ $U^{s \times t}$: everything not referenced” above), $shortfall = \max(0, used + R_s - capacity_s)$, and `ReserveStatus.violated` is exactly $shortfall > 0$ – the boolean gates.py’s `reserve-override` gate reads directly (`docs/internals/80-gates.md`). It is deliberately its own module, not a method on `Storage`: `heuristic.py` needs to evaluate the identical formula against a *candidate* assignment, not only the current one (`storage_of=` overrides which storage each disk is treated as sitting on), and `pve-storage-drs show-load`’s reporting needs it against the current assignment today. Both call the same function (AGENTS.md section 5) – `show-load`’s use of it is already exercised end-to-end (see `cli.py`’s `_render_show_load_human/_json`). `optimize.py`’s MILP path needs the same (C4)/(C5) invariant but cannot call a Python function from inside a solver’s constraint system – see 91-optimize.md for how it encodes the equivalent bound directly as a scaled linear constraint instead. The section 14 worked example’s initial state (`tests/unit/test_reserve.py`) is the proof this reproduces the plan’s own arithmetic exactly, not merely a self-consistent unit test.

free_space resolution: soft_s/hard_s, resolved once, here

Section 5.3.1’s `soft_s/hard_s` pair is resolved onto `Storage.free_space_soft_bytes/.free_space_hard_bytes` in `_build_storages()`, by `_resolve_free_space()` – the same place and the same per-storage `pattern_resolve_reserve_factor()` already uses for `reserve_factor`, and for the same reason: a `/.../` pattern’s `free_space` entry applies to every storage it matches, a literal entry overrides it, and only a real storage’s `capacity_bytes` (known here, not in `config.py`) can resolve a percentage. The mandated order is inheritance, then percent-to-bytes conversion, then the two section 11.1 hard rules against the *written* values ($hard_s \leq soft_s$, $soft_s < C_s$ – both raise `TopologyError`, exactly like the pattern-expansion rules above, since both need the cluster inventory `config.py` never has), and only then the deprecated `snapshot_reserve.min_free_bytes` fold ($soft_s = \max(soft_s, min_free_bytes)$) – “validate as written, then fold”: folding first would let a written $hard > soft$ hide behind a large `min_free_bytes` and surface as a startup failure only once the operator deletes the deprecated key, exactly the upgrade path the fold exists to keep safe. A *null* `hard` is not a written value, so it is the one thing that is **not** settled before the fold: it means $hard_s = soft_s$, and that is the *folded* `soft_s`. Read before the fold it would leave $hard_s = 0$ for a config carrying only `min_free_bytes` and drop the deprecated floor from section 8.1’s transient charge without a word (a `min_free_bytes`-only config gets no deprecation warning); a *written* `hard` stays as written. `_resolve_free_space()` returns a `ResolvedFreeSpace`: the pair plus two display-only strings naming the level each half came from (global, storage entry, pattern `/re/`, percent-converted, folded), carried on `Storage.free_space_soft_source/_hard_source` for `verify-storages` and read by nothing else. By the time `compute_reserve_status()` reads `storage.free_space_soft_`

bytes, or `schedule.transient_invariant_ok()` reads `.free_space_hard_bytes`, both are plain, already-resolved byte constants – section 5.3.1’s own grammar (percentages, patterns, the deprecated key) is never seen again past this module.

(C2) format eligibility: `storage_type/allowed_formats`

Storage also carries `storage_type` (from `GET /storage`’s own type field, never guessed) and `allowed_formats` – the disk formats that type can actually hold, from a fixed table keyed by the same storage-type partition `_default_format()` already draws for the fallback format of a content listing with no format field of its own: a block-backed type (`lvmthin`, `zfspool`, `rbd`, `iscsi`, `iscsidirect`) holds raw only – there is no image container on the array, a volume is the raw block device – and a file-backed type (`dir`, `nfs`, `cifs`, `cephfs`) holds whatever `qemu-img` formats PVE offers a regular file for. Ordinary (non-thin) `lvm` is the one block-backed exception: current PVE versions also accept `qcow2` there, since PVE 9.2 added snapshot support on plain LVM by formatting the LV itself as a `qcow2` image rather than using it raw (the same mechanism `_resolve_disk_size_and_format()`’s approximate-size note above describes) – `lvmthin` needs no such carve-out, since its snapshots are native LVM-thin COW, never `qcow2-on-the-LV`. Section 5.3 (C2)’s “s cannot hold the disk’s format” is topology. `storage_accepts_format(storage, disk.format)` – one function, called by both MILP backends (`91-optimize.md`) and by `heuristic.py`’s own candidate-generating helpers to fix `x_{d,s}=0` (or exclude the pair from the neighbourhood the heuristic searches) for every ineligible target. Unlike the free-space floor, format eligibility needs no per-run resolution step of its own – it is a pure function of `storage_type`, computed once when `Storage` is built and read directly thereafter.

The load model

What does this page answer? How does `loadmodel.py` turn six raw Prometheus series into the single per-disk `l_d` section 4 defines, and what happens when the data for one disk cannot be trusted? Describes `proxmox_storage_drs/loadmodel.py`.

One function, one group, already-built input

`compute_group_load()` takes an already-built `topology.Group` (one group’s D/S, section 5.1) and a `metrics.PrometheusClient` and returns a `GroupLoad`: one `DiskLoad` per disk (`l_d`) and one `StorageLoad` per storage (`L_s`, `u_s`), plus the group’s `u*` and whether the whole group is idle. It never touches the PVE API and never builds topology itself – matching `reserve.py`’s identical “given the join, evaluate one formula” framing, not a coincidence: `cli.py` calls it exactly once per group, and the solver (`heuristic.py`’s `run_heuristic()`, via `GroupLoad.load_by_disk_key()`) consumes that same set of per-disk `l_d` values against every *candidate* assignment it considers, rather than recomputing load per candidate.

Six queries, not six-times-groups queries

The six raw quantities (section 3.4) are fetched with **one instant query each**, unfiltered by group – `sum by (vmid, device) (...)` already returns every disk Prometheus currently reports, and picking out one group’s keys in Python is free, whereas six *more* Prometheus queries per group is not. `read_time_ns/write_time_ns` are converted from nanoseconds to seconds here, engine-side (`_combined_raw`), for the same reason F-03 moved `read_factor/write_factor` engine-side: one tested constant beats the same `/1e9` repeated across deployed PromQL strings.

Known limitation, not a bug: a config with more than one group re-runs these same six unfiltered queries once per group – correct (each group’s coverage/rejection decisions are independent) but wasteful for a many-group cluster. Fetching once and slicing per group would be a straightforward follow-up if a real deployment’s group count ever makes this Prometheus load worth avoiding; nothing about `compute_group_load()`’s per-group signature forces the current one-fetch-per-call shape, it is just what the first implementation does.

Coverage rejection excludes a disk from the group total, not just from its own ℓ_d

Section 3.4: “reject a disk whose sample coverage over W is below `window.min_coverage` and fall back to its last known load from `state.json`... never treat missing data as zero load.” Two things worth being explicit about, since the plan states the rule but not this mechanism:

1. **The rejected disk’s raw contribution is excluded from $T_g/0_g/B_g$ entirely**, not merely capped or zeroed in place — one noisy or half-missing series must not bias every *other* disk’s normalized share. ℓ_d for every accepted disk is computed from the accepted-only totals.
2. **The rejected disk’s own ℓ_d is substituted afterward**, from `last_known_loads` if the caller has one (keyed by `topology.Disk.key`), or flagged `0.0` if not. `DiskLoad.flagged_reason` is set either way, so a caller can never mistake a flagged `0.0` for a genuinely idle disk — `show-load` renders it as an explicit Δ line, never silently.

`state.py` (section 11.2) now provides the real `last_known_loads` source: `cli.py`’s `show-load` and `plan` both pass `state.load_vector_for_group(state, group.name)` through unchanged, exactly the mapping this function already expected (see `docs/internals/15-state.md`). `compute_group_load()` itself needed no change at all — the parameter was shaped for this from the start. A group with no recorded balance yet (a fresh `state.json`, or none on disk) still gets `None`, and every coverage-rejected disk with no seed is flagged `0.0`, exactly as before.

tpmstate0/unusedN are never rejected; efidisk0 is

Section 3.4’s own note: `tpmstate0` and `unusedN` are not QEMU block devices and legitimately emit no `blockstat` series — $\ell_d = 0$ for them is correct, not a data-quality problem. `_is_metrics_expected_absent()` encodes exactly this device-name distinction so these two kinds are never flagged for low coverage, while `efidisk0` — a real QEMU drive that *should* appear — is held to the same `min_coverage` bar as `scsi/virtio/ide/sata`. Getting this backwards either way is a real bug: exempting `efidisk0` would hide a genuine metrics gap on it; not exempting `tpmstate0/unusedN` would spam a flag on every run for every cluster, for a “gap” that isn’t one.

$T_g = 0$ is “idle”, computed only from accepted disks

`GroupLoad.idle` is `True` exactly when every *accepted* disk’s `raw_t` is zero — the guard section 4 requires (“if $T_g = 0$ the entire group is idle, skip it”). An all-rejected group (e.g. a total Prometheus outage mid-window) also comes out `idle=True` by this same computation, which is not quite literally accurate (the truth is “unknown”, not “idle”) but is harmless: both mean “do not act”, which is the only decision that follows from either. A caller that wants to distinguish the two can — every disk’s `flagged_reason` is still there to check.

L_s/u_s are the *current* assignment, not a candidate one

`StorageLoad` sums ℓ_d over disks whose `Disk.current_storage` already points at that storage — today’s state, exactly like `reserve.py`’s `compute_reserve_status()`. $u_s = L_s / c_s$, where c_s is `storage.capability_weight` — a per-storage config knob (default `1.0`, see `config/drs.example.yaml`) an operator lowers to say “this storage should carry a smaller *share* of the group’s load than an equal-weighted one would,” e.g. `0.5` for a storage with half the spindles of its peers. u_s is therefore load-per-unit-of-declared-capability, not a byte-fill percentage — the gate/objective’s notion of “imbalanced” is about I/O share, not free space (that is $(C_4)/(C_5)$ ’s job, `docs/internals/60-topology.md`). `GroupLoad.average_utilization(u*)` is the group-wide counterpart, $(\sum_d \ell_d) / (\sum_s c_s)$ — total load over total declared capability across every storage in the group, section 5.3 (C6). The solver (`heuristic.py`’s `evaluate_assignment()`, `optimize.py`’s MILP encoding — see `90-heuristic.md/91-optimize.md`) re-sums the identical per-disk ℓ_d values against each *candidate* assignment it considers, using its own `storage_`

of() lookup in place of `Disk.current_storage`; `loadmodel.py` itself needed no change for that, since `l_d` does not depend on which storage a disk is currently on.

compute_disk_load_series(): the same blend, as a time series

`compute_group_load()` reduces each raw quantity to one already -quantile'd scalar per disk via `quantile_over_time` at query time. Section 10's forecaster needs the opposite: the raw per-disk `l_d` signal, unreduced, over a range and step of its own choosing (typically `forecast.required_range_seconds()`, not `window.lookback` — section 10.1 is explicit these are “genuinely different things”). `compute_disk_load_series()` fetches the identical six `rate(...)` expressions `_fetch_raw_quantity()` builds, `query_range`'d instead of wrapped in `quantile_over_time` and `instant_query`'d, then runs section 4's exact normalize-then-weight-then-rescale blend once **per timestamp** instead of once. `_combine_raw_values()/_blend_loads()` are that formula factored out to plain-number arguments so both this function and `compute_group_load()` call the identical implementation (AGENTS.md section 5) — extracted, then proven behavior-preserving by `test_loadmodel.py`'s full existing suite passing unchanged before any series-specific test was added, the same regression-by-refactor discipline `reserve.transient_charge_ok()`'s own extraction used.

Every disk in the group gets an entry, even an empty one — unlike `compute_group_load()`, this function does not apply `window.min_coverage` at all: a forecaster's own `required_range()` is a much longer, coarser signal than that rule was built to validate, and a sparse history is exactly what the forecaster itself needs to see to distrust its own fit. This is section 10's raw material for the section 7.3 saturation guard: `cli.py`'s `saturation_forecast_inputs()` calls it once per group, then feeds the result through `forecast.storage_upper_bound()` to get each move's `l_hat_src/l_hat_dst` — see [96-payback.md](#) for how `payback.py` uses those two figures to reject a move that would push a target storage's own forecast load past that storage's configured `saturation_load`.

What loadmodel.py is, and is not, responsible for

`loadmodel.py` is the load computation the rest of the engine consumes — `compute_group_load()`'s `l_d/L_s/u_s` feed `gates.py`'s drift/imbalance gates ([80-gates.md](#); `reserve.py`'s reserve override is computed separately, straight from the group's disks/storages, not through this module), and `compute_disk_load_series()` feeds the section 7.3 saturation guard above — but this module decides none of those verdicts itself. `pve-storage-drs show-load` reports `l_d/L_s/u_s` directly; deciding whether a group should be re-balanced at all is `gates.py`'s job, not this one's.

Gating: deciding whether to act at all

What does this page answer? How does `gates.py` turn a `loadmodel.GroupLoad` and per-storage `reserve.ReserveStatus` into the section 6 act/no-act verdict, and why isn't cooldown handling here? Describes `proxmox_storage_drs/gates.py`.

One function, three gates, in the order section 6 lists them

`evaluate_group_gates()` is pure — no network I/O, no state.json access — taking a group's already-computed `GroupLoad`, a `storage id -> ReserveStatus` mapping (the caller's job, via `reserve.compute_reserve_status()` — this module never recomputes it, per AGENTS.md section 5's “one implementation of every rule”), the group's `GatesConfig`, and the load vector recorded at the group's last *executed* balance (`last_load`, `None` if there isn't one). It checks, in section 6's own order, stopping at the first that decides:

1. **Reserve override.** `.violated` is `reserve.ReserveStatus.violated` — `True` exactly when a storage's committed usage plus its snapshot-reserve floor would exceed capacity (the shortfall arithmetic is `docs/internals/60-topology.md`'s “`reserve.py`: one (C4)/(C5) evaluator, shared”). Any storage in

reserve_statuses with .violated set forces act=True immediately, bypassing drift and imbalance entirely — section 13's "safety is not subject to hysteresis" made literal, not just documented intent.

2. **Drift gate.** $\|l_{\text{now}} - l_{\text{last}}\|_1 / \|l_{\text{last}}\|_1 \geq \text{gates.drift_threshold} - l_{\text{now}}/l_{\text{last}}$ are this run's and the last-executed-balance's per-disk load vectors (each entry one disk's l_d , section 4; see docs/internals/70-loadmodel.md), and $\|\cdot\|_1$ is the sum of absolute values over every entry. Vectors are aligned over the **union** of disk keys (a disk absent from one side contributes its full load in the other as drift — a new or deleted disk is a genuine change to the group's I/O profile). last_load=None skips this gate outright, per section 6's own degenerate-case table — not "treat as zero drift", which would make an operator's very first run fail to act on an already-imbalanced cluster.
3. **Imbalance gate.** $(\max_s u_s - \min_s u_s) / u^* \geq \text{gates.imbalance_threshold} - u_s$ is each storage's own fill fraction and u^* the group's average across storages (GroupLoad.average_utilization; both section 4, see docs/internals/70-loadmodel.md), so the gate fires once the spread between the fullest and emptiest storage, relative to the group's average fill, crosses the threshold. $u^* == 0$ (idle group, section 4) always means no-act — there is nothing to spread evenly.

last_load is real now, for show-load and plan

state.py (section 11.2) exists: cli.py's _last_loads_by_group() reads state.path once per run and hands each group's slice of last_balance.load_vector (re-keyed to plain topology.Disk.key, or None if that group has no recorded balance yet) to evaluate_group_gates() as last_load — both show-load and plan do this identically (cli.py calls the one function, not two copies). evaluate_group_gates() itself needed no change at all: it already took last_load as a parameter and treated None correctly (see docs/internals/15-state.md for the read side, and this page's next section for what is still missing).

This module only ever reads last_balance. gates.py itself never calls state.with_recorded_balance() — that write happens once, in cli.py's _handle_apply(), after any group whose plan actually executed at least one migration ("moved"/"draining" outcomes; see docs/internals/92-execute.md). Until a group's first apply run writes one, last_balance.load_vector for that group stays exactly what it was the last time a human (or a test) wrote it by hand; a config with no state.json on disk, or a fresh install, still behaves exactly as this page originally described — last_load=None, drift gate skipped, decision reduces to "reserve override, else imbalance".

show-load's gate line is diagnostic, not a real decision

pve-storage-drs show-load prints each group's verdict (Group <name> → ACT: <reason> / → NO ACTION: <reason>) reusing GateDecision.reason verbatim — deliberately the single source of truth for the wording, rather than a second, similarly-but-not-identically phrased string living in cli.py. Its verdict now reflects real drift history whenever state.json has one, exactly like plan's does — but show-load's printed verdict is still not what apply actually gates an execution on; it is plan/apply's own gate evaluation, run fresh at the moment a plan is built or applied, that is authoritative, not the possibly stale line show-load printed a moment (or a run) earlier.

Cooldowns are not this module's job

gates.cooldown_per_disk/cooldown_per_storage decide which *individual* disks or storages are movable once a plan is already being built — the same kind of decision (C2)'s other pin reasons make in topology.py ((C2) is the plan's per-disk eligibility constraint: a pinned disk cannot be reassigned at all, see docs/internals/60-topology.md), not a group-wide act/no-act verdict, so this module still has no cooldown logic of its own. Both are now implemented, in the modules this page always said they belonged with: the per-disk cooldown is a topology.py (C2) pin (docs/internals/60-topology.md), and the per-storage cooldown is a heuristic.py target-eligibility filter (docs/internals/90-heuristic.md), both reading state.py's cooldown data (docs/internals/15-state.md).

The heuristic solver

What does this page answer? How does `heuristic.py` turn a group’s loads into a target assignment, why is the section 5.4 objective its own function rather than baked into the search, and what does this module deliberately not do yet? Describes `proxmox_storage_drs/heuristic.py`.

`evaluate_assignment()` is the objective, kept separate from the search

Section 5.5 requires it explicitly: “the heuristic must use the **same** feasibility and objective functions as the MILP path so the two backends are directly comparable.” `evaluate_assignment()` takes a Group and an arbitrary candidate Assignment (`dict[Disk.key, storage id]`, not necessarily reachable from the current state by any sequence of moves) and returns an `ObjectiveBreakdown` — the section 5.4 objective’s five terms kept separate, not collapsed into a single number, because `explain` (not yet written) needs the arithmetic shown, and because the tests cross-checking this against section 14’s worked example need to assert each term individually, not just a total that could be right for the wrong reasons.

This is the same design choice `reserve.py` made first: a pure function of *data*, with no notion of “the current run” baked in. `heuristic.py` reuses `reserve.compute_reserve_status()` directly for (C4)/(C5), passing it a `storage_of` callback closed over the candidate assignment being evaluated — the extension point `reserve.py`’s own docstring already anticipated (“the solver will pass a candidate assignment through the identical shape”) before this module existed to use it.

Repair is unconditional, not weight-driven

Section 13: “the reserve is never traded against balance.” The heuristic makes this literal rather than merely well-weighted: `_repair()` runs *before* `_descend()` and fixes any (C5) violation by moving disks off the worst-violating storage, one iteration at a time, regardless of what the objective weights say — there is no `beta/gamma` value large enough to switch this off (`beta/gamma`, the objective’s per-move weights — `objective.beta_move_count` on how many disks moved and `objective.gamma_move_bytes_per_tib` on how many bytes moved — are the two terms a migration itself costs; see `objective.spread_metric` below for `alpha`, the imbalance weight, and “`w_v: kappa weighted...`” further down for `kappa`, the VM-affinity weight). `objective.reserve_violation_penalty` still appears in `ObjectiveBreakdown.reserve_penalty_term`, but only for *reporting* a residual that repair could not fix (physically impossible, not merely unattractive) — by the time `_descend()` runs, every reserve violation `_repair()` could resolve already has been, so in the common case this term is exactly zero and does not influence `descend`’s choices at all. `test_reserve_violation_is_repaired_even_with_beta_high_enough_to_forbid_balance_moves` is the regression test for this: `beta=100` (so no purely-cosmetic balance move could ever be worth a migration) still sees the repair move happen.

A candidate is judged by the group-wide total shortfall, not the source storage’s own. Moving a disk off a violating storage always reduces *that* storage’s own shortfall — it has fewer bytes, and if anything a smaller largest-disk requirement — which makes “does the source’s shortfall fall” a tempting but wrong test: it says yes even when the disk only relocates the problem to whichever storage receives it, or makes the group’s total worse. An earlier version of `_repair` used exactly that test and would oscillate a disk back and forth between two storages neither of which can fully hold it, “successfully” reducing the current worst offender’s shortfall on every iteration while never making net progress. Requiring the *group* total to strictly decrease fixes this and is also what makes the loop’s iteration bound (movable disks times storages, generously sized rather than tightly) actually a correct termination argument rather than a lucky one. `test_repair_does_not_oscillate_when_no_target_can_fully_absorb_the_violation` is the regression test: a 3 TiB disk that cannot fit, alone, on either of two 8 TiB/`reserve_factor: 2.0` storages is moved exactly once (repairing what can be repaired, from a 3 TiB group-wide shortfall down to 1 TiB), not shuffled back and forth forever.

Descend explores swaps, not only single moves

Section 5.5 is explicit that swaps are not an optional refinement: “when both storages are near their capacity limit, no single move is feasible, and only an exchange of two disks can improve the balance.” `_descend()` evaluates every single-disk move *and* every pairwise swap of two movable disks each iteration, applying whichever most reduces the objective, and stopping when nothing does or after `heuristic_` iterations. Both are plain re-evaluations of `evaluate_assignment()` against a trial dictionary — no separate swap-feasibility logic exists, because a swapped assignment is just another candidate the same objective function scores. `test_descend_uses_a_swap_when_no_single_move_is_feasible` constructs exactly the capacity-locked scenario the plan describes (two storages each with headroom smaller than any single disk) and checks the only reachable improvement — a swap — is the one found.

Descend also tries relocating a whole VM at once — a real gap, found by dogfooding

Neither of the two candidates above can reach a third kind of improving move: relocating *every* one of a multi-disk VM’s movable disks to one new storage, together. A single-disk move can’t get there when the objective’s κ (VM affinity) term is large enough to make moving just one of the VM’s disks a net loss on its own — it temporarily *fragments* the VM (paying κ) before a second move could reunite it elsewhere, and `_descend()` only ever takes a step that is itself improving. A swap can’t either: it exchanges two disks’ positions with *each other*, never relocates a whole disk set to a third storage together.

This was not a theoretical gap — it was found by testing against a real production cluster (not by review): a two-disk VM sat entirely on one storage that also carried a large amount of *pinned* (immovable) load, with `kappa_vm_affinity` at its default 0.50. Every individual disk move `_descend()` tried was a net loss once fragmentation was paid for, so it found *zero* improving moves at all and left the cluster at its full, 196%-imbalance starting point — even though CP-SAT, given the exact same data, found the two-disk relocation immediately. A dependency-free heuristic that can get stuck this badly on a real, unremarkable cluster undermines the one thing section 5.5 asks it to be: “the path for very large groups,” reliable with no optional solver installed at all.

The fix adds a third candidate family, tried in the same “evaluate everything, apply whichever wins” step as the other two: `_vm_relocation_candidates()` groups movable disks by `vmid` (skipping any VM with only one movable disk — that case is already the plain single-move candidate), and for each such VM, `_descend()` tries moving *all* of its disks to each other storage in one trial. Scored by the exact same `evaluate_assignment()` call as everything else, so it can never itself choose a worse assignment, and it respects `cooldown_storages` as a destination exclusion the same way the other two candidates do. `test_descend_relocates_a_whole_multi_disk_vm_neither_single_moves_nor_swaps_can_reach` reproduces the production scenario’s numbers by hand (a single move raises the objective by +0.43; the joint move lowers it by -0.14) and confirms `_descend()` only finds the improvement once this candidate exists. `_best_of()/_single_move_trials()/_swap_trials()/_vm_relocation_trials()` are the flake8-complexity-driven extraction that let all three candidate families share one “keep whichever trial scores lowest” loop rather than three copies of it.

The storage cooldown excludes a destination, never a source

Section 6: “a storage involved in a migration within `cooldown_per_storage` accepts no new incoming moves.” `run_heuristic()`’s `cooldown_storages` parameter — a plain `frozenset[str]` its caller (`cli.py`, via `state.active_storage_cooldowns()`) computes, not a `state.State` this module reads itself — is threaded through to `_descend()` only. Both of `_descend()`’s neighbourhoods respect it: the single-move loop skips any `target.id` in `cooldown_storages`, and the swap loop skips a pair whenever *either* storage a disk would land on is in cooldown (a swap always sends one disk to each of the two storages it touches, so either side landing on a cooldown storage blocks the whole swap). A disk already resident on a cooldown storage is always free to move away from it — the rule is about new arrivals, not existing occupants —

which is why the filter only ever appears on the destination side of a candidate, never the source.

`_repair()` never receives `cooldown_storages` at all — a deliberate exemption, not an oversight, matching this module's own repair-is -unconditional stance above: a storage actively needed to resolve a live (C4)/(C5) violation is not deferred for having been written to recently, the same reserve-override principle `gates.py` and `payback.py` already apply to the drift/imbalance gates and the payback test respectively. `test_run_heuristic_repair_ignores_storage_cooldown` is the regression test: a violation whose only viable full repair target is in `cooldown_storages` is still repaired. `test_descend_blocks_new_arrivals_onto_a_cooldown_storage` and `test_descend_still_allows_a_disk_to_move_away_from_a_cooldown_storage` cover `_descend()`'s own two sides of the rule, both against the section 14 fixture.

objective.spread_metric: two different quantities, not one rescaled

Section 5.4/(C6) offers two imbalance forms, and `evaluate_assignment()` implements both: "l1" (default) is $\alpha * \sum(e_s)$, every storage's absolute deviation from u^* summed; "minmax" is $\alpha * \max(u_s)$ — the plan's own $t \geq u_s$ for all s , the raw utilization of the single hottest storage. These are not the same quantity with a different aggregation: $\max(u_s)$ and $\max(e_s)$ can disagree, because a storage sitting far below u^* produces a large deviation e_s that minmax was never meant to notice (the manual's own words: "indifferent to a second nearly-as-bad storage" — nearly-as-bad on the *high* side; a cold storage is not what minmax was designed to react to at all). `ObjectiveBreakdown` carries both `spread_e` (deviations) and utilization (raw u_s) regardless of which metric is active, computed once in the same loop, so switching metrics never means computing something the other mode didn't already have on hand.

REVIEW.md's Q-01 found this config knob silently ignored by an earlier version of this module (always L1, `objective.spread_metric` never read) — the same class of gap P-01/P-02 found in `pve.py/topology.py`. `test_spread_metric_minmax_is_indifferent_to_a_second_nearly_as_bad_storage` is the regression test, built directly from the manual's own claim: two scenarios with an identical hottest storage but a materially different second-worst one score identically under minmax and differently under l1.

Proof this reproduces the plan, not just itself

`tests/unit/test_heuristic.py` doesn't stop at "the assignment matches the prose." At `delta_capacity_spread: 0` it asserts `ObjectiveBreakdown.total` equals 3.384685 (three-move plan) and, at `beta_move_count: 0.50, 4.009685` (two-move plan) — the exact totals section 14.3's reworked arithmetic gives once kappa is weighted by w_v (below), not values this module invented and then asserted against itself. Getting both to six decimal places is strong evidence the objective's six terms, their units (TiB for size, average in-flight I/O for load), and the search that picks among them are all correct together, not merely internally consistent.

w_v: kappa weighted by a VM's own I/O, and D^{big}

Section 5.4's two strengthenings of the affinity term, added after live dogfooding found a plan whose entire value was reuniting VMs rejected by `payback.py`'s aggregate test (the plan whose entire value had nowhere to be counted — see 96-payback.md). `compute_vm_weights()` computes $w_v = \max(1, l_v / l_{\text{bar}})$ once per group — l_v a VM's total load across every one of its disks (pinned included), $l_{\text{bar}} = T_g / |V|$ where T_g is the group's total load and $|V|$ is the count of **every** distinct vmid with a disk in the group, not only the (possibly narrower) set of vmids the kappa sum itself ranges over once `objective.affinity_counts_pinned_disks` has excluded a pinned-only VM from it — section 14.7's own worked number ($l_{\text{bar}} = 6.0/3$, not $6.0/2$, with VM 309 pinned out of the sum but still counted here) is what caught this the first implementation got wrong. `evaluate_assignment()` folds w_v into `fragmentation_term` in place of the old flat per-VM count; both MILP backends (`optimize.py`) import the identical function so a per-vmid coefficient is folded into their own kappa term the same way, rather than reimplementing the weighting twice (AGENTS.md section 5).

`D^big = {d : z_d >= migration.tiny_disk_bytes}` (default 64 MiB) is the second change: `move_count_term/bytes_moved_term` sum only over moved disks at or above that threshold, so an `efidisk0/tpmstate0` reunion costs nothing on the beta/gamma side of the ledger — `moved_disk_keys` itself is untouched, so a tiny disk still shows up as a real move everywhere else (reporting, `schedule.py`'s ordering, `payback.py`'s move list). `ObjectiveBreakdown.affinity_debt` (the raw, `w_v`-weighted-but-not- κ -scaled A) and `raw_affinity_debt()` exist purely so `payback.py`'s benefit can consume the identical quantity κ charges here, never a second, independently-derived one (REVIEW.md R-01's discipline, extended to the third term).

What this pass deliberately does not do

- **“Polish” (heuristic step 4)** — reuniting a fragmented VM when doing so does not worsen imbalance beyond `gates.imbalance_threshold`. Not implemented: the section 14 fixture's exact three-move and two-move solutions are both reachable by `repair+descend` alone (proven by the tests above), because the objective's own κ term already makes `descend` prefer co-location whenever it is not too costly, and (since the fix above) `descend`'s own whole-VM co-relocation candidate now reaches the two-or-fewer-disk case directly. What's left for polish is strictly narrower than it once was: an *N-way rotation* across three or more storages (disk A needs $S1 \rightarrow S2$, disk B needs $S2 \rightarrow S3$, disk C needs $S3 \rightarrow S1$ in a cycle) where no disk's own move improves alone and no two disks share a VM — outside every candidate `_descend()` tries today. The one fixture that exists to validate this module does not exercise it. A real gap, tracked here rather than silently absent.
- **(C2) format-compatibility eligibility** — `topology.Storage` does not yet carry the storage type/format information that rule needs (it is resolved internally in `topology.py`'s `_default_format` but never exposed on `Storage`), so every group storage is currently an eligible target for every movable disk. The section 14 fixture is homogeneous (three identically-typed storages) and does not exercise this gap either. `heuristic.py` is wired into `cli.py`'s `plan` command, together with `schedule.py` (section 8's move ordering) — see 95-schedule.md and docs/manual/27-plan.md. `apply` is fully implemented (see 92-execute.md); `explain` (section 12) reuses the identical plan pipeline (`cli._plan_group()`) and narrates `ObjectiveBreakdown`'s six terms individually — the reason this dataclass keeps them apart instead of collapsing to `.total` in the first place (its own docstring) — alongside which disks are pinned and which VMs that leaves fragmented across more than one storage. See 40-cli-and-logging.md and docs/manual/30-safety-and-status.md.

best_single_disk_alternative(): why explain says nothing moved

Added after dogfooding against a real cluster where the gate decided ACT (every disk eligible, nothing pinned) and the solver's own optimum was still to move nothing — the exact “ κ outweighs the fix” shape the whole-VM co-relocation fix above exists to handle when it goes the *other* way: here, only a two-disk VM's split could meaningfully help, and `objective.kappa_vm_affinity`'s penalty for that split legitimately cost more than the imbalance was worth. `plan`'s one-line verdict cannot tell this case apart from “the solver never tried” from the outside.

`best_single_disk_alternative()` closes that gap for `explain` specifically (never called from `plan`'s own path — it exists purely to explain a decision already made, not to make one): it evaluates every (movable disk, other group storage) pair with the same `evaluate_assignment()` the solver itself uses — section 5.3's one-move neighbourhood, exhaustively rather than sampled, since a group's disk-count $\times$ storage-count is always small enough for `explain` (unlike `plan`, which must stay fast enough to run every cycle) to afford it — and returns the one with the lowest resulting objective total, i.e. the move closest to having been worth taking. `cli._render_no_moves_lines()` prints its `ObjectiveBreakdown` alongside the baseline's, term by term, so an operator can see exactly which term(s) rejected it and, if they disagree with that tradeoff, which weight to change. See docs/manual/29-explain.md's “no moves made: ...” section for the rendered example and IMPLEMENTATION_PLAN.md section 9.5's “As built, added later” note.

The MILP backends: CP-SAT and CBC

What does this page answer? How does `optimize.py` turn section 5.3's constraint set into a real CP-SAT/CBC model, why is the reserve solved in two lexicographic stages rather than one big-M objective, and what does `cli.py`'s backend dispatch actually do when a MILP solver is unavailable or fails? Describes `proxmox_storage_drs/optimize.py`.

Both backends solve, then hand off to `heuristic.evaluate_assignment()`

`solve()` builds a model, extracts the winning $x_{\{d,s\}}$ assignment, and immediately calls the *same* `heuristic.evaluate_assignment()` the heuristic backend uses to produce the reported `ObjectiveBreakdown` — one implementation of the section 5.4 objective, shared by every backend (AGENTS.md section 5; `heuristic.py`'s own module docstring was written anticipating exactly this: “built specifically so `optimize.py` can call the identical function”). A modeling mistake in this module can therefore make the solver choose a *suboptimal* assignment at worst — it can never make the tool *believe* an assignment is better than it is, since the number that ends up in `plan`'s output is never this module's own objective value.

Lexicographic, not big-M — and why that is the only mode implemented

Section 5.3 describes two ways to keep a reserve violation from being “traded away” by a large enough balance gain: a single-stage objective with a calibrated penalty P (big-M), or a two-stage **lexicographic** solve — minimize $\sum r_s$ alone first, then fix that total as a hard constraint and minimize the real objective. `optimize.py` implements only the second, both because the plan itself says to (“use the lexicographic solve by default... needs no calibration, its correctness does not depend on T_g ”) and because there is no `solver.*` config knob that ever selects big-M at runtime — it exists in the plan text to be *compared against* the lexicographic solve, a comparison tests/fixtures/generate_expected.py already does by exhaustive enumeration on `reserve-tradeoff.yaml`, not something this module needs a second code path for. $\sum r_s > 0$ in a lexicographic result provably means *no* assignment could do better, at any weight — never merely “unattractive”.

That proof depends on stage 1 actually being solved to proven optimality, not merely to `solver.mip_gap` (REVIEW.md S-07). An earlier revision applied the same `mip_gap` to both stages: CP-SAT's stage 1 accepted FEASIBLE (an incumbent, not a proven minimum) whenever the gap was satisfied, and PuLP's own `LpStatus` string is “Optimal” whether CBC proved the bound or merely stopped because `gapRel` was satisfied — either way, stage 2 then pinned $\sum r_s$ to that unproven incumbent (`==`), which can both accept a shortfall up to the gap *and* forbid finding less of it. Fixed by forcing stage 1's own gap to 0 in both backends regardless of the configured `solver.mip_gap` (which now applies to stage 2's real objective alone), requiring CP-SAT's stage 1 status to be exactly OPTIMAL (never FEASIBLE), and changing stage 2's slack constraint from `==` to `<=` in both backends — monotone-safe (stage 2 can never do worse than the now-proven stage-1 value) rather than brittle against a hypothetical disagreement between the two solves. Cheap in practice: stage 1 is a near-feasibility problem that closes instantly on realistic groups, so forcing an exact proof costs nothing measurable.

CP-SAT: section 5.5's exact integer scaling

Every coefficient is folded and rounded exactly as section 5.5 specifies:

- `_LOAD_SCALE` ($K = 10^6$) scales every load-valued quantity (e_s, t, u^*). (C6)'s per-(disk, storage) coefficient is $a_{\{d,s\}} = \text{round}(K \cdot l_d / c_s)$ — folded and rounded *once*, per the plan's own warning against computing $\text{round}(K/c_s)$ and multiplying separately (which rounds the capability weight itself and would make CP-SAT and CBC disagree for a non-integer c_s).
- `_WEIGHT_SCALE` ($W = 10^4$) scales every objective weight ($\alpha/\beta/\gamma/\kappa$ — section 5.4's imbalance, move-count, moved-bytes and VM-affinity weights, `objective.alpha_spread/beta_move_count/`

`gamma_move_bytes_per_tib/kappa_vm_affinity` in config, the same four heuristic.py computes ObjectiveBreakdown from — see 90-heuristic.md). The γ term folds z_d into its own coefficient ($\text{round}(\gamma \cdot W \cdot K \cdot z_d^{\text{TiB}})$) rather than factoring out a standalone $\gamma_{\text{scaled}} = \text{round}(\gamma \cdot K / 2^{20})$ — the plan’s own worked example of why that shortcut is wrong: at the default $\gamma = 0.05/\text{TiB}$, it rounds to zero and CP-SAT would silently stop caring about disk size when choosing what to move.

- Every size-valued quantity ($Z_s, R_s, r_s, z_d, C_s, U^{\text{sect}}, \text{soft}_s/\text{hard}_s$) is a whole-MiB integer (`_mib()`), already exact, needing no scale of its own. `soft_s` is storage. `free_space_soft_bytes` — resolved once per storage by `topology.py` (60-topology.md), never a scalar threaded in from config; `r[s.id] >= _mib(s.free_space_soft_bytes)` is the model’s other one-sided bound on R_s , alongside the `reserve_factor ·  $Z_s$` one below.
- `_RESERVE_FACTOR_SCALE` is this module’s own addition, not named in the plan: $(C5)$ ’s $R_s \geq f_s \cdot Z_s$ multiplies a *variable* (Z_s) by `reserve_factor`, which section 5.5’s “fold constants into a coefficient” recipe does not directly cover (that recipe is for a constant multiplying a *binary* decision variable). Expressed instead as $\text{SCALE} \cdot R_s \geq \text{round}(f_s \cdot \text{SCALE}) \cdot Z_s$ with $\text{SCALE} = 10^6$ — a single linear constraint, correct to within $1/\text{SCALE}$ MiB, utterly negligible next to any real disk size.

Section 5.5 also asks for two build-time assertions guarding this folding, and `_cpsat_objective_terms()` now has both (REVIEW.md S-09): `_assert_nonzero_when_weighted()` catches exactly the γ trap the paragraph above describes — a non-zero configured weight whose *rounded* coefficient collapsed to 0 (only ever a sub-kilobyte disk at the default γ , per `test_gamma_trap_assertion_fires_end_to_end_for_a_sub_kilobyte_disk`) — and `_assert_objective_magnitude_within_int64()` checks a coarse, deliberately conservative worst-case sum of every term against 2^{62} , an order of magnitude under CP-SAT’s own $2^{63}-1$ domain limit. Neither applies to CBC, whose continuous, unscaled model has no analogous overflow risk.

`AddHint(x[d,  $\sigma_0(d)$ ], 1)` warm-starts every candidate from the current assignment, in both stages (harmless when unused, and stage 1’s own optimum is frequently “keep everyone where they are” when nothing already violates (C5)).

CBC: “direct transcription”, deliberately not scaled

The plan’s own words for this backend: “continuous e_s, Z_s, r_s are fine.” `_cbc_feasibility_constraints()/_cbc_objective_terms()` write the identical constraints CP-SAT does, but with plain floats — no `_LOAD_SCALE/_WEIGHT_SCALE` anywhere, because CBC needs none of CP-SAT’s integer-coefficient discipline. `solver.mip_gap` and `solver.time_limit_seconds` map onto `pulp.COIN_CMD(gapRel=..., timeLimit=...)` directly.

Every variable is built with `_lp_variable()`, a thin wrapper around the direct `pulp.LpVariable(name, ...)` constructor — **not** `prob.add_variable(...)`, PuLP v4’s replacement, despite an earlier version of this module having switched to it (REVIEW.md S-01, found by running the CBC tests against the exact pulp version Debian trixie packages). `LpProblem.add_variable` does not exist before pulp 3.3.1; trixie packages 2.7.0, and section 2.1 designates CBC-through-python3-pulp as *the* packaged solver path, so the “modernize to v4” choice crashed on the platform this project targets, while working fine — and passing every test — against a hand-picked newer pulp installed by hand. Direct `LpVariable(...)` construction works unchanged across every pulp version from 2.7.0 through 3.3.2 (bisected by wheel inspection); on 3.3+ it also raises a `DeprecationWarning` recommending `add_variable`, which `_lp_variable()` suppresses explicitly with a `warnings.catch_warnings()` block — the warning is real, but the alternative it recommends is the one that cannot run on the deployment target, so silencing it is the correct fix, not a workaround. `_pulp_solve()` similarly turns a `PuLPSolverError` — raised when pulp is importable but the cbc binary either is not on PATH or fails to execute — into this module’s usual `None`, matching `solve()`’s “never raises, cli.py’s fallback cascade handles it” contract instead of a bare traceback. That scenario is not the Debian package split an earlier version of this page claimed: python3-pulp’s own control file already Depends: coinor-cbc (confirmed by `rmadison/apt-cache show` against trixie, not just assumed), and

debian/control here now makes both Depends of pve-storage-drs itself too, so a Debian install cannot end up with one and not the other short of a deliberately broken one. The catch earns its keep from the two cases that remain: a non-Debian pip install pulp with no system cbc at all, and a cbc invocation that fails for some other reason (a malformed model, a killed subprocess) — either way, still “this backend cannot produce a plan”, never a bare traceback.

kappa*w_v and D^big: the same weighting in both backends

Section 5.4's $w_v = \max(1, l_v/l_{\text{bar}})$ is data, not a variable — computed once per group by `heuristic.compute_vm_weights()` (imported by this module, never recomputed here) and folded into a **per-vmid** kappa coefficient, exactly like u^*/b_{bar} are already folded elsewhere in this file. CP-SAT builds `kappa_scaled_values: dict[int, int]` in place of the old single flat `kappa_scaled` — `round(kappa * W * K * w_v)` per vmid, each still guarded by `_assert_nonzero_when_weighted()` — and `_assert_objective_magnitude_within_int64()`'s own worst-case bound sums `abs(k)` over that dict instead of multiplying one flat value by `num_vmid`s. CBC does the continuous equivalent: $\text{kappa} \cdot w_v$ multiplies each vmid's own `pulp.lpSum(y[v, s.id] ...)` term directly.

$D^{\text{big}} = \{d : z_d \geq \text{migration.tiny_disk_bytes}\}$ restricts beta/gamma to movable disks at or above the configured threshold — not a coefficient of 0 for an excluded disk, but the disk's terms skipped entirely (so `_assert_nonzero_when_weighted()` never fires for one; a deliberately excluded coefficient is not the rounding-to-zero bug that assertion exists to catch). Both changes are exercised end to end, not just at the coefficient level, by `test_optimize.py`'s `test_tiny_disk_bytes_lets_a_tiny_disk_reunite_with_its_vm_for_free` — a 1 MiB `efidisk0` that is not worth a full beta migration at `tiny_disk_bytes: 0` becomes free to reunite with its VM once the threshold covers it, on both backends — and by `tests/unit/test_affinity_repair_fixture.py`, which runs section 14.7's whole fixture through `solve()` for both backends and confirms they agree with the heuristic.

(C1)/(C3)/(C4)/(C5) are shared code, factored once per backend

`_cpsat_feasibility_constraints()/_cbc_feasibility_constraints()` build exactly the constraints both lexicographic stages need identically — only the *objective* differs between stage 1 (`Minimize( $\sum r_s$ )`) and stage 2 (the real section 5.4 objective, with $\sum r_s$ fixed to stage 1's result as a hard constraint). Each is called twice, with fresh variables each time, rather than trying to mutate one model's objective in place — simpler to reason about than partial reuse, at the cost of building the constraint set twice per solve (irrelevant next to a MILP solve's own running time).

(C3)'s $y_{\{v,s\}}$ linking honors `objective.affinity_counts_pinned_disks` exactly as the plan's own text describes it: **false** (default) links y to x over D^{mov} only, so a pinned disk never forces $y_{\{v,s\}}=1$ for its storage; **true** ranges the constraint over all of D , which this module implements as forcing $y_{\{v,s\}}=1$ outright wherever a pinned disk of v already sits (the natural reading of “ $x_{\{d,s\}} \leq y_{\{v,s\}}$ for $d \in D$ ” when $x_{\{d,s_0(d)\}}$ is a constant 1, not a variable, for a pinned disk).

cli.py's dispatch: auto cascades, an explicit backend still falls back

`cli._solve_group()` implements `solver.backend`'s four values: “auto” tries `cpsat` then `cbc`; “`cpsat`”/“`cbc`” try only that one; “heuristic” skips `optimize.solve()` entirely. Whichever backend `solve()` cannot use (library not importable, or no feasible solution within `solver.time_limit_seconds`) returns `None` — never an exception — and `_solve_group()` moves to the next one in the cascade, ending at `heuristic.run_heuristic()` if every MILP attempt failed. This applies **even to an explicitly forced backend**: section 13's failure-mode table says “solver infeasible or timing out -> fall back to the heuristic; never emit a partial/unvalidated assignment”, with no carve-out for `solver.backend: cpsat` specifically. The one difference: falling back from an *explicit* backend logs a warning (an operator who named `cpsat` to reproduce a specific result should not have to diff - - json output to notice auto quietly ran instead);

falling back within auto itself is expected and silent. `plan`'s human and `--json` output both report which backend actually produced each group's plan (`solver: cpsat (optimal) / "solver_backend": "cpsat", "solver_status": "optimal"`), precisely so that distinction is never hidden.

The storage cooldown: enforced in both stages, both backends (REVIEW.md S-04)

`_cpsat_feasibility_constraints()`/`_cbc_feasibility_constraints()` both take `cooldown_storages` now and add one constraint per `(d, s)` pair where `s` is in it and is not `d`'s current storage: $x_{\{d,s\}} = 0$. Applied inside the shared feasibility-constraint builder, it lands in *both* lexicographic stages automatically (built once per stage, per `solve()`'s own architecture) — a disk cannot be assigned to a cooldown storage if it is not already on, in either the reserve-minimization stage or the real-objective one, matching section 6's "accepts no new incoming moves" and `heuristic._descend()`'s own identical `target.id in cooldown_storages` check. A disk already resident on a cooldown storage is left free to leave it, or to stay — the cooldown only ever blocks an *arrival*.

An earlier revision of this module left the parameter accepted but unenforced, reasoning that cooldowns were inert anyway since nothing wrote `state.json`'s cooldown timestamps yet. That reasoning stopped holding the moment `execute.py/apply` (phase 7) started calling `state.with_recorded_cooldown()` — from that point on, `solver.backend: auto`'s default cascade to CP-SAT/CBC could plan straight through a cooldown the heuristic would have respected, silently disagreeing with it on the exact case the cooldown exists for. Fixed as above.

Not replicated: `heuristic._repair()`'s own exemption from this same cooldown when fixing a live (C4)/(C5) violation (section 13's reserve-override principle — a repair move is never deferred for a cooldown). The fix here is one constraint applied uniformly to both solve stages, including stage 1's reserve-shortfall minimization itself, so a cooldown storage stays excluded even when it is the *only* way to resolve a violation — narrower than the heuristic's behaviour in that one specific edge case (a group simultaneously mid-violation and mid-cooldown on its one viable target). Reported honestly rather than silently: stage 1's own slack is genuinely 0 achievable-within-constraints with the cooldown storage excluded, so `plan` still reports an accurate number, just a more conservative one than the heuristic would compute for the identical input. Replicating the exemption exactly would need a second, repair-only relaxation of the same constraint or a restructured two-phase solve — a real, separately-scoped piece of work.

(C2) format-compatibility eligibility is implemented, in the same place and the same way as the cooldown fix just above: `_fixed_zero_pairs()` is the one generator both `_cpsat_feasibility_constraints()` and `_cbc_feasibility_constraints()` call to decide which `(disk, storage)` pairs get $x_{\{d,s\}}=0$, yielding a pair whenever the storage is in `cooldown_storages` **or** `topology.storage_accepts_format(s, d.format)` is false — one shared function rather than the cooldown check and the format check duplicated inline in each backend (which is what pushed `_cpsat_feasibility_constraints()` over this project's own complexity limit the first time both were written inline; factoring the *decision* out, not just the loop, is what brought it back under). A disk already resident on a storage is never fixed away from it by either rule.

Deliberately not implemented in this pass

- **A live cluster's numpy/pandas footprint.** `ortools` pulls in `numpy` (and `numpy` pulls in nothing further this project cares about) purely as its own transitive dependency; nothing in this project imports `numpy` directly. A recent `numpy`'s bundled type stubs use syntax newer than this project's `python_version = "3.11"` mypy target, which mypy's `follow_imports = "skip"` (used for `ortools.*` itself) cannot rescue, because a stub-level *parse* error happens before mypy ever gets to apply a per-module setting to it. This surfaces whenever `ortools` (via `pip install -e .[solver]`) or `statsmodels`'s own `[forecast]` extra happens to pull in an affected `numpy` version into the *same* venv mypy runs against – verified directly (extending the `follow_imports = "skip"` override to

pulp.* / statsmodels.* / numpy.* itself does not rescue it either, it is that hard a parse error). make typecheck does not risk this at all: it runs mypy against its own dedicated .venv-typecheck (make venv-typecheck, .[dev] only), never the .venv that make test/make cov install solver/forecast into for full backend coverage – a *separate* venv, not a reused one later cleaned up, so nothing either target does to .venv can ever reach it. See the Makefile’s own comment on TCENV. Both MILP backends were verified for real during development (pip install -e .[solver], all of test_optimize.py passing and reproducing the section 14 and reserve-tradeoff.yaml fixtures’ exact numbers for both CP-SAT and CBC) – see tests/unit/test_optimize.py’s own docstring. That first pass installed pulp 3.3+ from PyPI, not the version actually packaged for the deployment target – a gap REVIEW.md’s tenth pass caught (S-01): python3-pulp 2.7.0+dfsg (Debian trixie, also what CI’s debian:trixie job installs) predates LpProblem.add_variable, PuLP v4’s variable-construction API, which an earlier revision of this module had adopted and which crashed immediately on that version. Fixed by constructing every variable with pulp.LpVariable(...) directly (works unchanged on every pulp release from 2.7.0 through 3.3.2, bisected by wheel inspection) and suppressing the resulting v4-migration DeprecationWarning on 3.3+ explicitly, plus catching PulpSolverError around prob.solve() (pulp can still import with no working cbc binary behind it – a non-Debian pip install pulp, say – even though python3-pulp itself Depends: coinor-cbc on Debian) so a missing solver returns None like every other “this backend cannot produce a plan” case, rather than a bare traceback. test_optimize.py’s CBC cases now pass against pulp 2.7.0 too, verified directly against a throwaway venv pinned to that exact version – the packaged path, not just the newer one pip happens to resolve.

That first verification pass was a one-off, done by hand. It no longer is, in either place this project runs tests, though the two get there differently. CI (.github/workflows/tests.yml’s “test” job) has one container, no separate venvs: coinor-cbc/python3-pulp are apt packages installed up front (CBC’s cases need nothing else), and ortools is pip installed as its own step, placed *after* make SYSTEM_TOOLS=1 typecheck runs, never before it – the numpy/mypy footprint above is exactly why that ordering matters there, since SYSTEM_TOOLS=1 points every tool straight at the one system Python, with nothing to isolate mypy from what that pip install just added. Locally, make test/make cov install solver/forecast into .venv themselves (loudly, never fatally, if that install fails – Makefile’s own SOLVER_EXTRAS), so test_optimize.py’s cpsat cases run there too without a developer needing to know to do it by hand; make typecheck never touches that venv at all, using its own .venv-typecheck instead, so there is no ordering to get right the way CI’s single environment needs.

Executing a plan: execute.py and its cli.py wiring

What does this page answer? Why does a “completed” move need three separate conditions, not one; why does a VM lock get waited out instead of failing the run; how is the injectable Clock used to test all of this without a real sleep; and what exactly does cli.py’s apply handler add on top of execute_plan() itself? Describes proxmox_storage_drs/execute.py and the _handle_apply()/_plan_group() half of proxmox_storage_drs/cli.py.

execute_plan() re-validates every move against the live cluster, not the plan

schedule.ScheduleResult.order was computed against a snapshot of the cluster taken at planning time. By the time a move near the end of a long confirm run is about to be issued, that snapshot can be stale in ways a live cluster routinely produces: an operator touched the VM, PVE’s own Dynamic Load Balancer moved it to another node, a snapshot appeared, an operator queued a pending config change on the disk about to move (section 3.8), or another process changed the target storage’s free space. _preflight() re-fetches the VM’s current node, config, running status, snapshot list, pending-change list and exclusion

tags fresh for every single move — deliberately bypassing the per-run topology cache (topology.py's section 3.5 cache exists to avoid re-fetching for *planning*, not to avoid re-validating immediately before a mutation). The exclusion re-check (section 9.2 step 3: “untagged for exclusion”, REVIEW.md S-06) is `_is_excluded_by_tag_or_vmid()`, a small duplicate of topology.py's own `vmid/tag` predicate rather than an import of it — that name is private to that module, matching this codebase's usual choice for a one-line filter (AGENTS.md section 5; optimize.py's `_movable_disks()` is the same precedent). And `_live_transient_check()` re-derives the section 8.1 transient invariant against the target's *live* content listing and capacity, not the in-memory model. Either kind of mismatch stops the group's run with “replan_needed” rather than trying to patch the plan around it — IMPLEMENTATION_PLAN.md section 9.2 is explicit: “abandon the remaining moves... do not attempt to patch it.”

The pending-change re-check (section 9.2 step 6) is the opposite choice from the exclusion re-check's deliberate duplication above: it calls `topology.pending_disk_reasons()` directly rather than re-implementing the `/pending` entry parsing, because that parsing is not a one-line predicate — topology.py exports it (unlike its underscore-prefixed siblings) specifically so this is the only place it lives (60-topology.md's “The pending-change pin”).

The live transient check: provisioned, never allocated

`_live_transient_check()`'s arithmetic is the identical section 8.1 formula `schedule.transient_invariant_ok()` checks against the in-memory model — the target must still hold $used_b + \sum z_m + \max(f_b \cdot \max(Z_b, \max z_m), hard_b) \leq C_b$ while the move is in flight, z_m each move's disk size, f_b the storage's snapshot-reserve factor, Z_b the largest disk already resident and $hard_b$ the transient free-space floor. Both call the same `reserve.transient_charge_ok()` (see 95-schedule.md); the live check differs only in where $used_b$ and C_b come from. The same *rule*, a different *data source*, not a second implementation of it (AGENTS.md section 5).

used_b is the sum of size over the target's live content listing — never storage_status()'s own used. The tool counts every disk at its *provisioned* size, on thin-provisioned storage (Ceph RBD, LVM-thin, ZFS) exactly as on thick, because a plan that fits only while the disks stay thin is one a growing guest can turn into a full pool, and nothing here can bound that growth (.agents/domain-invariants.md rule 2b; over-provisioning is never considered). The planner's $used_b$ is $\sum z_d + U^{ext}$ — the provisioned sizes of the managed disks plus the foreign volumes on the storage — which is the listing's total. PVE's *used* is the *allocated* figure: equal on a thick pool, far lower on a thin one (a pool with 3.36 TiB provisioned and 1.45 TiB allocated was seen on the dev cluster). A live check built on it could only ever confirm the plan; it could never notice that other provisioning had filled the pool, in provisioned terms, since planning — the one thing a live re-check exists to catch. C_b still comes from `storage_status()`'s total, since a LUN can be resized after the run started.

Two properties keep the listing figure safe to trust:

- **It fails safe.** A PVE API error reading either endpoint, or a listed volume with neither *size* nor *approximate-size* (its *size* then cannot be established, and planning's habit of skipping such a foreign volume with a warning is not available to a check that decides whether to touch the storage now), refuses the move as “replan_needed” with the reason in the outcome's detail — never a pass on a partial figure, never an exception that crashes the run. *approximate-size* counts at its value, the same fallback tier planning uses (`topology.content_item_size()`).
- **One rule for the sizes.** `topology.content_item_size()` is the single place the *size* → *approximate-size* order lives, shared by the planner's disk and foreign-volume sums and this check.

No double counting under concurrency. With `execution.max_concurrent_migrations` above 1 the invariant carries one charge z_m per move of this run already in flight onto the same target, so those

moves’ own mirror targets must not also be summed from the listing: on RBD (and other block storage) the new volume is listed at its full provisioned size the moment `move_disk` allocates it, and would be charged twice. `_LiveCheck.listed_volid`s — every volid the target’s listing held at the moment a move launched — is stored on the launched move as `_InflightMove.target_baseline_volid`s. `_inflight_target_volid`s() then leaves out of the sum **at most one volume per in-flight move**: one not in that baseline, of the *same* VM, whose size is one of the two sizes `move_disk` allocates a mirror target at (`_is_mirror_target()`). As the operator describes PVE’s behaviour (it has not been read from PVE’s source): between storages of the same thin kind the target is the same size as the source image, i.e. `Disk.size_bytes` from its content listing; between different storage types, or from thin to thick, it is the disk line’s `size=` in the VM config. The pre-flight already parses that line, so `_PreflightResult.config_size_bytes` is carried onto `_InflightMove.config_size_bytes` and a volume of either size matches — the two can differ when a volume was resized outside PVE or its storage rounds sizes. If PVE allocates some third size, nothing matches, the target is counted as well as charged, and the check is merely stricter than necessary. The same two sizes decide what a move is *charged*: `_move_charge_bytes()` returns the listed size, except when the move changes the kind of volume made — `Storage.storage_type` differs between source and target, or a qcow2 disk lands on a target that cannot hold qcow2 and PVE writes it raw — where the target is allocated at the config’s `size=` and the larger of the two is charged. A same-kind move keeps the listed size, since there the target is a copy of the source image and charging more would refuse moves for a discrepancy that does not apply. `_live_transient_check()` takes the candidate’s charge as a number and computes each in-flight move’s own the same way from its stored `source/target/config_size_bytes`. The tool never passes `format=` to `move_disk` and (C2) keeps a qcow2 disk off a storage that cannot hold it, so the conversion branch is a guard rather than something a plan produces today; the plan-time check in `schedule.py` keeps using the listed size. The matching is deliberately narrow for the same reason: a foreign volume that appeared since the launch, or a leftover of the same VM that was already there (it is in the baseline), is never excluded — wrongly keeping a volume only tightens the check, wrongly dropping one would weaken it. Where nothing matches yet (the short window between `move_disk` returning its UPID and the allocation), the move is charged by its `z_m` alone, which is exactly right. The sequential executor has no in-flight set and excludes nothing. Because the whole in-flight period, mirroring and draining, keeps a move in the set, its target volume is excluded and its `z_m` charged for as long as it is — counted once either way.

`transient_charge_ok()` takes a *list* of charges, not one disk, so the concurrent executor calls it with every move currently in flight against the same target (see “Concurrent execution” below).

Why “done” needs four conditions, not one

IMPLEMENTATION_PLAN.md section 9.3.2’s completion criterion is deliberately stronger than “the task succeeded”:

```
move m from a to b is DONE ⇔ task(upid) exitstatus == OK
    ∧ volume(m) absent from GET /storage/{a}/content
    ∧ config(vmid(m)).lock is empty
    ∧ elapsed(drain) ≥ min_wipe_seconds(m)
```

A storage with `saferemove` enabled keeps a storage-level lock on the *source* for as long as it takes to zero the old volume at `saferemove_throughput` (10 MiB/s by default) — hours or days for a large disk, entirely after the `move_disk` task itself has already reported success. `_wait_for_move_completion()` polls the task first (unbounded — a move that was ever scheduled already passed `migration.max_single_move_duration` at planning time, so there is no separate timeout for the task loop itself), then, only when `execution.source_release.wait` is true and the source actually `saferemoves`, polls the source’s own content listing and the VM’s lock together, bounded by `execution.source_release.timeout_seconds`. Hitting that bound is not a failure — it is “draining”: the mirror is done, the wipe is still running, and the

next run will see the storage as it actually is.

The fourth condition, min_wipe_seconds, was added after the first three turned out not to be enough (section 9.3 point 2's "found dogfooding" paragraph: VM 101's scsi1 move satisfied all three — task OK, volume gone, lock clear — and the very next move_disk for the same VM still failed with PVE's own can't lock file ... - got timeout, because PVE's wipe cleanup was still finishing at the OS level after those API-visible signals had already cleared). `_poll_move_once()` computes it as `payback.compute_wipe_duration_seconds(size_bytes, source.saferemove_throughput_bytes_per_sec)` — the identical formula `payback.compute_move_cost()` already uses for the planning-time cost estimate (AGENTS.md section 5: one implementation, two callers) — and folds it into the same `drain_start` clock the volume/lock poll already tracks, so it costs no extra API call. It is None, and so has no effect at all, whenever the storage has no configured `saferemove_throughput` — this only tightens the criterion where PVE's own config gives an exact number to tighten it with.

`execute_plan()`'s own `largest_by_storage` bookkeeping (section 8.1's `Z_s`, the largest resident disk on a storage — needed for *later* moves in the same run that check the same target's live transient invariant) updates on "moved" **and** "draining", not "moved" alone: the mirror itself is physically complete either way, so a target storage's own accounting must already include this disk regardless of whether its *source* has finished releasing.

A "draining" outcome also adds that move's *source* to a `drained_storages` set `execute_plan()` carries for the rest of the run (REVIEW.md S-05): the next move whose own source or target is in that set is skipped outright, `_drained_skip_outcome()` reporting it without a single API call. Section 9.3 asks for exactly this ("exclude it as both source and target for the remainder of the run") — a draining storage holds a storage-level lock for as long as the wipe runs, so a subsequent `move_disk` touching it would either queue behind a potentially day-long wipe (the task-status poll loop is unbounded) or fail outright, tripping `abort_on_failure` for no reason a human would consider a real failure. An earlier revision of this module tracked the (C4) *accounting* consequence of a draining target (the paragraph above) but never actually excluded the storage from later moves in the same run — found by a review pass, not a test, since nothing at the time exercised two moves sharing a storage within one run.

VM locks are an open set, waited out, never whitelisted

`_wait_for_unlocked()` treats any non-empty `config.lock` as "wait," full stop — never a lookup table of "harmless" values to skip past (`.agents/domain-invariants.md` rule 5: a future PVE version can add a lock value this codebase has never seen, and guessing wrong risks a half-completed operation on someone else's backup or snapshot). What happens after `execution.locks.wait_timeout_seconds` depends on `execution.locks.on_timeout`:

- "skip" (the default) — report the move "skipped", continue with the rest of the plan.
- "abort" — report it "failed" and set `MoveOutcome.always_stop`, which `execute_plan()`'s stop condition checks *independently* of `execution.abort_on_failure`. This was a real design bug caught by re-reading the manual's own wording before any test existed: the manual describes `on_timeout: abort` as "abort **the run**," a stronger and unconditional promise, distinct from `abort_on_failure`'s general "stop after any plain task failure" policy, which must still be respected on its own terms for an ordinary `move_disk` failure. Folding both into one flag would have made `abort_on_failure: false` silently defeat an operator's explicit `on_timeout: abort` choice.

The move_disk task's own flock: a retry as the fallback under min_wipe_seconds

`min_wipe_seconds` above (section 9.3 point 2) closes the VM-101 race in the one case it can be computed exactly — a configured `saferemove_throughput`. Section 9.3 point 3 is the fallback for everything that condition cannot cover: no throughput configured, an inaccurate one, or `can't lock file '/var/lock/qemu-`

`server/lock-<vmid>.conf'` - got timeout surfacing as a `move_disk` task's own `exitstatus` for some other momentary reason entirely. PVE clears the config lock: line before it releases the config file's own flock, so there is nothing `_wait_for_unlocked()`'s pre-flight check could ever have polled to see this coming — it is a task-level failure, not a lock the executor was ever in a position to wait out in advance.

`_is_task_lock_timeout()` matches that exact PVE message against a failed move's `exitstatus`. `_execute_one_move()` (sequential) and `_poll_inflight_once()` (concurrent) both reissue `move_disk` on a match, up to `execution.locks.task_retry_limit` extra attempts, recording the crash-recovery on `inflight_finished/on_inflight_started` pair for the old and new UPID exactly as the first attempt did (section 11.2 — a retried move is still always exactly one UPID in `state.json` at a time). The sequential path sleeps `execution.locks.task_retry_backoff` on the injected `Clock` before reissuing, matching every other wait loop in this module; the concurrent path deliberately does not sleep inline — blocking one in-flight move would stall every other candidate and in-flight move the same way a blocking lock wait would (see “What `execute_plan()` deliberately does not do” above) — a retry there is instead picked up on the next poll cycle, spaced by `execution.poll_interval_seconds` unless something else already made progress that cycle. A retry that still exhausts the limit is reported “failed” exactly like any other task failure, going through the same `abort_on_failure/orphan-detection` path.

The injectable `Clock`

Every wait loop in this module measures elapsed time via `clock.now()`, never by counting `clock.sleep()` calls — so a test's fake clock, whose `sleep()` advances its own `now()` instantly instead of blocking (`.agents/testing.md`: “no real time.sleep; inject the clock”), exercises the *exact* timeout arithmetic production does, at zero wall-clock cost. `_REAL_CLOCK` is a module-level singleton (`Clock` holds only function references, never per-call state, so sharing one instance is safe) used as `execute_plan()`'s default — a bare `Clock()` there would be a flake8-bugbear B008 violation (a function call in a default expression).

Orphaned volumes: reported, never deleted

`_detect_orphan_volumes()` runs only after a “failed” outcome, cross-referencing the target storage's content listing against every volume the VM's config actually still references (via the shared `topology.parse_disk_spec()/topology.DISK_KEY_RE` — the same disk-spec parser `topology.py` itself uses, not a second one). A volume owned by the VM but not referenced anywhere in its config is reported in that move's own `MoveOutcome.orphaned_volumes` and logged at warning — never removed automatically (`.agents/domain-invariants.md` rule 6). A `PveApiError` while checking (the storage or the VM briefly unreachable right after a failure) is caught, logged, and degrades to an empty tuple rather than turning an already-failed move into a crash.

`confirm`'s callback lives in `cli.py`, not here

`execute.py`'s `ConfirmCallback` type is `Callable[[ScheduledMove], str]` returning “y”/“n”/“a”/“q” — `execute_plan()` calls it, interprets the four answers, and raises `ValueError` on anything else, but never itself talks to a terminal. `cli.py` is the only module allowed to call `print()/input()` (its own module docstring), so `cli._confirm_move_interactively()` is where the actual prompt text lives and where an out-of-set answer is *retried* rather than turned into the `ValueError` `execute_plan()` would otherwise raise — a typo at the keyboard should not look like a bug report.

What `execute_plan()` deliberately does not do

- **No re-plan loop inside this module.** A “replan_needed” outcome stops the group's run cleanly here; re-invoking the whole `gate/solve/schedule/payback` pipeline from the newly observed state is `cli.py`-level orchestration across multiple `execute_plan()` calls (`cli._run_auto_group()`, `auto` mode only — see below), not something this module does itself. `dry-run/confirm` never re-plan at all: the operator re-runs apply by hand once ready.

- **Section 7.3’s saturation check is planning-time only, not execution-time, concurrently or sequentially.** The defer check itself is enforced before either executor ever sees a flagged move — it is excluded from `schedule_result.order` at planning time (`docs/internals/96-payback.md`) — but neither executor re-checks the ceiling *during* execution against the live in-flight set, so section 8.1 point 4 of `concurrency_ok` (summing `w_role` over everything actually in flight right now, mirroring or draining) stays a documented gap the concurrent executor inherits rather than closes.
- **Crash recovery is not this module’s own job.** `execute_plan()` accepts `on_inflight_started/on_inflight_finished` callbacks (`execute.InflightCallback`) and calls them around each `move_disk` — writing what they actually persist, and section 13’s startup scan that reads it back, both live in `crashrecovery.py/cli.py`. See [93-crashrecovery.md](#).

cli.py: one planning pipeline, shared by plan and apply

`_plan_group()` is `_handle_plan()`’s former per-group loop body, extracted so `_handle_apply()` calls the *same* function rather than re-deriving “gate, then solve, then schedule, then payback” a second time (`AGENTS.md` section 5) — `IMPLEMENTATION_PLAN.md` section 9.2’s own re-plan protocol says to re-run exactly this pipeline, not a parallel one apply keeps to itself. `_GroupPlan` carries every stage’s result (or `None` for a stage that never ran because an earlier one stopped short, e.g. `load_error` set or the gate said `NO ACTION`) so both handlers can tell those cases apart without guessing from missing dict keys.

`_handle_apply()` acquires `state.py`’s advisory lock **before** touching PVE or Prometheus at all — a second concurrent apply invocation (any mode, including dry-run) exits 0 immediately without paying for either round-trip first, matching section 11.2’s “a live PID means another instance is running: exit 0 quietly.” Because an operator’s `[q]uit` (or a `replan_needed`, though that one is per-group and does not otherwise propagate) can stop the whole run before every group is even planned, `_render_group_plan_human()/_render_group_plan_json()` — the shared per-group renderers behind both `plan`’s and `apply`’s output — look up `gate_decisions` with `.get()`, not `[]`: `plan` always plans every group first, but `apply` genuinely can leave a later group completely unvisited, which must render as “not evaluated this run,” not a `KeyError`. This was caught by a test exercising the quit path, not by inspection.

After a group’s execution, `_record_executed_moves()` decides which moves counted as “executed” for `state.json`’s own bookkeeping — “moved” and “draining” (the mirror itself completed either way, mirroring `execute.py`’s own (C4) accounting choice for those same two statuses), never “would_move”/“skipped”/“failed”/“replan_needed”. It reads straight off the final `ExecutionResult.outcomes`, never a `_GroupPlan`’s own `schedule_result.order`: a payback refusal (S-02) can already make outcomes a reordered subset of that order, and in auto mode a re-plan (`_run_auto_group()`, below) can execute moves from a *later* plan attempt that never appeared in the group’s first-attempt order at all — an earlier revision iterated the stale order and silently dropped those later moves’ cooldowns. For every move that counts, it calls `state.with_recorded_balance()` (this group’s measured load vector, feeding the next run’s drift gate) and `state.with_recorded_cooldown()` — a fresh disk cooldown for every executed disk at its *new* location, and a fresh storage cooldown for **both** of the move’s storages, source and destination (section 6: “a storage involved in a migration ... accepts no new incoming moves” covers both; `REVIEW.md` S-03 — an earlier revision of this code recorded the destination only, which meant no configured `gates.cooldown_per_storage` could ever protect a still-draining *source*, exactly the scenario section 9.3’s knob-sizing rule and `verify-storages`’ own warning describe). Recording both endpoints is independent of `heuristic.py`’s own *enforcement*, which stays destination-only: a cooldown storage is excluded as a move/swap target, never as a source a disk may still leave (`docs/internals/90-heuristic.md`). The accumulated state is written once, at the very end of the whole run (every group, not per-group), through `state.save_locked_state()` — never `state.save_state_atomic()`’s `rename`, which would silently detach the very lock `acquire_lock()` just took (see `docs/internals/15-state.md`’s account of that bug) — and only then released via `state.release_lock()`, in a finally so a mid-run exception never

leaves the lock held.

The payback gate: apply refuses on its own verdict (REVIEW.md S-02)

Section 7 calls the payback rule “a **hard acceptance test on the finished plan**, not merely a soft γ penalty” — a plan a move’s size already costs γ points against in the solver’s own objective (docs/internals/91-optimize.md) can still be rejected wholesale here, after the fact, rather than just scored lower going in: a verdict plan shows the operator, not a suggestion. An earlier revision of `_handle_apply()` computed `payback_result` and then ignored it entirely on the execution path: a plan whose aggregate economics failed, or whose one move exceeded `migration.max_single_move_duration` (`payback_result.rejected_moves`, the *hard* per-move rule §7.3 separates from the aggregate one), was executed anyway. `_apply_payback_gate()` is the fix: it never calls `execute_plan()` at all for a move in `rejected_moves`, or for *any* move in the group when not `payback_result.aggregate_ok` — both report a “skipped” outcome (“refused: ...”) through the exact same `MoveOutcome/ExecutionResult` shapes `execute.py` itself produces, so neither the renderers nor `state.json`’s bookkeeping need a special case: a refused move was never “executed” (it is not “moved”/“draining”), so it earns no cooldown and contributes nothing to `last_balance`. A plan that clears both checks is passed to `execute_plan()` with its individually-rejected moves (if any) simply absent from `order` — which is also what makes `_wait_for_move_completion()`’s own docstring claim (“a move that was accepted at planning time already passed `migration.max_single_move_duration`”) true by construction rather than merely asserted. In confirm mode, the group’s payback lines print once, before the first prompt — an operator confirming moves one at a time sees the tool’s own verdict before being asked anything, not only in the post-run report.

auto mode: timewindow.py and _run_auto_group() (phase 8)

`execute_plan()` itself grew two auto-only budgets, both `None` (inactive) unless `cli.py` passes real values: a deadline (an absolute instant, checked against `clock.now()` before starting each move, together with that move’s `payback.MoveCost` duration estimate from `move_costs_by_key`) and a `max_migrations` count, decremented once per move actually attempted (“moved”/“draining”/“failed” — never “skipped”/“would_move”/ “replan_needed”). Both stop the whole run cleanly (never mid-move) via the same `_auto_budget_stop_outcome()` helper, since remaining time only ever decreases and a spent cap stays spent — there is never a reason to skip one move for a budget reason and then try a later one anyway.

The deadline is actually checked **twice** for a locked VM: `_auto_budget_stop_outcome()`’s pre-flight check runs before any network call for this move, but a VM-lock wait (`_wait_for_unlocked()`) can burn a large, unpredictable share of the same budget — up to `execution.locks.wait_timeout_seconds`, hours by default — entirely *after* that first check already said yes. `_execute_one_move()` re-checks the identical condition (`_deadline_exceeded()`, the one shared implementation of “does this much more time still fit”) right after a lock clears and before committing to `move_disk`, refusing the move there instead if the wait alone consumed what was left. Caught by re-reading section 9.1 against the code before any test existed, not by a failing test.

`timewindow.py` is a small, dependency-free module answering exactly one question, `current_deadline(windows, now) -> datetime | None`: `None` means no restriction at all (no `time_windows` configured), and a non-`None` value is either the close time of whichever configured window covers `now`, or `now` itself when none currently do (zero remaining budget — deliberately not a separate “not in any window” signal, so a caller’s “does the next move fit before the deadline” arithmetic refuses correctly without a second check). It works in **local time**, not UTC — a documented, deliberate choice (the plan does not name a timezone for this setting) matching `systemd.timer`’s own `OnCalendar=` default and how an operator actually thinks about a maintenance window. The classic overnight-window bug — a `days: [fri]` window tagged Friday still needing to match at 2am *Saturday* — gets its own two-piece matching

logic (`is_window_active()`) and its own regression tests.

`timewindow.py` itself never reads the clock (every function takes `now` as a parameter); `cli._real_local_now()` is what production actually passes. It resolves the host's real IANA zone from `/etc/localtime`'s own symlink target (the standard mechanism on Debian and virtually every Linux distribution) rather than settling for `datetime.now().astimezone()`'s *fixed*-offset result — the difference matters specifically for `window_close()`'s “closes tomorrow” case, which combines today's `tzinfo` with tomorrow's date: a real `zoneinfo.ZoneInfo` re-resolves its own UTC offset for that date, a frozen offset does not, so an overnight window could otherwise stay “active” up to an hour longer than configured on the two nights a year the local zone's clocks actually change. `_real_local_now()` falls back to the frozen-offset form only when the zone name cannot be determined at all, at which point that same narrow, two-nights-a-year discrepancy returns — a residual, documented limitation, not a silent one.

`cli._run_auto_group()` is where section 9.2's re-plan protocol actually lives: a loop, bounded by `execution.max_replans_per_run`, that calls `_apply_payback_gate()` and then, only if the *last* outcome it produced was “replan_needed”, re-fetches topology fresh (`build_topology()` — not just re-running `_plan_group()` on the same Group object, since whatever triggered the mismatch can mean the group's own membership changed) and calls `_plan_group()` again before retrying. A re-plan that concludes NO ACTION (or hits a load error) ends the loop quietly, not as a failure — section 9.2's own words, “the gates may well conclude no further action is needed.” Exceeding the replan cap rewrites the final `stop_reason` to name the setting explicitly, rather than reporting the *symptom* (the last mismatch) as if it were the cause.

One deliberate simplification: everything the human/JSON report shows for a group (`gate_decisions[name]`, `schedule_results[name]`, etc.) still comes from the group's *first* planning attempt, even after auto re-plans it — only the accumulated `ExecutionResult.outcomes` (every attempt's outcomes, concatenated) and `last_balance`'s recorded load vector reflect what actually happened. Returning a second `_GroupPlan` from a successful re-plan would need `_handle_apply()`'s reporting dicts to cope with the *first* attempt's fields being live while a later one supersedes them, and — worse — a re-plan that lands on NO ACTION would have to report a group whose `schedule_result/payback_result` are `None` despite the group having ACTed and moved something moments earlier. Reporting the first attempt's plan alongside the full outcome history is simpler and does not lose information: the report already shows *why* it stopped and *what happened next* per outcome.

`execution.max_migrations_per_run` is a per-*invocation* budget, shared across every group `_handle_apply()` visits — `_run_auto_group()` returns the updated remaining count, which its caller threads into the next group's call rather than resetting it.

Concurrent execution: `_execute_concurrent()` and its non-blocking helpers

`execute_plan()` dispatches to `_execute_concurrent()`, an entirely separate orchestration loop from `_execute_sequential()` above, whenever `mode == "auto"` and either `execution.max_concurrent_migrations` or `max_concurrent_per_storage` is configured above its default of 1. Every other case (dry-run, confirm, or auto at the default caps) uses `_execute_sequential()` completely unchanged — this is a genuine regression guarantee, not just a claim: every test written before concurrency existed still passes unmodified, because none of them touch the new dispatch condition.

Why a second loop, not one generalized loop. The sequential loop's `_wait_for_unlocked()` and `_wait_for_move_completion()` both *block*, sleeping in a tight loop until their own condition resolves. A concurrent executor cannot reuse either directly: blocking on one move's lock or completion would stall every *other* in-flight or candidate move for as long as that wait lasts, which defeats the entire point of running several at once. Both were refactored into a non-blocking step plus a thin blocking wrapper *before*

concurrency was added at all, so the refactor's own correctness could be verified by the full pre-existing test suite passing unchanged, independent of any new concurrent-specific test:

- `_check_lock_once()` is the one live lock read; `_wait_for_unlocked()` is now just that read in a loop.
- `_poll_move_once()` is section 9.3.2's completion criterion as a single step, returning `None` while still in progress or `(status, detail)` once terminal; `_wait_for_move_completion()` is now just that in a loop. `_MoveWaitState` carries the one thing that has to survive across calls (`drain_start`, once the source-release phase begins) since there is no longer a local variable's lifetime to hold it.

`_launch_decision()` is section 9.2's five pre-flight re-checks plus section 8.1's generalized transient invariant, for one poll cycle, without blocking — the concurrent counterpart to `_execute_one_move()`'s own pre-flight-then-lock-then-live-check sequence. `_LockWaitTracker` plays the same role for a lock wait that `_MoveWaitState` plays for a move's completion: start/warned survive across cycles so the executor logs the "VM is locked" warning once, not once per poll, and knows when `execution.locks.wait_timeout_seconds` has actually elapsed. The generalized invariant itself lives in `reserve.transient_charge_ok()` (`docs/internals/95-schedule.md`) — `_launch_decision()`'s only job is to assemble its inputs, and it does so by calling the same `_live_transient_check()` the sequential executor uses, passing the in-flight moves whose target is the candidate's (`_inflight_onto()`): the check reads the live provisioned `used_b` and `capacity`, takes `Z_b` from this run's own (`C4`) `largest_by_storage` tracking, and charges every other in-flight move's disk size alongside the candidate's own — see "The live transient check" above for why the in-flight targets are left out of the listing sum. On a "launch" verdict the target's listing at that instant travels back on `_LaunchDecision.target_baseline_valids` and is stored on the new `_InflightMove`.

Strict FIFO, deliberately. `_advance_pending()` only ever considers `pending[0]` — the same move a sequential run would try next. If it cannot launch yet (a per-storage cap, or a lock that has not cleared), the executor does not skip ahead to a later candidate that might launch immediately; it waits (polling everything already in flight in the meantime) and re-evaluates the same head next cycle. Several moves genuinely run concurrently once launched — nothing ever blocks waiting for one to finish before starting another — but *which* move launches next is always the scheduler's own next-in-line choice. A more sophisticated scheduler could reorder around a blocked head to keep every slot busy; this one can only ever under-deliver on throughput doing that, never launch a move out of the payback-scored order `schedule.order_moves()` computed, matching this codebase's existing tolerance for a documented, safe simplification over a more optimal but harder-to-verify one (`schedule.py`'s own deferred ordering-priority-2 and staging are the same trade).

A stop condition drains, it does not abandon. Once a budget is exhausted, a move fails under `execution.abort_on_failure`, or a pre-flight/live-invariant mismatch produces `"replan_needed"`, `_execute_concurrent()` stops calling `_advance_pending()` (no further moves launch) but keeps polling whatever is still in flight to its own natural conclusion before returning — their outcomes, and their crash-recovery callbacks, must still fire. Whatever is still in pending at that point (never even pre-flighted) is abandoned unreported, exactly like `_execute_sequential()`'s own early return already leaves every move after the one that triggered a stop completely unaccounted for.

Crash and two-instance recovery: `crashrecovery.py`

What does this page answer? How does an apply run notice a `move_disk` left running by a crashed previous run, or by a second instance running concurrently; how does `state.json`'s `inflight_upids` actually reach disk *before* the move it names can crash the engine; and what does a discovered `vmid` do to the rest of the run? Describes `proxmox_storage_drs/crashrecovery.py`, `state.py`'s `with_inflight_upid()/without_inflight_upid()`, and the `execute.py/cli.py` wiring around them.

The two failure modes section 13 names, one mechanism

“Engine crashes mid-move” and “Two DRS instances running” are different causes with the same shape: a `move_disk` task keeps running against a VM this run does not know about. `state.json`’s `inflight_upids` field exists so a run notices either case *before planning anything* (section 13’s own words) rather than proposing a second, conflicting move against the same disk:

1. **A local trace.** `execute.py` writes a UPID into `inflight_upids` immediately after `move_disk()` returns for it, and clears it once that task itself has finished — see “Writing before the crash” below. A process that dies mid-move (killed, OOM, host reboot) leaves that UPID behind; the *next* run’s startup scan finds it.
2. **A cluster-wide scan.** `state.json`’s own `fcntl flock()` (`docs/internals/15-state.md`) is real, kernel-enforced mutual exclusion for two instances on the *same host*, but cannot see another node at all. `PveClient.cluster_tasks()` (`GET /cluster/tasks`) is the one read that actually crosses the cluster — `crashrecovery.py` scans it for a still-running `qmmove` task from this tool’s own configured user that the local trace does not already know about. `IMPLEMENTATION_PLAN.md` is explicit that this is what degrades “two instances running” from “conflicting” to merely “slow and redundant” — the correct fix is still “run the timer on exactly one host” (`cron/systemd` configuration, not this code); this is the safety net for when that is not honoured.

Either way, `reconcile_inflight()` reports the affected `vmid` for `cli.py` to exclude from this run’s planning — **never** force-cancelled, never assumed safe to ignore.

`parse_upid()`: PVE’s UPID grammar, confirmed live

A UPID is `UPID:{node}:{pid}:{pstart}:{starttime}:{type}:{id}:{user}`: `id` is the `vmid`, as a string, for a VM-scoped task type (`qmmove` among them); it is empty for a cluster-scoped one (`vzdump`, say). This shape is not general PVE-API knowledge taken on faith — `.agents/domain-invariants.md` rule 10 forbids stating unverified PVE behaviour — it was confirmed live against the real dev cluster (`[[dev-cluster-access]]`) by reading an actual `qmconfig` task’s own UPID before `parse_upid()` was written to depend on it. `parse_upid()` returns `None` for anything that does not parse as a UPID at all (a foreign or malformed string), never raises — a scan finding one unparseable entry must degrade to “cannot use this one,” not abort.

`/cluster/tasks`’ own convention

`PveClient.cluster_tasks()` (`GET /cluster/tasks`) returns every node’s recent/active tasks as `{node, type, id, upid, user, starttime, saved}`, plus `endtime/status` — **both present once the task has finished, both absent while it is still running**. This is a different convention from the per-task `PveClient.task_status()` endpoint (`GET /nodes/{node}/tasks/{upid}/status`), whose own status field is the string “running” or “stopped” — confirmed live against the same cluster alongside the UPID grammar above, and the reason `_scan_cluster_for_foreign_inflight()` uses “endtime” in task or “status” in task to filter obviously-finished candidates cheaply, then `_task_is_running()` (below, wrapping `task_status()`) to actually confirm one that looks live — never the reverse, and never treating `/cluster/tasks`’ own status string as if it used `task_status()`’s “running”/“stopped” vocabulary.

`reconcile_inflight()`: both halves

```
def reconcile_inflight(
    client: PveClient, state: State, auth: AuthConfig
) -> tuple[frozenset[int], State]:
```

Split into two helpers purely to stay within this project’s flake8 complexity limit:

- `_reconcile_recorded_upids()` re-checks every UPID `state.json` already remembers via `_task_`

`is_running()` (which wraps `task_status()`, not `/cluster/tasks`' own convention — see above). One whose task has since finished is dropped from the returned `inflight_upids` (an ordinary reconciliation of a move the engine never got to clear itself, logged at INFO); one still running is kept and its `vmid` excluded. A malformed or `vmid`-less recorded entry is dropped outright — there is no future run in which it could ever resolve either.

- `_scan_cluster_for_foreign_inflight()` is the cluster-wide half described above, skipped entirely (`frozenset()`, no API call at all) when `expected_task_user()` cannot determine this tool's own configured user (neither `token_id` nor `username` set — config validation makes this unreachable in practice, but the function degrades safely rather than guessing). `already_known` (the still-inflight subset of `state.json`'s own recorded UPIDs) is subtracted first, so a run's own in-flight move never logs a spurious “found a foreign task” warning about itself.

`_task_is_running()` returns `None`, not an exception, when the check itself fails (the node is unreachable, say) — logged at WARNING and treated by both callers as “assume still running.” That is the safe direction to guess wrong in: excluding a VM that turned out to be free costs one skipped move on this run; assuming free a VM that was not risks a second move `_disk` racing the first.

Writing before the crash: `execute.py`'s callbacks

Persisting `inflight_upids` only at the *end* of a whole apply run — the way `last_balance/cool downs` are persisted — would never survive the exact crash this mechanism exists to recover from: if the engine is killed mid-move, nothing after that point ever runs. `execute_plan()` therefore accepts two optional callbacks, `execute.InflightCallback = Callable[[str], None]`:

- `on_inflight_started` fires in `_execute_one_move()` immediately after `move_disk()` returns a UPID — before anything else happens with it, including the (potentially very long) wait for it to finish.
- `on_inflight_finished` fires immediately after `_wait_for_move_completion()` returns, for *any* outcome including “failed” and “draining”. A “draining” source is no longer tracked by this UPID at all once its task itself has finished — only by its own content-listing poll (`state.without_inflight_upid()`'s own docstring) — so it is cleared here exactly as promptly as a “moved” outcome, not left recorded until the source actually releases.

Neither call is wrapped in `try/finally`: if something inside the wait itself raises (a network failure mid-poll), `on_inflight_finished` simply never fires and the UPID stays recorded — which is correct, since the move might still be running and the next startup's scan must still find it.

`cli.py`'s `_make_inflight_callbacks()` is what the two callbacks actually do: write straight to disk via `state.with_inflight_upid()/without_inflight_upid()` followed by `state.save_locked_state()`, through the same `LockHandle_handle_apply()` itself holds for the whole run. This is the one deliberate, narrow exception to this codebase's otherwise-pure functional state-threading style (`_handle_apply()`'s state variable is reassigned throughout, never mutated) — `_InflightStateBox` is a mutable box purely so a closure passed several call-frames deep into `execute_plan()` can still reach it. `_handle_apply()`'s own `finally` block saves `state_box.value`, never the plain state variable, for exactly this reason: if an exception propagates out of a group's execution, a callback earlier in that same group can already have written a newer `inflight_upids` straight to disk than whatever state was last assigned in the surrounding loop, and saving the stale local variable would silently clobber that write.

cli.py: the startup scan, before planning anything

`_handle_apply()` calls `_reconcile_inflight_and_fold_exclusions()` immediately after building the PVE client, before `build_topology()` for planning — section 13's own “before planning anything.” A non-empty result is folded into `exclude.vmid s` on the `ResolvedConfig` used for the rest of this run, reusing `topology.py`'s existing (C2) `vmid/tag` pin rather than a second exclusion mechanism (AGENTS.md section

5: one implementation); the report shows such a vmid pinned with the generic “excluded by config (exclude.vmid)” reason, and `crashrecovery.py`’s own `logger.warning()` calls are what tell the operator *why* it is there. This runs for every mode, including dry-run — an accurate dry-run report should already reflect a vmid a real apply run would also have excluded, and reconciling a since-finished recorded UPID away is harmless (and useful) even when nothing gets executed this run.

Ordering the moves

What does this page answer? How does `schedule.py` turn a target assignment into an ordered, transient-feasible sequence of moves, and what happens when no order exists? Describes `proxmox_storage_drs/schedule.py`.

One loop, reusing `heuristic.py`’s objective for the ratio

`order_moves()` implements section 8.2’s pseudocode directly: while there are pending moves, find the ones that are transient-feasible right now, schedule the best by persistent-objective-reduction-per-byte (moves that resolve a currently-violating storage win outright, regardless of that ratio), apply it, repeat. “Persistent-objective reduction” sums the α , δ and $\kappa \cdot w_v$ terms — the parts of section 5.4’s objective whose improvement outlives the plan, as opposed to β/γ , which are one-time migration costs a completed move has already paid — computed by calling `heuristic.evaluate_assignment()` twice (before/after the candidate move) and diffing `.imbalance_term + .capacity_spread_term + .fragmentation_term`. Not a second formula: the same one `heuristic.py` already computes, so a plan’s ordering agrees with whatever `objective.spread_metric/kappa_vm_affinity` is configured exactly the way the assignment that produced it did. The $\kappa \cdot w_v$ term joined this sum alongside section 5.4’s w_v weighting itself (previously $\alpha + \delta$ only) — a fragmenting move now visibly costs more in the ranking, not only in the target assignment the solver already chose.

`cost_m` is `z_d` — except below `tiny_disk_bytes`, where it is 0

Section 8.2’s ratio is “persistent-objective reduction / `cost_m`”. `cost_m` here is simply the disk’s size in bytes, not `payback.py`’s real duration-based cost (mirror time plus, when `saferemove` is on, wipe time — and `wipe_throughput` is a per-storage config value, so two candidate moves off different sources can have genuinely different wipe costs even for the same `z_d`). `migration.bwlimit_bytes_per_sec` is one global config value, so every move’s *mirror* duration alone is $z_d / bwlimit$, a constant divided into every disk’s size equally — dividing a reduction by `z_d` and dividing it by $z_d / bwlimit$ produce the **same ordering** (`bwlimit` is a positive constant common to every candidate). So `cost_m = z_d` reproduces the true mirror-only ordering exactly, but is an approximation once a group has moves whose `saferemove` wipe cost differs enough to change the ranking `payback.py`’s full cost would produce. Scoring candidates by `payback.compute_move_cost()` instead of `z_d` is a natural follow-up, not yet done: this module predates `payback.py` and has not been revisited to consume it (REVIEW.md R-04).

One exception is implemented today: a disk below `migration.tiny_disk_bytes` gets `cost_m = 0` regardless of its real `z_d`, and the ratio is `+inf` rather than a division — section 8.2’s own words, “`cost_m = 0` ranks first: free value, delivered before anything pays.” This is not merely “falls back to raw reduction” (which would not necessarily beat a competing large candidate’s own high ratio): a zero-cost move is ranked ahead of *every* nonzero-cost one outright, so a tiny `efidisk0/tpmstate0` reunion is always scheduled before a real migration that happens to reduce the objective by more in absolute terms.

The transient invariant, called with a single-move set always

Section 8.1 defines a generalized invariant over a whole in-flight set M specifically so it can be “implement[ed] as the single feasibility predicate and call[ed] with $M = \{m\}$ for the sequential case” — this module is that sequential case, always. `transient_invariant_ok()` checks one move against state, the assignment as of immediately before that move starts; every previously-scheduled move is assumed **fully**

complete by then (mirror finished, and any safermove wipe finished, source genuinely freed) before this one begins. That models `schedule.order_moves()`'s own job correctly regardless of `execution.max_concurrent_migrations` — it always computes one, strictly *-sequential order*; that order becomes a launch queue several moves can run against concurrently only once `execute.py`'s own executor gets hold of it (92-execute.md's "Concurrent execution" section) — this module itself never reasons about overlapping in-flight windows, and does not need to, since `execute.py`'s own live re-checks (below) are what actually enforce the invariant against whatever really ends up running together.

The actual arithmetic — $used_b + \sum(z_m) + f_b * \max(Z_b, \max(z_m)) \leq C_b$ (storage b's bytes already used, plus every in-flight charge landing on it, plus its reserve headroom sized off the larger of what already resides there and the largest single charge in flight, must not exceed its capacity — f_b is the storage's own `reserve_factor`, z_m each in-flight move's disk size, Z_b the largest disk already resident, C_b the storage's capacity in bytes) — lives in `reserve.transient_charge_ok()`, taking a *list* of charges rather than one disk, so it degenerates to section 8.1's original single-move form when called with `[disk.size_bytes]` (what this module does) and generalizes correctly to several moves landing on the same storage at once when `execute._launch_decision()` calls it with more than one, for real, under concurrent execution (AGENTS.md section 5: this is the *one* place that formula is written, not two functions that happen to agree). `execute._live_transient_check()` — called by both the sequential executor and `execute._launch_decision()` (concurrent) — calls it with live numbers instead of this module's model-derived ones: capacity from `storage_status()`, and `used_b` summed from the target's content listing at *provisioned* sizes, so it stays the same quantity this module computes rather than PVE's lower allocated used on a thin pool — see 92-execute.md.

Section 5.3.1's transient floor is folded into the check the same way (C5) folds `soft_s` into the steady-state one — $\max(f_b * \max(Z_b, z_d), hard_b)$ — except the floor here is `hard_b` (`target_storage.free_space_hard_bytes`), not `soft_b`: section 8.1's own formula does not mention a floor at all, but there is no reason a storage's configured minimum free space should stop applying just because a migration happens to be in flight, and `hard_b` (not `soft_b`) is what this transient state is allowed to relax to — with the default `hard: null` (`hard_b = soft_b`) the two floors coincide and the check is exactly as strong as it would be with `soft_b`; an operator who sets `hard` below `soft` is deliberately buying room for a bounded, in-flight dip on a storage the finished plan still leaves compliant (`.agents/domain-invariants.md` section 2a; `topology.py` resolves both onto the `Storage` this function already receives, so no separate parameter threads either one in here).

The heuristic can accept a residual violation; the scheduler cannot execute one

`heuristic.py`'s repair step can leave a storage still violating (C5) if no reachable assignment can fully clear it — a genuine, sometimes unavoidable outcome the objective's `reserve_penalty_term` reports rather than hides (see 90-heuristic.md). When `order_moves()` is asked to schedule the move that leads to exactly that residual state, the transient check correctly says no: landing a disk on a target that still would not satisfy (C5) is not transient-feasible, full stop, and this module never treats "the heuristic already decided this was the best available" as a reason to relax that. The result is a deadlock report for that move (section 8.3 option 3, "report... never force a move that breaches the reserve") — an honest "this cannot be safely executed", not a false all-clear. See `test_plan_reports_a_deadlock_when_even_the_best_target_still_violates` in `tests/unit/test_cli.py` for the exact scenario reproduced end to end.

final_assignment is the schedule's own truth, not the heuristic's target

`ScheduleResult.final_assignment` is the assignment produced by actually applying order in sequence, starting from the current placement — every disk, not only the moved ones. When nothing deadlocks it is identical to the heuristic's target assignment, but when deadlocked is non-empty (fully or, more subtly,

only partially) it is not: a caller reporting an “after” utilization/spread must evaluate `final_assignment`, not `heuristic.HeuristicResult.assignment`, or it reports numbers for moves that were never actually scheduled (REVIEW.md R-02 — `cli.py`’s plan originally did exactly this, correct only when `order_moves()` scheduled every move a plan proposed).

What this pass deliberately does not do

- **Reasoning about concurrency at scheduling time.** This module always produces one, strictly-sequential *order* — `execute.py`’s own executor is what actually runs several of that order’s moves at once under `execution.max_concurrent_migrations > 1`, re-validating section 8.1’s generalized invariant live against whatever really ends up in flight together (92-`execute.md`’s “Concurrent execution” section). `payback.py`’s move-duration estimates would let this module choose a smarter, concurrency-aware *order* (e.g. front-loading moves that can provably run together) instead of leaving that entirely to `execute.py`’s own strict-FIFO launch discipline, but that scheduling -time optimization is not implemented.
- **Ordering priority 2** (“moves that free space a later move needs”) and **staging** (section 8.3 option 1). Both are refinements over a plan that is already feasible and safe without them — priority 2 only affects how well a plan front-loads its value if interrupted, and staging only helps in a genuine pebble-motion deadlock. Omitting them can only make this module report a deadlock in a case a more sophisticated scheduler would have found a way through; it can never produce an order this module thinks is safe but is not.
- **Cooldowns**, for the same reason `gates.py` doesn’t implement them yet: `state.json` (section 11.2), which would record the timestamps a real cooldown check needs, does not exist.
- **The section 7.3 saturation check** — belongs with `payback.py` (implemented; see 96-`payback.md` for why the check itself is deferred).

Wired into plan, not yet into apply

`cli.py`’s plan command calls `heuristic.run_heuristic()` then `schedule.order_moves()` for every group the gate says to act on, and renders the result (see `docs/manual/27-plan.md`). Nothing calls `schedule.py` from apply yet, because apply itself does not exist (`execute.py`, phase 7+) — plan only ever computes and prints.

Migration cost and the payback test

What does this page answer? How does `payback.py` turn a scheduled move into a cost, a plan into a benefit, and the two into an accept/reject verdict — and why does a reserve-fixing plan always pass? Describes `proxmox_storage_drs/payback.py`.

Cost and benefit are computed from data other modules already have

`compute_move_cost()` needs only a `schedule.ScheduledMove` and its source topology. Storage — no new fetch, no new state. `duration_mirror` is `z_d / migration.bwlimit_bytes_per_sec`; `duration_wipe` is `z_d / saferemove_throughput` when `migration.account_saferemove_wipe` and the source has `saferemove` on, else zero. `cost_load_seconds` — in the same **load-seconds** unit (average in-flight I/O requests multiplied by seconds) `compute_benefit_load_seconds()` produces below, which is what makes the two comparable at all — is `duration_mirror * (migration.source_load_weight + migration.target_load_weight) + duration_wipe * migration.wipe_load_weight`: the extra in-flight I/O the migration itself imposes on the source and the target while the mirror runs, plus, for as long as the old volume takes to be zeroed, the extra load a running `saferemove` wipe imposes on the source alone. It is zero below `migration.tiny_disk_bytes` (section 5.4/7.1) — `duration_mirror/duration_wipe` are still the real numbers, so `exceeds_max_duration/saturation_deferred` still fire normally for a tiny disk that happens to be throttled hard enough; only the economic charge is waived.

`compute_benefit_load_seconds()` implements section 7.2's $\text{benefit} = (\alpha_{\text{spread}} * (E_{\text{before}} - E_{\text{after}}) + \Delta_{\text{capacity_spread}} * (F_{\text{before}} - F_{\text{after}}) + \kappa_{\text{vm_affinity}} * (A_{\text{before}} - A_{\text{after}})) * H$ (sections 12 and, for the third term, 5.4/7.2's affinity-payback fix). H is `migration.payback_horizon_seconds` (default 365d, converted to seconds) — the length of time the plan's improvement is assumed to keep paying off, which is why both sides of the payback ratio end up in the same **load-seconds** unit: benefit is a load-shaped quantity held for H seconds, and cost (below) is a duration in seconds multiplied by a load-shaped weight. `compute_benefit_load_seconds()` itself takes the pre-plan/post-plan pair of `heuristic.raw_spread()` values (E , the *raw*, unweighted imbalance quantity), `heuristic.raw_capacity_spread()` values (F , the raw data-spread quantity, section 5.3 (C7)) and `heuristic.raw_affinity_debt()` values (A , the raw — w_v -weighted but not κ -scaled — affinity debt, section 5.4), plus `objective.alpha_spread/delta_capacity_spread/ kappa_vm_affinity` as explicit weight parameters, and multiplies the weighted sum by `migration.payback_horizon_seconds`. **Never** pass `ObjectiveBreakdown.imbalance_term/.capacity_spread_term/ .fragmentation_term`: those are the same quantities already scaled by `alpha_spread/delta_capacity_spread/kappa_vm_affinity` for the section 5.4 *solver* objective, and passing them here would double-apply the weight (REVIEW.md R-01 — earlier code passed `imbalance_term` directly; only invisible while `alpha_spread`'s default of 1.0 made the two numerically identical). Applying the weights *inside* `compute_benefit_load_seconds()`, as explicit parameters rather than baked into an already-scaled input, is what keeps R-01's principle intact — now extended to all three terms, not just the first two. `cli.py`'s plan handler computes all six raw values for the pre-plan and post-plan assignments and passes them, and all three weights, through.

The κ term's own inclusion here is itself the fix for a real live failure: a plan whose entire value was reuniting VMs (two 528 KiB `efidisk0s` among its four moves) scored benefit 9 load·s vs cost 232 load·s → ratio 0.0388 and was refused, because the pre-fix formula had no way to value affinity at all — see `IMPLEMENTATION_PLAN.md` section 7.2's own account, and `tests/fixtures/affinity-repair.yaml` (section 14.7) for the fixture built to reproduce it. $A_{\text{before}} - A_{\text{after}}$ can be negative — a balance move that *splits* a VM must pay for the fragmentation out of its α gain — and `compute_benefit_load_seconds()` returns that negative contribution as computed, never clamped; `evaluate_plan_payback()`'s own acceptance test already rejects a genuinely negative benefit correctly.

headroom_src/headroom_dst: a plan formula this project cannot fill in

Section 7.1 writes $\text{duration_mirror_d} = z_d / \min(\text{bwlimit}, \text{headroom_src}, \text{headroom_dst})$. Neither `headroom_src` nor `headroom_dst` is defined anywhere else in `IMPLEMENTATION_PLAN.md`, and no config field or topology.Storage attribute represents a per-storage effective throughput ceiling distinct from the one global `migration.bwlimit_bytes_per_sec`. `compute_move_cost()` uses $z_d / \text{bwlimit}$ only — exactly what the formula reduces to whenever neither storage's own throughput is the binding constraint, which is the case `bwlimit` exists to enforce in the first place. This is recorded as a known simplification of the plan's own underspecified formula, not silently worked around.

The reserve-override exemption

`evaluate_plan_payback()`'s aggregate ratio test is $\text{benefit_load_seconds} \geq \text{migration.payback_ratio} * \text{total_cost_load_seconds}$ (default `payback_ratio`: 10.0 — a plan's benefit must be worth at least ten times what executing it costs, both sides in the same load-seconds unit `compute_benefit_load_seconds()` produces above). `PaybackResult.ratio` is that same benefit / cost division exposed for reporting; the test itself never divides, to stay well-defined when `total_cost_load_seconds` is 0.

Section 7's payback test weighs a move's cost against the *balance* benefit it buys. That framing has an edge it does not name: a plan resolving an active (C4)/(C5) violation, or section 5.3.1's configured free-space requirement, is not optional the way a balance-driven move is, and its "benefit" under the section 7.2 formula can easily be zero or even structurally zero — moving the only loaded disk in a

two-storage group between the two storages changes which one carries it without changing E at all, no matter how urgently the move is needed for capacity reasons. Rejecting that move on economic grounds would contradict section 13's own "the reserve is never traded against balance," which `gates.py` already treats as absolute for the drift/imbalance gates. `evaluate_plan_payback()` applies the identical rule, but as an **outcome** trigger, not a per-move flag: it takes two extra parameters, `current_shortfall_bytes` and `final_shortfall_bytes` — $\sum r_s$ (`reserve.total_shortfall_bytes()`) on the group's current assignment and on the plan's *executed* endpoint respectively — and `repair_exempt = final_shortfall_bytes < current_shortfall_bytes` always passes the aggregate ratio test when true, regardless of the computed ratio. The hard per-move `max_single_move_duration` rule is not exempted this way — section 7.3 lists it as applying "regardless of the aggregate test" precisely because it is an operational limit (a mirror that takes that long has other costs an economic ratio does not capture), not an economic one.

This replaced a per-move flag (`ScheduledMove.resolves_reserve_violation`, still there but narrowed to section 8.2's own priority-1 scheduling signal — "this move's source was violating when scheduled first" — never read by `payback` anymore). The flag fired whenever *any* move's source was violating at scheduling time, which is provably a superset of the outcome trigger ($\sum r_s$ can only fall if some storage's `used/Z_s` falls, which needs a disk to leave a storage that was therefore violating when it left — so every outcome-exempt plan was already flag-exempt, never the reverse): a plan that moves a disk off a violating storage but leaves the group no less short is flag-exempt but not outcome-exempt. The two extra parameters are computed once, in `cli.py`'s `_plan_group()` — the function's sole production caller — from objects it already has in hand: `current_shortfall_bytes` is `reserve.total_shortfall_bytes()` over the current assignment (group.disks' own `current_storage`); `final_shortfall_bytes` is the same sum over `payback.executed_assignment()`'s result, `schedule_result.final_assignment` with every disk a hard per-move rule excluded (`exceeds_max_duration` or `saturation_deferred`, both already known from `move_costs` by then) held back at its current storage — "what the plan will really run," not merely what got scheduled, so a repair a hard rule then blocks is correctly *not* exempt.

The revert test, `payback.repair_markers()`, is the per-move report a caller shows an operator ("which move carried the repair"): a move is `repair: true` iff holding its own disk back on its current storage — against that same `executed_assignment()` result — would strictly raise $\sum r_s$. This is deliberately a *different* question from the trigger: the trigger asks whether the plan as a whole reduced $\sum r_s$; the marker asks, move by move, which ones the plan's own repair actually depends on. The two can disagree in either direction — a redundant-repair plan (two moves off a violating storage, either alone sufficient) repairs with no move individually marked, since holding *either one back alone* still leaves the other to do the job — and the marker also catches an *indirect* repair: a move whose own source never violated anything, but which empties the destination another repair needs (section 14.8's `free-space-repair.yaml`, `roomy`'s own move, is exactly this case). Both `executed_assignment()` and `repair_markers()` take the same Group and assignment shape `reserve.py`'s functions already use, so the revert test is two extra `total_shortfall_bytes()` calls per move, not a second implementation of (C5)'s arithmetic.

`test_plan_json_output` and `test_plan_json_output_accepts_payback_when_saferemove_is_off` in `tests/unit/test_cli.py` exercise both halves of this end to end: the same reserve-driven, zero-benefit move is accepted when its duration is within the limit and rejected (correctly, via the hard rule, not the economic one) when a slow `saferemove` wipe pushes it over `max_single_move_duration` — and, because that hard rule then excludes the move from `executed_assignment()`, the plan is *not* `repair_exempt` either in that second case, even though the plan's aspirational target would have repaired the violation.

The section 7.3 saturation guard: `compute_move_cost()`'s optional target

The guard defers a move (never rejects it outright) when the load it would add to either endpoint, forecast over the mirror, would push that storage past an operator-declared ceiling: a move is deferred whenever `L_during(s) > saturation_ceiling * N_s` for either endpoint s . N_s is that storage's own

`storages[].saturation_load` — an operator-supplied number, in the same average-in-flight-I/O-requests unit every other load figure in this codebase uses, above which the operator judges the storage should not run for a sustained period; it is optional, and a storage that never sets it is never checked at all (below). `saturation_ceiling` is `migration.saturation_ceiling` (default 0.85), the fraction of `N_s` a move's own forecast load is allowed to reach. $L_{\text{during}}(s) = L_{\text{hat}}(s, \text{duration_mirror}) + \omega_{\text{role}}(s) : L_{\text{hat}}(s, \text{duration_mirror})$ is the forecast upper bound of `s`'s own load over the move's mirror duration — `forecast.storage_upper_bound()`, section 10.1, summed over the disks *currently* resident on that storage, not the moving disk's own hypothetical arrival, since during mirroring it is still served from `src` — and $\omega_{\text{role}}(s)$ is the same per-role load charge the cost formula itself uses for a mirroring move, `migration.source_load_weight( $\omega_{\text{src}}$ )` when `s` is the source or `migration.target_load_weight( $\omega_{\text{dst}}$ )` when `s` is the target (never both on the same endpoint).

`compute_move_cost()` stays pure (the module docstring's own promise: “nothing fetches anything”) by taking the guard's inputs already computed, rather than fetching a forecast itself: `target`, and `l_hat_src/l_hat_dst` — the caller's own already-computed `L_hat(s, duration_mirror)` for each endpoint, per the formula above (its ω_{dst} charge below already accounts for the moving disk's own mirror-write traffic to `dst` separately, so `l_hat_dst` itself must never double-count it). Left at their defaults (`target=None`, both 0.0) the check is simply inactive — every call site written before this existed, and `cli.py`'s own dry-run/plan paths that have not been updated to compute a forecast, keep working unchanged.

`mirror_duration_seconds()` is `compute_move_cost()`'s own `duration_mirror_d` arithmetic, factored out so a caller can learn it *before* calling `compute_move_cost()` — a genuine ordering dependency: the guard's forecast horizon is this move's own mirror duration, but `compute_move_cost()` is also what turns that forecast into the `saturation_deferred` verdict. A caller wanting the check active must: call `mirror_duration_seconds()`, use it as the horizon for `forecast.storage_upper_bound()` against each endpoint's own resident disks, then call `compute_move_cost()` with the results.

`_saturation_deferred()` is where $L_{\text{during}}(s) > \text{saturation_ceiling} * N_s$ above is actually evaluated per endpoint — the same two $\omega_{\text{src}}/\omega_{\text{dst}}$ config values `compute_move_cost()`'s own cost formula already uses, not a second pair invented for this check (AGENTS.md section 5). `MoveCost.saturation_deferred/PaybackResult.deferred_moves` mirror exceeds_max_duration/rejected_moves's existing shape exactly, but are kept as distinct fields — section 7.3 itself draws the same distinction (“reject the move” vs. “defer the move to a later run”), and a report should be able to say which of the two happened, one hard and always active, the other best-effort and silently inactive wherever `saturation_load` is unset. `PaybackResult.accepted` now requires neither being non-empty.

What this pass deliberately does not do

- **The 3-retry re-solve-with-doubled-beta/gamma loop** (section 7.3) on aggregate payback failure. A real UX refinement — it converges on the smaller subset of high-value moves rather than abandoning the run — but not a correctness requirement: reporting accept/reject plainly already satisfies “reject a plan whose cost outweighs its benefit.” Implementing the loop means re-invoking `heuristic.py/schedule.py` with adjusted weights from inside `cli.py`'s own orchestration, which is where it belongs when it is built, not half-done inside this module now.
- **Automatically dropping an individually-rejected move** and re-solving without it. Reported instead (`MoveCost.exceeds_max_duration`, `PaybackResult.rejected_moves`) — the same “report, never force” choice `schedule.py`'s deadlock reporting already makes for an unschedulable move.
- **The section 7.3 saturation-ceiling defer check beyond its mirroring -phase reading.** `compute_move_cost()` now implements $L_{\text{during}}(s) \leq \text{saturation_ceiling} * N_s$ for each endpoint (see the section above), but only at the mirroring-phase horizon the section's own header names (“push either endpoint above ... during *the mirror*”) — not a second, separate check for the *draining* phase (ω_{wipe} — `migration.wipe_load_weight`, the same weight the cost formula above charges for a

running wipe — over `duration_wipe_seconds`), which the full generalized in-flight-set model implies but which needs `schedule.py` to reason about overlapping moves, something it does not do (`95-schedule.md`). `N_s` unset on a storage still skips the check for that endpoint entirely, exactly as the plan’s own words allow (“fully supported... loses only this one advisory check”).

Wired into plan and apply alike, via `_plan_group()`

`cli.py`’s `_plan_group()` (`docs/internals/92-execute.md`’s “one planning pipeline, shared by plan and apply”) computes a `PaybackResult` for every group the gate acts on, right after scheduling its moves, and both plan’s render functions and apply’s payback gate (`_apply_payback_gate()`) consume it — a rejected or deferred move never reaches `execute.py` regardless of mode. `_saturation_forecast_inputs()` is what builds the saturation guard’s own forecaster and per-disk load history, once per group, only when at least one of its storages configures `saturation_load` — skipped entirely otherwise, so a cluster that never sets it pays no Prometheus cost for a check it cannot use. `_compute_one_move_cost()` then derives each move’s own `l_hat_src/l_hat_dst` (`forecast.storage_upper_bound()` over each endpoint’s *currently* resident disks, at that move’s own mirror duration) before calling `compute_move_cost()`. `compute_wipe_duration_seconds()` is also used by `verify-storages`’s existing wipe-time warning — one implementation of the formula, not two (`AGENTS.md` section 5).

`anonymize.py`, `collect.py`, `replay.py` — diagnostic bundles

`IMPLEMENTATION_PLAN.md` section 16 is the specification. This page is about the shape of the implementation, for whoever changes it next.

Why three modules, not one

- **`anonymize.py` is pure.** No I/O beyond two small salt-file helpers, no dependency on `pve.py/metrics.py`. It is the allowlist and the pseudonym function, nothing else, and it is imported by both `collect.py` (to write a bundle) and `tests/corpus/validate_corpus.py` (to check one) — the “single implementation” `AGENTS.md` section 5 asks for, so the audit cannot silently drift from what the collector permits.
- **`collect.py` is the one thing that talks to the real cluster** for this feature. It never imports `replay.py`.
- **`replay.py` reads a bundle back.** It imports `collect.py` only for two tiny pure functions (`hash_query_text/hash_label_name`) — the cache-key scheme has to agree exactly between the module that writes a bundle and the one that reads it back, so it lives in one place, written first.

Capture: recording clients, not a second code path

`capture_bundle()` does **not** re-implement what `verify_metrics()` or `build_topology()` do. It wraps the real `PveClient/PrometheusClient` in `RecordingPveClient/RecordingPrometheusClient`, hands those to the exact same functions a live `plan/show-load/verify-metrics` run already calls, and lets every response pass through on its way back to the caller.

- `RecordingPveClient` overrides each of `pve.py`’s twelve read methods individually: catches `PveApiError`, records the outcome (`ok/http_error/empty`) in the manifest’s `calls` list, and returns a safe empty default (`{}/[]`) on failure rather than raising — section 16.2’s “failures are recorded, never rendered as absence” applied at the transport boundary, so `build_topology()` itself never needs to know it is running under capture.
- `RecordingPrometheusClient` overrides only `_get()` (the one funnel every public method routes through) and, unlike the PVE side, **does not** swallow a failure — it records and re-raises, matching what the real client does, because `verify_metrics()`’s own error handling already covers that case correctly. It additionally accumulates every (`path`, `params`, `response`) triple it has ever served, in call order.

That accumulator is the reason `collect.py`'s own driver functions (`_drive_group_series()`, `_drive_label_values()`) look thin: they only need to *issue* the calls this bundle's own §16.2 requires. Once `verify_metrics()` and both drivers have run against one `RecordingPrometheusClient`, `_anonymize_captured_prometheus()` walks the whole accumulator exactly once and turns it into bundle files — which is what makes capture complete by construction rather than by having correctly guessed every internal query `metrics.py` might issue (the bare per-metric sample check, the coverage probe, the observed-spacing probe, `label_values("__name__")` — none of which `collect.py`'s own driver functions construct by hand).

Range queries: chunk, then stitch by query text

A capture range can span days; a single `query_range` call over it can hit a backend's own `max_samples` limit. `_issue_range_chunks()` issues one `range_query()` call per day-sized chunk and does **nothing else** — no stitching happens at issue time. `_anonymize_captured_prometheus()` groups every accumulated `query_range` call by its *query text* and merges all of them — however many chunks, plus `verify_metrics()`'s own single, differently-windowed spacing probe if its text happens to coincide — into one (`start`, `end`, `step`, `result`) via `_stitch_range_captures()`: widest start/end span, one series per disk, every point from every chunk deduplicated by timestamp. One file, one query text, regardless of how many separate HTTP calls produced its content.

now has to be controllable, all the way down

Section 16.1's determinism promise ("two captures of an unchanged cluster produce byte-identical bundles") only holds if every query genuinely depends on `capture_bundle()`'s own `now` parameter and nothing captures `time.time()` on its own. Two functions did, originally, independent of this feature: `metrics.compute_disk_coverage()` and `metrics._check_observed_spacing()`. Both gained an optional `now: float | None = None` (default: the real wall clock, so every *live* caller is unaffected) that `collect.py` — and, since the same problem applies to a replayed `plan/show-load/explain/verify-metrics` run, `cli.py`'s `_now_for()` — now pass explicitly. `loadmodel.compute_group_load()` picked up the same optional `now`, forwarded to `compute_disk_coverage()`, for the same reason.

Node selector: anonymized names, captured verbatim

The node-scoping selector (`resolve_node_selector()`, section 3.4) is built into the *query text* this module captures and writes into a bundle file's "query" field. If it were built from the real node list, two things would go wrong at once: the real node names would leak into the bundle, and a later --replay run — which reconstructs the identical selector from `ReplayPveClient.node_names()`, itself pseudonymized — would build different query text and never match. `_capture_prometheus_files()` maps known nodes through `mapper.node()` *before* handing them to `resolve_node_selector()`, so the captured text and the replayed text are built from the same (anonymized) inputs by construction.

Anonymization: allowlist, then pseudonym, then timestamp

`anonymize.py`'s `Mapper` is a plain dataclass holding the salt plus one piece of real state: `_vmid_map`, because a `vmid` pseudonym cannot be a pure function of the salt and the `vmid` alone the way a node or storage pseudonym is. `Mapper.vmid()`'s own hash lands in a fixed range (100..999999); when two different real `vmids` hash to the same candidate, the second one keeps stepping to `candidate + 1` (wrapping back to the low end of the range) until it finds a value nothing has claimed yet — linear probing, the same collision-resolution technique an open-addressing hash table uses. That has to be *order-independent* — the same original `vmid` must map to the same pseudonym regardless of which order a caller happens to visit VMs in. `register_vmid()` is the only place that matters: it sorts every not-yet-seen `vmid` before assigning, once, up front. Every other `Mapper` method (`node()`, `storage()`, `tag()`, ...) is a pure function of the salt and its own argument and needs no registration at all.

Fail closed, everywhere. A method that cannot resolve its input to something already known — a storage

outside every group, a UPID naming a node that has left the cluster, a vmid never registered — returns `None`. Every caller in `collect.py` treats `None` as “drop this record”, never as “pass the original through” and never as “invent a placeholder”. This is what section 16.3 means by “unmapped means dropped”: the failure mode of a smaller bundle is preferred over the failure mode of a leak, unconditionally.

Free text gets a second look. A handful of `verify_metrics()` finding messages embed real values directly in human-readable prose (a sample series’ raw label dict, a coverage gap’s `vmid:device`) — the allowlist alone does not catch this, since it operates on structured keys, not on substrings of a message string. `collect._redact_finding_message()` is a deliberately broad regex substitution — any whole number matching an already-registered real vmid, any occurrence of a real node name — applied to every finding message, not just the ones known to need it. Over-redaction (rewriting a coincidentally vmid-shaped number that was not actually a vmid) is the safe direction to be wrong in here.

Replay: real subclasses, not lookalikes

`ReplayPveClient(PveClient)` and `ReplayPrometheusClient(PrometheusClient)` are genuine subclasses — `super().__init__()` with an inert transport (`api=None`, a session whose `get()` raises) — specifically so that every existing `client: PveClient/client: PrometheusClient` type annotation in `topology.py`, `metrics.verify_metrics()`, `loadmodel.py` and `cli.py` accepts one with zero changes anywhere else. `ReplayPveClient` overrides each of the eleven read methods directly, keyed by the bundle’s own directory layout (vmid-only for VM files — the bundle does not key by node, since `topology.py` always already knows the right node before calling). Neither `move_disk` nor `task_status` can be reached in practice (apply is refused before any handler gets this far), but both raise `PveApiError` anyway — defense in depth costs one raise each.

`vm_pending()` is the one method here that does not raise `BundleError` on a missing file: a bundle captured before section 3.8 added the call has no `pve/vm-pending/` directory at all, and a missing file there is read as “nothing pending” rather than a corrupt bundle. A bundle captured since always writes the file for every considered VM (`collect.py`’s `_capture_pve_vm_files()`), even when the list is empty, so this fallback can only trigger on a bundle that genuinely predates the feature — it is not a general “missing means empty” rule for this class.

`ReplayPrometheusClient` overrides only `_get()`. Its `/api/v1/query` branch is an exact match on the (anonymized) query text — an instant query carries no separate time parameter, so there is nothing else to key on. Its `/api/v1/query_range` branch is *not* an exact match on the literal requested start/end: see “Range queries” above for why a bundle stores one superset span per query text, and `replay.py`’s own module docstring for the trimming logic that serves a narrower request (a different forecaster’s own, smaller `required_range()`) out of it. A request outside the stored span, or at a different step, is a `BundleError` naming the query and both ranges — never a silent empty result.

`bundle_reference_now()` returns the bundle’s own rebased capture instant from `manifest.json`’s `capture.synthetic_now_epoch` — the same value `cli._now_for()` substitutes for `datetime.now(timezone.utc)` everywhere a handler builds a query time window under --replay. The manifest never carries the real capture date (section 16.3): what is stored is now already shifted by the same whole-week offset every other timestamp in the bundle goes through, so only hour-of-day and day-of-week survive.